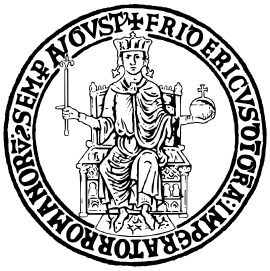
Università degli Studi di Napoli Federico II



Scuola Politecnica e delle Scienze di Base

Dipartimento di Ingegneria Elettrica e Tecnologie dell'Informazione

Corso di Laurea in Informatica

Semantic Pruning di metadati per l'imputazione di valori mancanti: analisi, metodologia e valutazione su RENUVER

Relatore
Prof. Gennaro Ventrone

Candidato
Gennaro Ventrone
N86004074

Anno Accademico 2025–2026

Sommario

L'imputazione di valori mancanti basata su *Relaxed Functional Dependencies* (RFD) affronta il problema dell'incompletezza dei dati sfruttando regole estratte dal dataset stesso. Il costo computazionale di questi approcci cresce con il numero di regole disponibili, e una parte di esse è ridondante: da qui l'interesse per il *semantic pruning*, la selezione delle RFD effettivamente rilevanti.

Questo lavoro studia il pruning semantico di un insieme di metadati applicato a Renuver, un algoritmo di imputazione basato su RFD. Dall'analisi del bytecode si derivano tre condizioni esatte: [V1], che caratterizza le regole che possono esercitare un veto sulle imputazioni; [V2], che individua una classe di regole la cui potatura è dimostrabilmente priva di effetti; [V3], che isola le regole provabilmente inerti. Da queste discende un teorema negativo: con potatura esterna, nessuna riduzione non banale può essere contemporaneamente sicura rispetto alla generazione dei candidati e al potere di veto.

La condizione [V2] è stata verificata empiricamente su 1 242 321 terne con zero discrepanze, e la sua applicazione produce un output identico a quello del sistema originale su 113 configurazioni su 113, su quattro famiglie di dataset (111 delle tre famiglie principali e 2 di una famiglia indipendente). Il teorema separa ciò che è dimostrato — la caratterizzazione delle rimozioni certificabili — da ciò che è misurato, e mostra che la riduzione garantita non supera il 20,9 % sui dati esaminati. Un'euristica di potatura aggressiva, denominata `A5b`, riduce le regole dell'84,2 % su Bridges migliorando al contempo il risultato di 69 imputazioni corrette e 72 errori in meno; sugli altri dataset la stessa strategia danneggia il risultato. Il lavoro identifica il meccanismo che governa questa differenza — il guadagno è il prodotto della quota di imputazioni ottenute per via esatta e del numero di imputazioni effettuate — e ne ricava un criterio diagnostico misurabile prima di potare. Su Glass e Restaurant una taratura più conservativa della medesima strategia non perde che una imputazione corretta su 97 configurazioni. Il lavoro mostra infine che la riduzione delle regole non implica un risparmio di tempo: la strategia che ne rimuove di più è anche quella che rallenta di più, e il tempo dipende dalla composizione dell'insieme potato più che dalla sua dimensione.

Indice

Introduzione **1**

Contesto e motivazioni 1
Problema affrontato 1
Domanda di ricerca 2
Obiettivi del lavoro 2
Contributi della tesi 3
Struttura dell'elaborato 4

1 Background e stato dell'arte **7**

1.1 Qualità dei dati e sue dimensioni 7
1.2 Valori mancanti 8
1.3 Data imputation 9
1.4 Relaxed Functional Dependencies 10
1.5 Ridondanza e riduzione di un insieme di regole 10
1.6 Misure di similarità e distanza 11
1.7 Semantic pruning: definizione operativa e collocazione 11

2 Analisi del sistema Renuver **13**

2.1 Obiettivi del sistema 13
2.2 Architettura generale 13
2.3 Flusso di imputazione e procedure di riempimento 14
2.4 Dataset impiegati 15
2.5 Il doppio ruolo delle RFD 15
2.6 Costo computazionale 17

3 Metodologia di semantic pruning **19**

3.1 Motivazioni 19
3.2 Dal sistema alle condizioni esatte 20
3.3 Lo spazio delle strategie 22
3.4 Le strategie di potatura 23
3.5 Perché la generazione non ammette una condizione esatta della stessa natura 25
3.6 Integrazione con il sistema: il contratto di potatura 26

4 Teoria: che cosa si può potare con garanzia **27**
4.1 Impostazione formale . . . 27
4.2 Un esempio minimo . . . 28
4.3 Il teorema negativo . . . 29
4.4 Dimostrazione . . . 29
4.5 Conseguenza sul target di riduzione . . . 30
4.6 Quanto è stretto il limite . . . 30
4.7 Portata del risultato . . . 31

5 Ipotesi e criteri di falsificazione **33**
5.1 Come sono nate le ipotesi . . . 33
5.2 Ipotesi primarie . . . 34
5.3 Domande esplorative . . . 36

6 Valutazione sperimentale **39**
6.1 Protocollo di verifica . . . 39
6.2 Metodologia statistica . . . 40
6.3 Setup sperimentale . . . 42
6.4 Dataset e configurazioni . . . 42
6.5 Metriche . . . 43
6.6 Termine di paragone banale . . . 44
6.7 Esperimenti condotti . . . 44
6.8 Studio di ablazione . . . 45

7 Analisi dei risultati **47**
7.1 La strategia che non perde nulla . . . 47
7.2 La riduzione classica, e perché non è sicura . . . 48
7.3 La strategia che produce il guadagno . . . 49
7.4 Il confine: dove il guadagno non si realizza . . . 49
7.5 Il meccanismo . . . 50
7.6 Il criterio diagnostico . . . 52
7.7 La prova sulla quarta famiglia . . . 54
7.8 Risultati negativi . . . 55
7.9 Il costo: perché ridurre non basta . . . 57

8 Discussione **59**
8.1 Esiti delle ipotesi . . . 59
8.2 Limiti dell'evidenza . . . 61
8.3 Collocazione e verifiche ulteriori . . . 62

9 Difetti e criticità del sistema **65**
9.1 Classificazione dei difetti . . . 65
9.2 Difetti del sistema studiato . . . 67
9.2.1 D1 — Ramo irraggiungibile nel calcolo della similarità . . . 67
9.2.2 D2 — Disallineamento di schema nei dataset Glass . . . 67
9.2.3 D3 — Righe malformate e indici non ricostruibili nei dataset Physician . . . 68

9.2.4 D4 — Eseguibile di build con istruzioni di debug 69
9.3 Errori rilevati e corretti nel codice sviluppato 69
9.3.1 D5 — Computo errato del budget nella politica sui veti 70
9.3.2 D6 — Indice di rischio anti-correlato 70
9.4 Che cosa insegnano questi difetti 71

Conclusioni e sviluppi futuri **73**
Sintesi del lavoro 73
Possibili sviluppi 73

Appendice A Riproducibilità dei risultati **77**
A.1 Struttura del materiale 77
A.2 Verifica automatica dei risultati 77
A.3 Verifica di equivalenza della ricompilazione 78
A.4 Comandi principali 79
A.5 Ambiente di esecuzione 79

Appendice B Tabelle complete dei risultati **81**
B.1 Campagna principale su Bridges 81
B.2 Studio di ablazione 84
B.3 Taratura conservativa 84

Bibliografia **87**

Introduzione

Contesto e motivazioni

La qualità dei dati è un requisito operativo prima che teorico: un'analisi condotta su informazioni incomplete produce conclusioni che nessun metodo statistico può recuperare. Quando un attributo di una tupla è privo di valore, la scelta non è fra "imputare" e "non imputare", ma fra imputare in modo informato e scartare il dato. È questa la ragione dell'interesse per la *data imputation*: ricostruire il valore mancante a partire dal resto dell'informazione disponibile.

Tra gli approcci recenti, quelli basati su *Relaxed Functional Dependencies* (RFD) hanno una proprietà che li distingue: le regole di imputazione non vengono fornite dall'esterno, ma estratte dal dataset stesso. Una RFD esprime un vincolo di dipendenza approssimato fra attributi — *se due tuple concordano entro certe soglie sugli attributi di sinistra, allora concordano entro una soglia sull'attributo di destra* — e può quindi essere usata sia per generare candidati all'imputazione sia per rifiutare quelli incoerenti. Il sistema Renuver [1] è un rappresentante di questa famiglia.

Il prezzo di questa ricchezza è computazionale: la fase di estrazione produce tipicamente migliaia di regole, e il costo dell'imputazione cresce con il loro numero. Molte di esse, però, non influenzano il risultato. Da qui il problema affrontato in questo lavoro: *si possono selezionare le RFD rilevanti senza alterare il comportamento del sistema?* È la domanda del *semantic pruning* applicato a un insieme di metadati.

Problema affrontato

Il sistema Renuver riceve in ingresso, oltre al dataset, un file di RFD. L'ipotesi di partenza — suggerita dalla struttura del sistema — è che una parte di quel file sia ridondante e che rimuoverla riduca il tempo di esecuzione senza degradare la qualità delle imputazioni.

Il problema si rivela però più sottile di così, per una ragione che questo lavoro porta alla luce in modo formale: nel sistema studiato le RFD svolgono *due ruoli distinti*, e i due ruoli impongono requisiti opposti. Una RFD serve a *generare* candidati all'imputazione, ma serve anche a *vetare* imputazioni che violerebbero il vincolo. Rimuovere una regola inutile per il primo ruolo non garantisce che essa sia inutile per il secondo, e nulla nel formato del file distingue i due casi.

Da questa osservazione discendono le due domande che strutturano la tesi:

D1. Che cosa si può potare con garanzia? Esiste una classe di regole la cui rimozione è dimostrabilmente priva di conseguenze, e quanto è grande?

D2. Che cosa si guadagna quando la garanzia si abbandona? Se si rimuovono regole anche a costo di alterare il comportamento, quali sono i guadagni e a quali condizioni?

Domanda di ricerca

Il lavoro muove da una domanda formulata in termini verificabili. Dato l'insieme di RFD che il sistema riceve in ingresso, *è possibile ridurne il numero almeno della metà mantenendo l'accuratezza e la copertura entro cinque punti percentuali dalla versione originale, e riducendo il tempo di esecuzione?* La domanda ha tre esiti distinti — la riduzione, la qualità delle imputazioni e il costo — e la risposta del lavoro non è uniforme sui tre.

Sul primo esito la risposta è *negativa*, e non per insufficienza delle strategie: il capitolo 4 dimostra che una potatura che operi sul solo file delle regole e non rinunci a nessuna garanzia non può superare, sui dataset esaminati, il 20,9 % di riduzione. Sul secondo esito la risposta è *condizionata*: la potatura può migliorare il risultato — su Bridges lo migliora in modo statisticamente significativo — ma soltanto quando la composizione delle imputazioni lascia spazio al miglioramento, e sugli altri dataset danneggia. Sul terzo esito la risposta è *negativa nel caso generale e positiva in un caso*: la riduzione delle regole non implica un risparmio di tempo, e la strategia che rimuove di più è quella che rallenta di più. Il contributo del lavoro è di stabilire questi tre confini e di fornire il criterio con cui riconoscere, su un dataset nuovo, in quale dei tre regimi ci si trovi.

La Tabella 1 anticipa la forma delle tre risposte, con il rimando alle sezioni in cui ciascuna è argomentata: serve a rendere esplicito fin dall'inizio che cosa il lavoro sosterrà e che cosa non sosterrà.

Obiettivi del lavoro

Gli obiettivi, in ordine di priorità:

- **Caratterizzare il sistema.** Derivare dal comportamento effettivo di Renuver — non dalla sua documentazione — le condizioni sotto le quali una RFD può influenzare un'imputazione, e tradurle in criteri di potatura verificabili.
- **Definire una metodologia di pruning.** Progettare e implementare strategie di selezione, distinguendo esplicitamente quelle che offrono una garanzia da quelle euristiche.
- **Valutare l'impatto.** Misurare l'effetto della potatura sulle prestazioni dell'imputazione — accuratezza, copertura, precisione e tempi — con un protocollo appaiato e test statistici, su più famiglie di dataset.

Tabella 1: Risposta alla domanda di ricerca, per esito. La tabella anticipa la struttura dell'argomentazione; le grandezze sono discusse nelle sezioni indicate.

esito	risposta	dove
riduzione ≥ 50 % *con garanzia*	**impossibile**: la classe di regole la cui rimozione è certificabile non supera il 20,9 % sui dataset esaminati, e il limite è del criterio, non dell'implementazione	capitolo 4, sezione 4.6
accuratezza e copertura entro cinque punti	**condizionata**: il risultato migliora dove la composizione delle imputazioni ha spazio per migliorare e la copertura non cala, e peggiora dove una delle due condizioni manca	sezione 7.3, sezione 7.4
riduzione del tempo di esecuzione	**non garantita**: il tempo dipende dalla composizione dell'insieme potato, non dalla sua dimensione; il caso migliore accelera di 11 volte, il peggiore rallenta di 251	sezione 7.9

Il criterio diagnostico della sezione 7.6 rende operativa la seconda risposta: decide a costo nullo se la leva del miglioramento esiste, e impone di misurare la copertura prima di adottare la potatura.

- **Spiegare i risultati.** Non limitarsi a misurare: individuare il meccanismo che determina quando la potatura conviene e quando danneggia.

Contributi della tesi

I contributi originali di questo lavoro sono i seguenti.

1. **Tre condizioni esatte derivate dal bytecode** [V1], [V2], [V3], che classificano le RFD in base al loro ruolo effettivo nel sistema. [V2] è la più rilevante: identifica le regole il cui potere di veto è implicato da un'altra regola sopravvissuta, e la loro rimozione è quindi priva di effetti.

2. **Un teorema negativo** (capitolo 4) che delimita il campo: se la potatura opera sul solo file delle regole e ogni rimozione è certificata con l'informazione ivi disponibile, allora le regole rimovibili sono quelle inerti oppure quelle il cui veto è implicato da una regola sopravvissuta. Il risultato è quantificato: la classe potabile con garanzia non supera il 20,9 % sui dataset studiati, e si dimostra che tale tetto è *intrinseco al criterio* e non un artefatto della sua implementazione (sezione 4.6). Il teorema separa ciò che è dimostrato — la caratterizzazione delle rimozioni certificabili — da ciò che è misurato, e la distinzione è mantenuta in tutto il lavoro.

3. **Una verifica empirica della condizione** [V2] (sezione 7.1) su 1 242 321 terne con zero discrepanze, e la conferma che la sua applicazione lascia l'output identico al sistema originale su 113 configurazioni su 113, su quattro famiglie di dataset.

4. **Un'euristica di potatura aggressiva** (`A5b`, sezione 7.3) che su Bridges rimuove l'84,2 % delle regole *migliorando* il risultato (69 imputazioni corrette in più, 72 errori in meno, 11 volte più veloce), e la misurazione del fatto che la stessa strategia danneggia gli altri dataset.

5. **La spiegazione del perché** (sezione 7.5). Il guadagno della potatura aggressiva si scompone nel prodotto di due fattori — la quota di imputazioni ottenute per via esatta e il numero di imputazioni effettuate. La potatura conviene solo quando entrambi crescono. Ne discende un criterio diagnostico misurabile prima di potare, che sostituisce la soglia euristica precedentemente adottata.

6. **Sei difetti documentati** (capitolo 9), quattro del materiale d'archivio e due del codice sviluppato, con impatto quantificato e procedura di riproduzione: con procedura di riproduzione: un ramo irraggiungibile nel codice di calcolo della similarità, un disallineamento di schema nei dataset Glass, indici di ground truth non ricostruibili nei dataset Physician, e una build con istruzioni di debug. A questi si aggiunge un difetto di correttezza nel codice sviluppato in questo lavoro, individuato e corretto.

7. **La caratterizzazione del costo computazionale** del sistema (sezione 2.6), ottenuta ricompilando il codice originale da sorgenti decompilati — con equivalenza verificata rispetto all'eseguibile di partenza — e strumentandolo: il 99 % del tempo di esecuzione è nella verifica dei veti, non nella generazione dei candidati.

8. **La confutazione della proporzionalità fra riduzione e tempo** (sezione 7.9). Il lavoro mostra che il tempo di esecuzione dipende dalla *composizione* dell'insieme potato e non dalla sua dimensione: la strategia che rimuove fino al 98 % delle regole rallenta l'esecuzione fino a 251 volte, e una variante che conserva più regole di un'altra risulta circa nove volte più lenta. La spiegazione candidata — la perdita della scorciatoia esatta per le tuple con una sola cella nulla — è dichiarata come non verificata.

Struttura dell'elaborato

Il capitolo 1 introduce i concetti preliminari e colloca il lavoro rispetto alla letteratura: qualità dei dati, valori mancanti, tecniche di imputazione, *Relaxed Functional Dependencies*, riduzione di un insieme di regole, misure di similarità e la nozione di semantic pruning adottata.

Il capitolo 2 analizza il sistema Renuver per disassemblaggio: architettura, flusso di imputazione, dataset impiegati, il doppio ruolo delle RFD, le due procedure di riempimento e la caratterizzazione del costo computazionale.

Il capitolo 3 presenta la metodologia: motivazioni, spazio delle strategie, le strategie implementate, le condizioni esatte di potatura derivate dal comportamento del sistema e il contratto di integrazione.

Il capitolo 4 formula e dimostra il risultato negativo che delimita il campo di applicabilità, ne trae la conseguenza sul target di riduzione e ne precisa la portata.

Il capitolo 5 espone come sono nate le ipotesi, le sette ipotesi primarie con il loro criterio di falsificazione e le domande esplorative.

Il capitolo 6 descrive la valutazione sperimentale: il protocollo di verifica e la metodologia statistica, l'infrastruttura, i dataset e le configurazioni, le metriche, il

termine di paragone banale, gli esperimenti condotti con il collegamento alle ipotesi e lo studio di ablazione.

Il capitolo 7 analizza i risultati: la potatura che non perde nulla, il confronto con la riduzione di ridondanza della teoria classica, la potatura che produce il guadagno e il confine in cui il guadagno non si realizza, il meccanismo che li spiega, il criterio diagnostico, i risultati negativi e il costo.

Il capitolo 8 discute gli esiti delle ipotesi, il quadro complessivo, i limiti dell'evidenza e la collocazione del lavoro rispetto alla letteratura.

Il capitolo 9 raccoglie i difetti e le criticità: quelli del sistema studiato, quelli del codice sviluppato in questo lavoro e le conseguenze generali che se ne traggono.

Il capitolo 9.4 sintetizza il lavoro e ne indica gli sviluppi possibili. L'Appendice A documenta la riproducibilità dei risultati e l'Appendice B raccoglie le tabelle complete delle campagne.

–1–
Background e stato dell'arte

Questo capitolo introduce i concetti su cui poggia il lavoro e li colloca rispetto alla letteratura. Il problema affrontato nasce da una proprietà del dato — l'incompletezza — e dalla possibilità di rimediarvi sfruttando regolarità osservate; il capitolo ricostruisce perciò il quadro della qualità dei dati e delle sue dimensioni, la natura dei valori mancanti e i meccanismi che li producono, le famiglie di tecniche di imputazione con le ipotesi che ciascuna richiede, le dipendenze funzionali rilassate e le misure di similarità che ne definiscono la semantica, la riduzione di un insieme di regole nella sua accezione classica, e infine la nozione di *semantic pruning* adottata in questo lavoro, con la sua collocazione rispetto alle linee di ricerca affini.

1.1 Qualità dei dati e sue dimensioni

La qualità dei dati non è una proprietà intrinseca del dato ma una relazione fra il dato e l'uso che ne viene fatto: un dataset è di qualità sufficiente rispetto a un compito, e la stessa raccolta può risultare adeguata per un'analisi descrittiva e inadeguata per una decisione operativa. La letteratura organizza questa relazione in *dimensioni*, e la classificazione più diffusa, derivata dallo studio delle aspettative degli utilizzatori, comprende accuratezza, completezza, consistenza, tempestività e unicità [2]. Le dimensioni non sono indipendenti — un dataset completo ma inconsistente è di uso difficile quanto uno accurato ma incompleto — e la loro rilevanza dipende dal compito.

Formalmente, la completezza di un dataset D con schema $\mathcal{A} = \{A_1, \ldots, A_m\}$ si misura come la frazione di celle valorizzate sull'insieme delle celle attese, $\mathrm{comp}(D) = 1 - |\{(t, A_i) : t[A_i] = \bot\}| \,/\, (|D| \cdot m)$; la sua violazione è un *valore mancante*, che è l'oggetto della sezione seguente. Nel lavoro la completezza è la dimensione di partenza — il sistema studiato esiste per rimediare alla sua violazione — mentre l'accuratezza è la dimensione lungo la quale si misura se il rimedio migliora o peggiora il dato. La ridondanza dei metadati, che non compare nelle classificazioni classiche perché riguarda il processo di arricchimento e non il dato, è il bersaglio della potatura ed è trattata nella sezione 1.5.

Il quadro complessivo è quello dei sistemi di data quality, in cui l'imputazione è una delle attività di miglioramento e va valutata insieme ai costi che introduce [3]. È questa duplice natura — miglioramento del dato e costo di elaborazione — a rendere sensato chiedersi se una parte dell'informazione impiegata sia superflua.

Tabella 1.1: Dimensioni della qualità dei dati rilevanti per il lavoro. Per ciascuna sono indicati il difetto corrispondente e il rapporto con il problema affrontato.

dimensione	difetto corrispondente	rapporto con questo lavoro
completezza	valori assenti in celle che dovrebbero essere valorizzate	è la dimensione di partenza: l'imputazione esiste per rimediare alla sua violazione
accuratezza	valore presente ma diverso da quello corretto	è la dimensione che l'imputazione può migliorare o peggiorare, ed è misurata dal ground truth delle configurazioni remodulate
consistenza	violazione di un vincolo fra attributi	è la proprietà che le RFD esprimono in forma rilassata, e che il sistema usa per rifiutare le imputazioni incoerenti
ridondanza	informazione duplicata o non necessaria	è la dimensione su cui agisce la potatura: un insieme di regole ridondante è un costo senza contropartita informativa

1.2 Valori mancanti

Un valore mancante non è un valore: è l'assenza di un'informazione. La distinzione classica riguarda il *meccanismo* che lo ha prodotto, poiché da esso dipende la legittimità di qualunque tecnica di ricostruzione [4].

Definizione 1 (Meccanismi di mancanza)**.** Sia X una variabile e M l'indicatore della sua mancanza. Si distinguono:

- **MCAR** (*Missing Completely At Random*): M è indipendente da X e da ogni altra variabile osservata;
- **MAR** (*Missing At Random*): M può dipendere dalle variabili osservate, ma non dal valore mancante X;
- **MNAR** (*Missing Not At Random*): M dipende dal valore mancante stesso.

La distinzione ha una conseguenza pratica diretta: solo sotto MCAR e MAR la distribuzione dei dati osservati è informativa rispetto al valore mancante, e solo in quei regimi un metodo di imputazione basato sulle correlazioni osservate può essere corretto in senso statistico [5]. Sotto MNAR la ricostruzione richiede di modellare il meccanismo di mancanza, e un metodo che non lo faccia produce stime distorte in modo non controllabile dai dati disponibili.

Nel contesto di questo lavoro la distinzione ha un ruolo operativo preciso. I dataset impiegati sono *remodulati*: i valori mancanti sono introdotti artificialmente a partire da un dataset completo, e il valore originale è conservato come ground truth. Il meccanismo è quindi, per costruzione, noto e controllato. Va però osservato che la rimozione artificiale è progettata per essere casuale, e ricade pertanto nel caso MCAR: i risultati del lavoro valgono per questo regime, e non si estendono automaticamente a mancanze strutturali, in cui l'assenza di un valore è essa stessa un'informazione — come accade quando un attributo non è applicabile a una parte delle tuple.

1.3 Data imputation

L'imputazione consiste nel sostituire un valore mancante con una stima ottenuta dal resto dell'informazione disponibile. Le famiglie di tecniche si distinguono per la natura dell'informazione sfruttata e per le ipotesi che devono valere perché la stima sia sensata.

Tabella 1.2: Famiglie di tecniche di imputazione. Per ciascuna sono indicati l'informazione impiegata, l'ipotesi richiesta e il comportamento rispetto alla copertura.

famiglia	informazione impiegata	ipotesi	copertura
statistica	distribuzione marginale o congiunta di una variabile	il modello globale è corretto; sotto MAR, stima non distorta	totale: un valore viene sempre prodotto
apprendimento automatico	relazioni fra attributi apprese dai dati osservati	i dati mancanti seguono il medesimo processo dei dati osservati	totale
similarità	somiglianza fra tuple (k-NN)	tuple vicine hanno valori simili	totale o quasi, secondo la soglia adottata
vincoli	dipendenze fra attributi, espresse come regole	il vincolo è valido sul dato reale	*parziale*: l'imputazione può essere rifiutata

I metodi statistici classici — media, mediana, moda, regressione — sono semplici e veloci e assumono una struttura globale che raramente sussiste; l'imputazione multipla per equazioni concatenate ne rappresenta la versione moderna e condizionata, in cui ogni variabile è stimata a partire dalle altre in modo iterativo [6]. I metodi di apprendimento automatico, come le foreste casuali applicate all'imputazione di dati misti, rinunciano a un'assunzione distributiva e catturano relazioni non lineari, al prezzo di un costo di addestramento e di una minore interpretabilità [7]. I metodi basati su similarità copiano il valore da tuple ritenute vicine, con la scelta della metrica e del numero di vicini come parametri critici [8]. I metodi basati su vincoli impiegano dipendenze fra attributi — funzionali, di inclusione, di ordine — e possono essere formulati come un problema di riparazione che massimizza la coerenza complessiva [9, 10]; è la famiglia a cui appartiene Renuver.

La scelta della famiglia non è neutrale rispetto alla valutazione. Un metodo statistico produce sempre un valore: la sua copertura è per definizione totale, e la sua qualità si misura sull'accuratezza. Un metodo basato su vincoli può invece *rifiutare* di imputare, quando nessun candidato soddisfa i vincoli: in quel caso la copertura è una metrica tanto rilevante quanto l'accuratezza, e le due vanno lette insieme. Questa osservazione è alla base delle metriche adottate nel capitolo 6, ed è anche la ragione per cui una modifica dell'insieme di vincoli — come la potatura studiata in questo lavoro — può cambiare il risultato in due direzioni opposte, migliorando la qualità delle imputazioni prodotte e riducendo il numero di celle che ricevono un valore.

1.4 Relaxed Functional Dependencies

Una dipendenza funzionale (FD) classica asserisce che due tuple concordanti su un insieme di attributi X concordano necessariamente su un attributo A. Su dati reali l'asserzione è troppo rigida: il rumore e le eccezioni la falsificano quasi ovunque. Le dipendenze funzionali *rilassate* introducono forme di tolleranza, tipicamente sulle soglie di confronto e sulle eccezioni ammesse [11].

Definizione 2 (Relaxed Functional Dependency)**.** Una RFD è un'espressione della forma

$$(A_1 \leq \theta_1,\ \ldots,\ A_k \leq \theta_k)\ \rightarrow\ (A_{\text{RHS}} \leq \theta_{\text{RHS}})$$

dove gli A_i sono attributi del dataset, $\theta_i \geq 0$ sono soglie di tolleranza e A_{RHS} è l'attributo di destra. La parte sinistra è detta LHS, la destra RHS.

La semantica della tolleranza dipende dal tipo dell'attributo: per un attributo numerico $A_i \leq \theta_i$ significa che due tuple possono differire di al più θ_i su A_i; per un attributo categorico significa che i valori devono coincidere, oppure che la loro distanza di stringa non supera θ_i. La distinzione è rilevante nel seguito, poiché il sistema studiato tratta i due casi con procedure diverse.

Due osservazioni sono necessarie per la lettura del lavoro.

Osservazione 3 (Ruolo doppio)**.** Una RFD non è soltanto un descrittore del dataset: è un *oggetto operativo*. Nel sistema studiato essa partecipa a due procedure distinte, la generazione di candidati per una cella nulla e la verifica di coerenza di un'imputazione. Le due procedure hanno requisiti opposti rispetto alla tolleranza: una soglia larga rende la regola permissiva nella generazione e severa nella verifica. Questo doppio ruolo è il vincolo che determina l'intero problema affrontato (sezione 2.5).

Osservazione 4 (Soglia nulla)**.** Il caso $\theta_{\text{RHS}} = 0$ merita attenzione separata. Una RFD con soglia di destra nulla richiede coincidenza esatta sull'attributo di destra, ed è quella che il sistema tratta con una scorciatoia dedicata. Nel seguito la distinzione fra regole a soglia nulla e regole a soglia positiva ricorrerà come variabile esplicativa.

1.5 Ridondanza e riduzione di un insieme di regole

L'insieme delle RFD estratte da un dataset è, come ogni insieme di dipendenze, ridondante: alcune regole sono implicati da altre, e la loro presenza non aggiunge informazione sui vincoli del dato. La teoria classica delle dipendenze funzionali affronta il problema con la nozione di *copertura*: due insiemi di dipendenze sono equivalenti se hanno la stessa chiusura, e un insieme è *minimale* se nessuna delle sue dipendenze è derivabile dalle altre per mezzo degli assiomi di Armstrong [12, 13]. Il calcolo di una copertura minimale è un'operazione standard, e la sua applicazione alle RFD è stata studiata come forma di semplificazione degli insiemi di regole.

Questa letteratura fornisce il termine di paragone naturale del lavoro, ed è importante chiarire perché non lo risolve. La nozione classica di ridondanza è *dichiarativa*: una dipendenza è superflua se la sua rimozione non cambia l'insieme dei vincoli soddisfatti. Nel sistema studiato il ruolo di una regola è invece *operativo*, e dipende da come

il programma la consuma: una regola partecipa alla generazione di un valore e alla verifica di coerenza, e la scorciatoia per le regole a soglia di destra nulla mostra che due regole equivalenti dal punto di vista logico possono avere costi ed effetti diversi. Ne segue che la ridondanza classica non implica la sicurezza della rimozione, e che il problema studiato non è un caso particolare del calcolo di una copertura minimale.

Il capitolo 4 formalizza questa distinzione. La conseguenza per la valutazione sperimentale è che il termine di paragone del lavoro non è una copertura minimale — che non garantirebbe l'equivalenza del comportamento — ma l'*equivalenza osservata* del sistema prima e dopo la potatura, misurata per confronto degli output. La scelta è più onerosa e più debole sul piano teorico, ma è l'unica che risponda alla domanda posta: se una regola possa essere rimossa senza che il sistema se ne accorga.

1.6 Misure di similarità e distanza

Il confronto fra tuple avviene attributo per attributo, e il risultato complessivo è una aggregazione dei confronti elementari. Nel sistema studiato l'aggregazione è una media dei contributi normalizzati sull'ampiezza del lato sinistro della regola. La normalizzazione ha conseguenze non ovvie: a parità di scostamento assoluto, un contributo è tanto minore quanto più grande è la soglia di tolleranza dell'attributo, sicché attributi con scale diverse non sono confrontabili direttamente.

Per gli attributi numerici il confronto elementare è la differenza assoluta, eventualmente normalizzata sull'intervallo dei valori osservati. Per gli attributi testuali si impiegano misure di edit distance, fra cui la distanza di Levenshtein, che conta il numero minimo di inserimenti, cancellazioni e sostituzioni necessari a trasformare una stringa nell'altra [14]; la sua versione normalizzata sulla lunghezza della stringa è impiegata per rendere confrontabili valori di lunghezza diversa, ed è quella adottata per la famiglia Physician, i cui attributi testuali designano la stessa entità con forme diverse. Anche in questo caso la soglia di tolleranza della regola agisce da parametro di accettazione.

La scelta della misura non è neutrale, come mostra il difetto D1 discusso nella sottosezione 9.2.1: nel sistema studiato l'attributo categorico non riceve la misura prevista, e il confronto ricade su una distanza di stringa applicata a etichette lunghe e prive di relazione. La circostanza è rilevante per l'interpretazione dei risultati, perché rende il comportamento del sistema dipendente da una misura che non è quella dichiarata.

1.7 Semantic pruning: definizione operativa e collocazione

Con *semantic pruning* si intende, in questo lavoro, la selezione di un sottoinsieme di un insieme di metadati — nel caso in esame, le RFD — guidata dal *significato* che quei metadati hanno per il sistema che li consuma, e non da criteri sintattici o statistici sul file.

La distinzione è sostanziale. Un criterio sintattico rimuove regole in base alla loro forma (duplicati, dimensione del lato sinistro, valore delle soglie); un criterio statistico le rimuove in base alla loro frequenza o alla loro resa osservata. Un criterio semantico richiede invece di sapere *che cosa il sistema fa* con ciascuna regola, e quindi di derivare le condizioni di rimozione dal comportamento effettivo del codice. La qualificazione "semantico" non è quindi un sinonimo di "euristico": indica la fonte dell'informazione impiegata, che è la semantica operativa del consumatore dei metadati e non una proprietà statistica del file.

Definizione 5 (Potatura sicura)**.** Sia $\mathcal{R}$ un insieme di regole e Π il programma che le consuma. Una rimozione $r \in \mathcal{R}$ è *sicura rispetto a una proprietà* P se il comportamento di Π rispetto a P è invariato quando r è assente.

La definizione è parametrica rispetto a P, e la scelta di P è il nodo del problema. Nel sistema studiato si possono isolare almeno due proprietà candidate — la capacità di generare candidati e il potere di veto — e si dimostrerà che esse non sono simultaneamente preservabili oltre una soglia quantificata (sezione 4.3).

Questa impostazione distingue il lavoro da tre alternative presenti in letteratura. La prima è la potatura *a posteriori*, che misura l'effetto di ogni rimozione eseguendo il sistema: corretta ma computazionalmente proibitiva, poiché richiede una esecuzione per candidato. La seconda è la potatura basata su un *proxy* offline, che stima la rilevanza di una regola con una grandezza calcolata sul dataset: economica, ma — come si mostrerà nel capitolo 7 — potenzialmente fuorviante, al punto da risultare *anti-correlata* con la rilevanza effettiva. La terza è la selezione guidata dalla semantica *dei nomi* dei metadati, che impiega modelli linguistici per inferire relazioni fra colonne a partire dalla loro denominazione [15] o per guidare l'esplorazione di fonti dati eterogenee [16]: si tratta di una linea di ricerca affine per ambizione — impiegare il significato invece della forma — ma distinta per oggetto, perché riguarda l'*interpretazione* dei metadati e non la loro *selezione* rispetto al consumo da parte di un algoritmo noto.

La metodologia adottata cerca una quarta via, intermedia fra le precedenti: derivare dal codice condizioni *sufficienti*, verificabili offline, la cui validità non dipende da una stima. È questa la scelta che rende il risultato negativo del capitolo 4 possibile e significativo: poiché le condizioni non sono euristiche, il loro limite è un limite del problema e non dell'implementazione.

–2–
Analisi del sistema RENUVER

Questo capitolo analizza il sistema oggetto di studio. L'analisi non si basa sulla documentazione ma sul *comportamento effettivo* del codice: il sistema è distribuito come archivio eseguibile senza sorgenti, e le proprietà su cui poggia l'intera metodologia sono state derivate per disassemblaggio. Ne emergono l'architettura e il flusso di imputazione, i ruoli che le regole vi svolgono — generazione dei candidati, verifica di coerenza e rivalutazione delle regole escluse dalla ricerca ordinaria — le due procedure con cui il sistema tratta le tuple incomplete, e la distribuzione del costo computazionale fra le sue fasi. Da questa analisi discendono le condizioni esatte di potatura su cui si fonda la metodologia, che sono esposte nel capitolo successivo insieme alle strategie che le impiegano.

2.1 Obiettivi del sistema

RENUVER [1] è un algoritmo di imputazione di valori mancanti basato su *Relaxed Functional Dependencies*. L'idea portante è che le regole di imputazione non debbano essere fornite dall'esterno, ma estratte dal dataset stesso: il sistema riceve in ingresso un insieme di RFD già estratte e le impiega per ricostruire le celle nulle.

Rispetto ai metodi statistici, l'approccio ha il vantaggio di produrre imputazioni motivate da vincoli osservati nei dati, e di poter *rifiutare* di imputare quando nessun candidato soddisfa i vincoli. Quest'ultima proprietà è deliberata: è preferibile una cella non imputata a una cella imputata in violazione delle dipendenze del dataset.

2.2 Architettura generale

Il sistema è distribuito come archivio Java eseguibile. L'analisi ha stabilito i seguenti fatti, rilevanti per il seguito.

- Il codice applicativo consta di 13 classi nel *package* di default, incluse in un archivio "grasso" che incorpora anche la libreria di collezioni `fastutil` (11 040 classi complessive). Le classi applicative sono compilate per la versione 52 del formato `class` (Java 8).

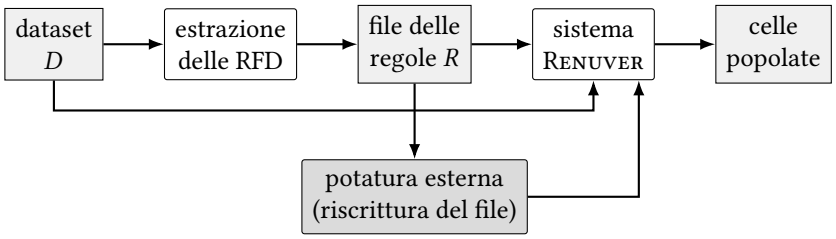


Figura 2.1: Flusso di elaborazione del sistema studiato e punto di innesto della potatura. La potatura riscrive il file delle regole prima che il sistema lo consumi: il dataset e l'eseguibile restano invariati.

- L'interfaccia a riga di comando accetta sei argomenti: la tipizzazione delle colonne, il file del dataset, il separatore, il file delle RFD, una soglia numerica e un selettore di euristica.
- **Il sesto argomento è ignorato.** Il metodo di avvio non lo legge: il sistema esegue sempre la medesima procedura di riempimento. Le altre procedure presenti nel codice — ricerca locale, riordino per numero di valori nulli, riordino per coinvolgimento nel lato sinistro — non sono raggiungibili dalla riga di comando. La verifica è stata condotta per disassemblaggio e confermata empiricamente. Il `README` dell'archivio documenta peraltro i parametri come `args0`–`args4` e `args6` — l'indice `args5` non compare — e precisa che "in questa versione è supportata soltanto l'euristica sequenziale `0`": la documentazione dichiara quindi un limite di *versione*, mentre il codice mostra che l'argomento non viene mai letto. Le due circostanze sono distinte e la seconda è più forte della prima, perché esclude che il parametro possa essere onorato da una versione futura senza modifiche al codice.
- La quinta soglia ha un *doppio ruolo*: filtra le RFD in ingresso e funge da valore iniziale della soglia di accettazione del punteggio di similarità. I due ruoli non sono separabili dalla riga di comando.

2.3 Flusso di imputazione e procedure di riempimento

Il flusso si articola in tre fasi.

1. **Costruzione degli indici.** Le RFD vengono raggruppate in *cluster*, dove un cluster è l'insieme delle regole che condividono lo stesso attributo di destra e la stessa soglia di destra. Contestualmente si individuano le tuple del dataset che presentano celle nulle.
2. **Trattamento delle tuple incomplete.** Per ogni tupla con celle nulle si tenta di riempirle, secondo due procedure distinte a seconda che la tupla abbia una sola cella nulla o più di una (sezione 2.3).
3. **Verifica e registrazione.** Ogni imputazione candidata è sottoposta a una verifica di coerenza rispetto all'insieme delle regole; solo se supera la verifica viene registrata.

2.4 Dataset impiegati

L'archivio comprende cinque famiglie di dataset, con caratteristiche molto diverse fra loro. La Tabella 2.1 ne riassume la struttura rilevante per il pruning.

Tabella 2.1: Caratteristiche delle famiglie di dataset. N è il numero di RFD alla soglia indicata; "celle valutate" è il numero di celle per cui è disponibile il valore di ground truth, che — come si dirà — non coincide con il numero di celle nulle del dataset.

famiglia	dataset	N (soglia)	celle valutate	esito dell'analisi
Bridges	25	1 830–18 844	14–70	utilizzabile
Glass	25	1 511–27 445	22–107	richiede correzione di schema
Restaurant	45	25–124	fino a 2 074	utilizzabile con metrica testuale
Physician	29	3 734	fino a 338 507	non utilizzabile
Nursery	0	60–1 693 byte	—	regole presenti, dataset assenti

Due circostanze vanno segnalate perché condizionano la valutazione sperimentale.

Le celle valutate non sono le celle nulle. Il valore di ground truth è fornito da un file separato, in formato `riga;attributo;valore`, che elenca soltanto le celle introdotte artificialmente. Il dataset può contenere altre celle nulle, preesistenti, che il sistema tratta ma che non vengono valutate. Di conseguenza copertura e accuratezza sono relative all'insieme delle celle elencate, non a tutte le celle nulle del dataset. La distinzione è necessaria per interpretare correttamente le metriche.

Il costo computazionale non è uniforme. I dataset Physician non remodulati contengono 103 592 righe e fino a 121 913 celle nulle: una singola esecuzione richiederebbe tempi non compatibili con una campagna sperimentale. Le versioni *remodulate* della stessa famiglia, di dimensioni ridotte, presentano a loro volta difetti nel materiale di ground truth (capitolo 9).

2.5 Il doppio ruolo delle RFD

Questa sezione stabilisce il vincolo che determina l'intero problema. L'analisi del codice mostra che una RFD partecipa a *due* procedure con requisiti opposti.

Ruolo generativo. Data una cella nulla, il sistema cerca tuple del dataset sufficientemente simili alla tupla incompleta; il punteggio di similarità è calcolato rispetto alle regole di un cluster, e il miglior punteggio determina il valore da imputare. In questo ruolo una soglia *larga* rende la regola più permissiva.

Ruolo di veto. Prima di registrare un'imputazione, il sistema verifica che essa non violi alcuna regola. In questo ruolo una soglia *stretta* rende la regola più severa.

Le due procedure non sono separabili nel formato del file: una RFD è una riga di testo, e nulla indica se essa serva all'una, all'altra, o a entrambe. Ne segue l'osservazione che governa il lavoro.

Algorithm 1 Verifica dei veti su una tupla incompleta, ricostruita dal bytecode. Il ciclo esamina *tutte* le regole dell'insieme e, per ciascuna, scansiona il dataset alla ricerca di un testimone che violi la soglia di destra.

input: tupla t con la cella $t[A]$ nulla, insieme di regole R, dataset D
for all $r \in R$ tale che $\mathrm{RHS}(r) = A$ **do**
    if $\mathrm{LHS}(r) \cap \mathrm{NullCols}(D) = \emptyset$ **then**
        continue ▷ nessun confronto possibile: la regola non può vetare
    end if
    cerca in D una tupla $s \neq t$ che concordi con t su ogni colonna di $\mathrm{LHS}(r)$ entro la soglia θ_r
    if esiste s e $t[A]$ differisce da $s[A]$ oltre $\theta_{\mathrm{RHS}}(r)$ **then**
        return incoerente ▷ veto esercitato da r
    end if
end for
return coerente

Osservazione 6 (Il pruning esterno è intrinsecamente conservativo)**.** Poiché rimuovere una regola ne elimina simultaneamente il contributo generativo e quello di veto, e poiché i due contributi non sono distinguibili dal formato, una strategia di potatura che operi sul solo file delle regole deve assumere che ogni regola rimossa possa essere necessaria al veto.

La Tabella 2.2 quantifica quanto questa assunzione sia stringente: la quasi totalità delle regole è, in linea di principio, in grado di esercitare un veto.

Un terzo ruolo, esercitato da una parte delle regole. All'avvio il sistema compie una propria riduzione dell'insieme: le regole con soglia di destra *esattamente nulla* il cui lato sinistro si comporta come una chiave sul dataset vengono tolte dall'insieme di lavoro, raccolte in una struttura separata e sottoposte a una procedura distinta, invocata quando la ricerca ordinaria di un candidato non produce alcun valore accettabile. Il log dell'esecuzione registra l'operazione ("`Removing key RFD...`"), e il numero di regole rimosse non è marginale: su `bridges_14_1` a soglia 9 l'insieme passa da 8 961 a 8 581 regole. La verifica della proprietà di chiave è inoltre quadratica nella dimensione del dataset, ed è la ragione per cui il preprocessing del sistema — che non è la potatura studiata in questo lavoro, ma una fase interna dell'eseguibile — domina il tempo sulle configurazioni grandi: sulla medesima configurazione 24,5 s di preprocessing contro 7,1 s di imputazione, e fino a 113 s su una configurazione Physician.

La circostanza ha due conseguenze. La prima riguarda la descrizione del sistema: i ruoli osservabili sono *tre*, non due, e il terzo è esercitato soltanto da una parte delle regole. La seconda riguarda la portata delle garanzie: la condizione [V2] e il teorema del capitolo 4 sono enunciati rispetto all'insieme delle regole così come compare nel file, mentre il sistema ne consuma un sottoinsieme determinato da un criterio proprio; la verifica sperimentale di equivalenza (sezione 7.1) copre l'interazione fra i due meccanismi, ma un trattamento formale che la includesse dovrebbe modellare anche quella riduzione. La questione è ripresa nella sezione 4.7.

Tabella 2.2: Quota di RFD che possono esercitare un veto, per famiglia e soglia. Una RFD può esercitare un veto solo se il suo lato sinistro coinvolge almeno una colonna che presenta celle nulle nel dataset.

famiglia	soglia	N	RFD che *possono* fare veto	quota
Bridges	3	1 830	1 818	99,3 %
Bridges	9	8 961	8 945	99,8 %
Bridges	15	18 844	18 828	99,9 %
Glass	3	1 511	1 476	97,7 %
Glass	9	11 648	11 564	99,3 %
Glass	15	27 445	27 327	99,6 %
Restaurant	3	25	23	92,0 %
Restaurant	6	124	116	93,5 %

Le due procedure di riempimento. Il trattamento di una tupla incompleta segue due procedure diverse, la cui distinzione è essenziale per interpretare i risultati.

Tupla con una sola cella nulla. Il sistema scorre i cluster della colonna interessata e si arresta al primo che produce un candidato valido. Esiste inoltre una scorciatoia: se la regola impiegata ha soglia di destra nulla, il valore della tupla candidata viene scritto direttamente e la procedura termina. Ne segue che, in questo ramo, la *posizione* di un cluster nell'ordine di scansione è determinante.

Tupla con più celle nulle. La procedura è iterativa: per un numero di round pari al massimo numero di cluster fra le colonne coinvolte, e per ciascun round, ogni cella nulla riceve i candidati del cluster corrispondente al round corrente. La scrittura del valore avviene senza verificare se la cella sia già stata riempita in un round precedente; un round successivo sovrascrive quindi il risultato del precedente, e un round i cui candidati fallissero tutti la verifica di coerenza può riportare la cella allo stato nullo. In questo ramo, dunque, il valore finale di una cella dipende dall'*ultimo* cluster produttivo della sua colonna.

La distinzione ha una conseguenza quantitativa che sarà ripresa nel capitolo 7: le due procedure non sono ugualmente rappresentate nei dataset. Su Bridges circa tre quarti delle celle nulle appartengono a tuple con più di una cella nulla; su Glass vale la situazione opposta, con la maggior parte delle celle in tuple con una sola cella nulla (fra il 58 % e il 91 % secondo la configurazione). A Restaurant la proporzione dipende dalla dimensione: le configurazioni fino a qualche centinaio di celle nulle sono a maggioranza mono-nullo, mentre le più grandi — oltre mille celle nulle, fino a 2 074 — sono a maggioranza multi-nullo.

2.6 Costo computazionale

Poiché il sistema è distribuito senza sorgenti, la determinazione di dove si concentri il tempo di esecuzione richiede di ricompilarlo e strumentarlo. La procedura — ricompilazione da sorgenti decompilati, con verifica di equivalenza rispetto all'eseguibile

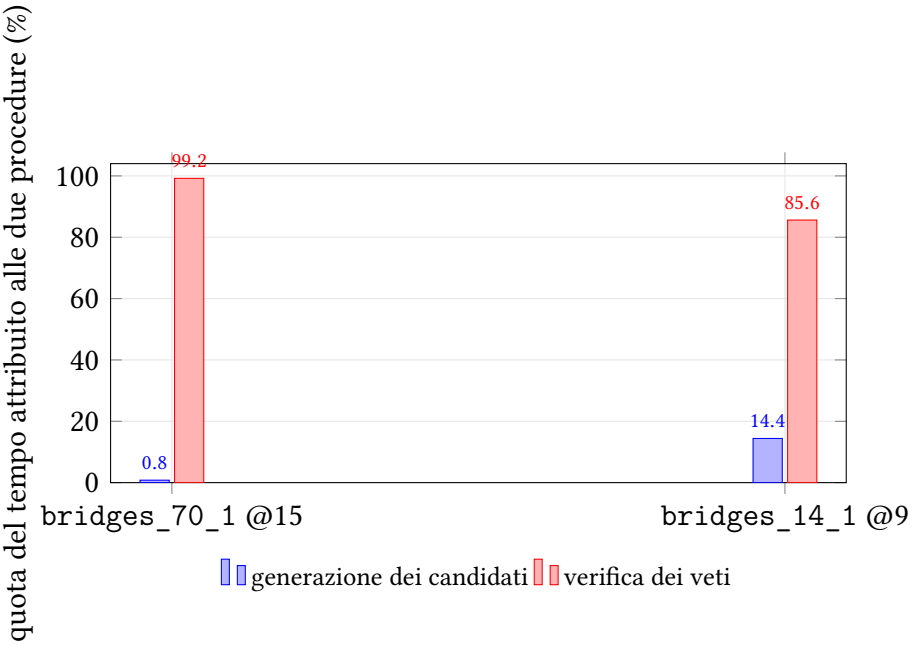


Figura 2.2: Ripartizione del tempo di esecuzione fra le due procedure, misurata per strumentazione su due configurazioni. La verifica dei veti domina in entrambe, e all'interno di essa il costo è quasi interamente nella scansione del dataset.

originale — è descritta nella sezione A.3; qui se ne riportano i risultati, che orientano la scelta delle strategie di ottimizzazione.

Tabella 2.3: Ripartizione del tempo di esecuzione fra le due procedure principali, misurata per strumentazione. La percentuale è calcolata sul tempo complessivamente attribuito alle due procedure.

configurazione	procedura	chiamate	tempo	quota
`bridges_70_1`, soglia 15	generazione candidati	531	1,4 s	0,8 %
	verifica dei veti	151	158,4 s	99,2 %
`bridges_14_1`, soglia 9	generazione candidati	293	1,5 s	14,4 %
	verifica dei veti	8	9,1 s	85,6 %

All'interno della verifica dei veti, la quasi totalità del tempo è assorbita dalla scansione del dataset: su `bridges_70_1` a soglia 15, 354 809 invocazioni della verifica di violazione assorbono 158,3 s su 158,4 s totali, mentre il confronto dei nomi di colonna — che pure è il primo test eseguito — costa 0,16 s.

Il risultato ha una conseguenza diretta e controintuitiva: **il tempo è nel veto, non nella generazione**. Ne segue che le strategie di potatura producono un guadagno temporale proprio in quanto riducono il numero di regole esaminate dalla verifica dei veti, e che qualunque ottimizzazione mirata alla sola generazione dei candidati non potrebbe che incidere su una frazione marginale del costo.

–3–
Metodologia di semantic pruning

Questo capitolo presenta la metodologia sviluppata. Le condizioni esatte di potatura, derivate dal comportamento del sistema, costituiscono il criterio con cui le strategie vanno lette: da esse discende che cosa si può rimuovere con garanzia e che cosa soltanto a costo di un rischio dichiarato. Il capitolo motiva l'approccio, organizza lo spazio delle strategie lungo le dimensioni che ne determinano la natura, descrive quelle implementate con il loro regime di garanzia, la loro riduzione e il loro costo, spiega perché la generazione dei candidati non ammetta una condizione esatta della stessa natura di quella che governa il veto, e definisce il contratto con cui la potatura si integra nel sistema. Le proprietà formali e il risultato negativo che delimita il campo di applicabilità sono oggetto del capitolo successivo.

3.1 Motivazioni

Le ragioni per potare l'insieme delle RFD sono tre, di natura diversa, e la loro compresenza spiega perché il problema non si riduca a un'ottimizzazione di prestazioni.

Ragione economica. Il costo dell'imputazione cresce con il numero di regole: la verifica dei veti esamina l'intero insieme per ogni imputazione candidata, e la sezione 2.6 mostra che è questa la voce dominante del tempo di esecuzione. Ridurre l'insieme riduce quel costo, e la riduzione è tanto più preziosa quanto più il sistema è impiegato su dataset di dimensioni maggiori.

Ragione di qualità. Una regola non è necessariamente benefica. Regole con soglie larghe competono nella generazione dei candidati e possono prevalere su regole più precise, producendo imputazioni peggiori di quelle che si otterrebbero senza di esse. La potatura può quindi migliorare il risultato, non soltanto accelerarlo: è la circostanza che il capitolo 7 misura, e che rende il problema interessante dal punto di vista della qualità dei dati e non solo dell'efficienza.

Ragione epistemica. Un insieme di migliaia di regole è esso stesso un oggetto difficile da interpretare: non è ispezionabile, non è confrontabile fra dataset e non consente di attribuire un comportamento a una causa. La selezione di un sottoinsieme

rilevante ha valore conoscitivo indipendentemente dal guadagno prestazionale, perché è la condizione per cui il comportamento del sistema diventa spiegabile.

La tensione che governa il problema. Le tre ragioni convergono su un'unica operazione — rimuovere regole — ma il sistema studiato assegna a ogni regola due ruoli distinti, la generazione dei candidati e il veto sulle imputazioni incoerenti, governati da requisiti opposti sulla stessa grandezza: una soglia stretta rende la regola più selettiva come generatore e più severa come veto, mentre una soglia larga la rende più permissiva come generatore e più debole come veto. Nulla, nel formato del file, distingue i due ruoli (sezione 2.5). Ne segue che ogni strategia di rimozione deve dichiarare *quale* dei due ruoli è disposta a sacrificare, e che la nozione di "potatura sicura" (Theorem 5) richiede di specificare rispetto a che cosa la sicurezza è pretesa. È questa precisazione a rendere possibile il risultato negativo del capitolo 4.

3.2 Dal sistema alle condizioni esatte

Algorithm 2 Ricerca dei copritori per la condizione [V2]. Il parametro μ limita la differenza di ampiezza fra il lato sinistro della regola coprente e quello della regola coperta; la sezione 4.6 mostra che il risultato non dipende dal suo valore.

input: insieme di regole R, dataset D, parametro μ
$C \leftarrow \emptyset$ ▷ regole la cui rimozione è garantita
for all $r \in R$ con $\mathrm{LHS}(r) \cap \mathrm{NullCols}(D) \neq \emptyset$ **do**
  for all $r'' \in R \setminus \{r\}$ con $\mathrm{RHS}(r'') = \mathrm{RHS}(r)$ e $|\mathrm{LHS}(r)| - |\mathrm{LHS}(r'')| \leq \mu$ **do**
    if $\mathrm{LHS}(r'') \subseteq \mathrm{LHS}(r)$ **and** $\theta_{r''}(c) \geq \theta_r(c)\ \forall c \in \mathrm{LHS}(r'')$ **and** $\theta_{\mathrm{RHS}}(r'') \leq \theta_{\mathrm{RHS}}(r)$ **then**
      $C \leftarrow C \cup \{r\}$ ▷ r'' copre r
      break
    end if
  end for
end for
return C

Le condizioni che seguono non sono ipotizzate a priori: sono ricavate dal comportamento effettivo del sistema, letto sul bytecode decompilato, e classificate in base al ruolo che una regola *può* svolgere. Esse costituiscono il fondamento della metodologia e la base del risultato negativo del capitolo 4.

Definizione 7 (Condizione [V1] — veto impossibile)**.** Sia NullCols l'insieme delle colonne del dataset che presentano almeno una cella nulla. Una RFD r *non può* esercitare alcun veto se

$$\mathrm{LHS}(r) \cap \mathrm{NullCols} = \emptyset.$$

La condizione è necessaria e sufficiente: la verifica di coerenza confronta valori, e il confronto richiede che entrambi i valori siano presenti. Se nessuna colonna del lato sinistro presenta valori mancanti, la verifica non può essere attivata.

Definizione 8 (Condizione [V2] — veto coperto)**.** Una RFD r è *coperta* se esiste una RFD r'' tale che:

(i) $\mathrm{RHS}(r'') = \mathrm{RHS}(r)$ e $\mathrm{LHS}(r'') \subseteq \mathrm{LHS}(r)$, con $\mathrm{LHS}(r'') \neq \emptyset$;

(ii) $\theta_{r''}(c) \geq \theta_r(c)$ per ogni $c \in \mathrm{LHS}(r'')$ (soglie *più larghe*);

(iii) $\theta_{\mathrm{RHS}}(r'') \leq \theta_{\mathrm{RHS}}(r)$ (soglia di destra *più stretta*).

Proposizione 9 (Correttezza di [V2])**.** *Se r'' copre r secondo la Definizione 8 e r'' sopravvive alla potatura, allora ogni veto esercitato da r è implicato da un veto di r''.*

Dimostrazione. Sia t una tupla che attiva il veto di r. Per definizione esistono una tupla s e una colonna $c \in \mathrm{LHS}(r)$ tali che s e t concordano entro $\theta_r(c)$ su c, e s e t differiscono su $\mathrm{RHS}(r)$ di più di $\theta_{\mathrm{RHS}}(r)$.

Poiché $\mathrm{LHS}(r'') \subseteq \mathrm{LHS}(r)$, la tupla s concorda con t anche su ogni colonna di $\mathrm{LHS}(r'')$: per la condizione (ii) le soglie di r'' sono più larghe, dunque l'accordo entro $\theta_r(c)$ implica l'accordo entro $\theta_{r''}(c)$. La condizione (i) garantisce che $\mathrm{LHS}(r'')$ non sia vuoto, sicché la verifica di r'' ha almeno una colonna su cui essere valutata. Infine, per la condizione (iii), la differenza su RHS che supera $\theta_{\mathrm{RHS}}(r)$ supera a maggior ragione $\theta_{\mathrm{RHS}}(r'')$. Dunque s e t attivano anche il veto di r''. □

Nota 10. La direzione delle disuguaglianze è controintuitiva rispetto alla nozione classica di dominanza fra dipendenze, dove si è soliti richiedere che la regola dominante sia *più stretta*. Qui occorre l'opposto: la regola coprente deve essere più permissiva sul lato sinistro e più severa su quello destro, perché è la severità sul destro a implicare il rifiuto.

Definizione 11 (Condizione [V3] — regola inerte)**.** Una RFD r è *provabilmente inerte* se

$$\mathrm{LHS}(r) \cap \mathrm{NullCols} = \emptyset \quad \wedge \quad \mathrm{RHS}(r) \notin \mathrm{NullCols}.$$

Una regola inerte non può né generare candidati — la colonna di destra non ha celle nulle — né esercitare veti — per la [V1]. La sua rimozione è quindi priva di effetti per costruzione.

Tabella 3.1: Dimensione delle tre classi. Le classi [V2] e [V3] sono disgiunte sui dati esaminati: la loro unione è quindi la riduzione massima ottenibile con le condizioni esatte.

famiglia	soglia	N	[V3] inerte	[V2] coperta	unione
Bridges	3	1 830	2 (0,11 %)	215	11,9 %
Bridges	9	8 961	6 (0,07 %)	1 531	17,2 %
Bridges	15	18 844	6 (0,03 %)	3 927	20,9 %
Glass	3	1 511	5 (0,33 %)	40	3,0 %
Glass	9	11 648	18 (0,15 %)	861	7,5 %
Glass	15	27 445	24 (0,09 %)	2 363	8,7 %
Restaurant	3	25	0	0	0,0 %
Restaurant	6	124	0	2	1,6 %

La Tabella 3.1 mostra due fatti che orientano il resto del lavoro. Il primo è che la classe [V3] è trascurabile su tutte le famiglie, non soltanto su Bridges: la ragione è strutturale, poiché richiede regole il cui lato sinistro impieghi esclusivamente colonne mai nulle, e tali colonne sono poche. Il secondo è che la classe [V2] è l'unica leva significativa, e cresce con la soglia fino a poco più di un quinto dell'insieme.

3.3 Lo spazio delle strategie

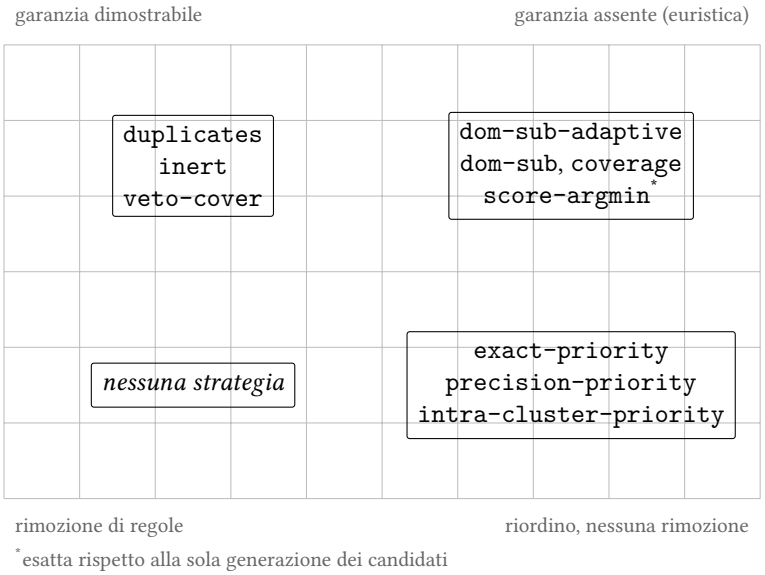


Figura 3.1: Collocazione delle strategie rispetto a che cosa modificano e a che cosa garantiscono. Il quadrante in basso a sinistra è vuoto per costruzione: una strategia che non rimuove regole non ha bisogno di garanzie, e una che le rimuove senza garanzia appartiene al quadrante in alto a destra.

Le venti strategie implementate non sono un elenco di tentativi: si distribuiscono lungo tre dimensioni indipendenti, che ne determinano la natura e le possibilità. La Tabella 3.2 ne descrive diciassette, quelle impiegate nel lavoro; le altre tre sono composizioni di più strategie e la variante classica adoperata come termine di paragone.

Che cosa cambiano. Le strategie di *rimozione* riscrivono il file eliminando regole; le strategie di *riordino* non eliminano nulla e modificano soltanto la posizione delle righe. La distinzione non è di intensità ma di natura: il riordino è l'unica classe che preserva entrambi i ruoli per costruzione, perché nessuna regola viene sottratta al sistema, e il suo effetto è quindi interamente mediato dal comportamento dell'algoritmo.

Che cosa garantiscono. Una strategia è *esatta* se la sua applicazione è dimostrabilmente priva di effetti — la rimozione è giustificata da una proprietà che si verifica sul file delle regole e sullo schema dei valori mancanti — ed *euristica* se la rimozione è plausibile ma non garantita. La distinzione è mantenuta in ogni fase: nelle strategie, nella politica di tutela dei veti e nella presentazione dei risultati.

Che informazione richiedono. Alcune strategie decidono guardando soltanto il file delle regole; altre richiedono un'analisi del dataset (il calcolo statico dell'argomento del minimo, la stima della copertura empirica). Il costo di questa analisi è un costo di preprocessing, che va confrontato con il risparmio ottenuto in esecuzione: la sezione 2.6 mostra che il confronto non è sempre favorevole.

Dalla combinazione delle tre dimensioni discende il criterio con cui le strategie sono presentate nel seguito: per ciascuna si dichiarano la famiglia, il regime di garanzia, l'informazione richiesta, la riduzione osservata e l'effetto misurato sulla qualità. Le strategie che non soddisfano il criterio di garanzia non sono presentate come alternative equivalenti ma come strumenti con un profilo di rischio esplicito.

3.4 Le strategie di potatura

Algorithm 3 Driver di potatura. Le rimozioni garantite sono esenti dal computo del budget: essendo prive di costo in termini di veto, addebitarle costituirebbe una doppia contabilizzazione.

input: regole R, dataset D, strategia S, politica π, frazione F
$R' \leftarrow S(R, D)$ ▷ trasformazione richiesta dalla strategia
$G \leftarrow \{r \in R \setminus R' : r \in C$ **or** $r \in I\}$ ▷ rimozioni garantite: C da [V2], I da [V3]
$H \leftarrow \{r \in R \setminus (R' \cup G) : \mathrm{LHS}(r) \cap \mathrm{NullCols}(D) \neq \emptyset\}$ ▷ rimozioni che sacrificano un veto
if $\pi =$ `protect` **then**
    $H \leftarrow \emptyset$
else if $\pi =$ `budget:F` **then**
    ordina H per indice di rischio crescente
    $H \leftarrow$ le $\lceil F \cdot |H| \rceil$ regole di H a rischio minore ▷ le altre sono ripristinate
end if
output: $R \setminus (G \cup H)$

La Tabella 3.2 riassume le strategie principali, con la garanzia associata, la riduzione osservata e l'effetto misurato sulla qualità[1].

La strategia garantita: `veto-cover`. Elimina le regole il cui veto è implicato da una regola sopravvissuta, secondo la condizione [V2] della sezione 3.2. È l'unica strategia di rimozione che offra una garanzia dimostrabile su entrambi i ruoli, e la sua riduzione è per questo limitata: cresce con la soglia e non supera il 20,9 % sui dataset esaminati. La sezione 7.1 ne verifica l'equivalenza di output.

[1]Le riduzioni e gli effetti provengono da esperimenti diversi, indicati nella colonna "effetto" e descritti nel capitolo 6: l'ablazione impiega venti configurazioni comuni su Bridges, l'esperimento di ordinamento quindici configurazioni alla soglia 9, la campagna principale le cinquanta configurazioni di Bridges. I valori non sono quindi confrontabili fra righe diverse della tabella.

Tabella 3.2: Strategie di potatura implementate. “Garanzia” indica se la rimozione è dimostrabilmente priva di effetti; “riduzione” è l’ordine di grandezza osservato sui dataset impiegati; “effetto” è la variazione di imputazioni corrette rispetto alla configurazione non potata, misurata nell’esperimento indicato. Le strategie citate nel seguito del lavoro sono in grassetto.

famiglia	strategia	garanzia	riduzione	effetto misurato
strutturale	`duplicates`	sì	0 %	nessuno
	`veto-dominance`	no	—	non impiegata
	`dom-exact`	no	—	non impiegata
esatta	**`veto-cover`**	**sì**	0 %–20,9 %	nessuno (113/113 configurazioni identiche)
	`inert`	sì	0,03 %–0,33 %	nessuno, per costruzione
	`score-argmin`	sì*	fino a 98 %*	nessuna perdita di copertura, ma rallentamento fino a 251×
euristica	**`dom-sub-adaptive`**	no	32 %–93 %	−20 corrette da sola (ablazione)
	`dom-sub`, `dom-sub-strict`	no	67 %–90 %	non impiegate da sole
	`used`, `empirical`	no	variabile	non impiegate da sole
	`coverage`, `lhs-cap`, `rhs-thr`	no	variabile	non impiegate da sole
di riordino	**`exact-priority`**	no	0 %	+25 corrette da solo (ablazione)
	`precision-priority`	no	0 %	+53 corrette (ordinamento)
	`intra-cluster-priority`	no	0 %	nessuno (15/15 configurazioni identiche)

*La garanzia di `score-argmin` riguarda la sola generazione dei candidati, non il potere di veto: la precisazione è essenziale ed è discussa nel sezione 7.9.

Le strategie esatte di portata limitata. `inert` rimuove le regole inerti secondo la condizione [V3]: la garanzia è totale e la riduzione trascurabile, circostanza che costituisce di per sé un risultato negativo sulla praticabilità di questa via. `score-argmin` preserva la generazione calcolando staticamente, per ogni cella nulla, l'argomento del minimo che il sistema sceglierebbe: può rimuovere fino al 98 % delle regole, ma non preserva il veto, e la sezione 7.8 mostra che il suo effetto sul tempo è opposto a quello atteso.

Le strategie euristiche. `dom-sub-adaptive` rimuove regole che risultano dominate rispetto a una relazione di sussunzione adattata ai dati, con una soglia di aggressività regolabile. È la strategia che produce le riduzioni maggiori, ed è anche quella il cui effetto sulla qualità dipende dal dataset: la combinazione impiegata nelle campagne principali (`A5b`) la associa al riordino per priorità di precisione.

Le strategie di riordino. Non eliminano regole e non sono soggette ad alcuna politica di tutela. `exact-priority` dispone per prime le regole che producono il valore esatto, `precision-priority` quelle con la precisione empirica più alta, `intra-cluster-priority` interviene sull'ordine interno ai cluster. Il loro effetto è mediato dalle due procedure di riempimento (sezione 2.3) ed è oggetto dell'esperimento di ordinamento i cui esiti sono riportati nella sezione 7.8.

Componibilità delle strategie. Le famiglie non sono intercambiabili e la loro composizione non è libera. Il *riordino* si compone con qualunque strategia di rimozione, perché non sottrae regole e non consuma budget di veto; la *rimozione garantita* si compone con l'euristica, ma il suo effetto si somma a quello dell'euristica soltanto se le due agiscono su regole diverse. `score-argmin` non si compone con le altre strategie di rimozione: la sua garanzia riguarda la generazione e vale rispetto all'insieme di partenza, sicché una rimozione successiva può invalidare l'argomento del minimo che essa ha calcolato staticamente. È la ragione per cui la combinazione impiegata nelle campagne è formata da un riordino e da una sola strategia di rimozione euristica, e non da più rimozioni in cascata.

3.5 Perché la generazione non ammette una condizione esatta della stessa natura

La condizione [V2] sfrutta una proprietà logica del veto: se una regola ne implica un'altra, la seconda è superflua. Per la generazione una proprietà analoga non sussiste, e la ragione è strutturale: il valore imputato dipende dal *miglior punteggio* fra i candidati di un cluster, e il miglior punteggio dipende dall'insieme delle regole presenti. Rimuovere una regola può quindi cambiare l'argomento del minimo anche quando quella regola non è mai essa stessa il minimo. La generazione è perciò una funzione dell'*insieme*, non un insieme di contributi indipendenti, e nessuna proprietà locale di una singola regola può certificare che la sua assenza non alteri l'esito.

Da qui la distinzione fra la condizione `score-argmin`, che preserva la generazione *calcolando* staticamente l'argomento del minimo sul dataset — quindi pagando un costo

di preprocessing proporzionale alla dimensione dei dati — e le strategie euristiche, che rinunciano a questa proprietà in cambio di riduzioni maggiori. La stessa asimmetria fra i due ruoli è la ragione per cui, come dimostra il capitolo 4, una potatura che operi sul solo file delle regole non possa superare la classe delle regole coperte e delle regole inerti senza rinunciare a una garanzia.

3.6 Integrazione con il sistema: il contratto di potatura

Preprocessing esterno. La potatura si inserisce fra l'estrazione delle RFD e l'esecuzione del sistema: il file delle regole viene riscritto secondo la strategia scelta, e il sistema viene eseguito sul file risultante. Il dataset non è modificato, e l'eseguibile non è toccato. La scelta ha il pregio di non richiedere alcuna modifica al sistema — che resta quello originale, verificato per checksum — ed è stata mantenuta come vincolo per l'intera durata del lavoro; la sezione A.3 mostra che il vincolo è stato rimosso per una parte dell'analisi, ma le strategie di potatura operano tutte nella forma esterna.

Politica sui veti. Poiché la rimozione di una regola può sacrificarne il potere di veto, ogni strategia di rimozione è accompagnata da una politica esplicita, scelta fra tre:

- `protect`: ogni regola in grado di esercitare un veto viene ripristinata. La riduzione si annulla, ma la garanzia è totale;
- `budget:F`: è consentito rimuovere una frazione F delle regole portatrici di veto, scelte secondo un ordinamento di rischio;
- `allow`: nessuna tutela. La riduzione è massima e il potere di veto è esplicitamente sacrificato.

Le rimozioni il cui veto è *provato coperto* dalla condizione [V2] sono esenti dal computo del budget: essendo prive di costo in termini di veto, addebitarle costituirebbe una doppia contabilizzazione. Questa precisazione, insieme a un difetto di correttezza della prima implementazione, è discussa nel capitolo 9.

Invarianti del contratto. Il contratto di potatura è soggetto a tre invarianti, verificate automaticamente dall'ambiente di valutazione. Il file prodotto deve essere un sottoinsieme del file di partenza con lo stesso formato, e il dataset non deve essere alterato; le righe devono conservare contenuto e ordinamento relativo, così che una strategia di riordino sia distinguibile da una di rimozione; se il file prodotto coincide con quello di partenza, il confronto è *vacuo* e va segnalato anziché conteggiato come esito nullo. La terza invariante è quella che nella pratica si è rivelata più utile, perché un numero non trascurabile di configurazioni produce una potatura vuota, e trattarle come evidenza di assenza di effetto avrebbe alterato le conclusioni.

–4–
Teoria: che cosa si può potare con garanzia

Questo capitolo formula e dimostra il risultato negativo che delimita il campo di applicabilità della potatura esterna. Dopo aver fissato che cosa si intende per comportamento del sistema, per attivabilità di una regola e per rimozione certificata — e dopo aver illustrato le definizioni su un caso minimo — il capitolo enuncia il teorema e lo dimostra, ne trae la conseguenza sul target di riduzione separando ciò che è dimostrato da ciò che è misurato, mostra che la classe certificata non è vuota e che il limite non dipende dal parametro di ricerca dei copritori, e ne precisa infine la portata, indicando che cosa resta fuori dal risultato.

4.1 Impostazione formale

Definizione 12 (Sistema e comportamento)**.** Sia D un dataset con schema $\mathcal{A}$ e sia R un insieme finito di RFD. Si indica con $B(R, D)$ il *comportamento* del sistema su D con l'insieme di regole R: l'insieme delle celle valorizzate e i valori loro assegnati. Due insiemi di regole R e R' sono *equivalenti* su D se $B(R, D) = B(R', D)$.

Il comportamento è l'osservabile del sistema: nella notazione dell'implementazione corrisponde ai file di output delle celle popolate e dei risultati. La nozione di equivalenza è quella che gli esperimenti del capitolo 7 verificano per confronto di impronte.

Definizione 13 (Attivabilità di una regola)**.** Sia $\mathrm{NullCols}(D)$ l'insieme delle colonne di D che presentano almeno una cella nulla. Una RFD r è *attivabile in veto* su D se $\mathrm{LHS}(r) \cap \mathrm{NullCols}(D) \neq \emptyset$; è *attivabile in generazione* su D se $\mathrm{RHS}(r) \in \mathrm{NullCols}(D)$.

Le due condizioni sono di natura diversa. La prima è necessaria e sufficiente, ed è la condizione [V1] della sezione 3.2: la verifica di coerenza confronta valori, e il confronto richiede che entrambi siano presenti. La seconda è necessaria ma non sufficiente: perché una regola generi un candidato occorre anche che esista, nella colonna di destra, un cluster con cui confrontare la tupla incompleta. Ai fini del teorema è però la sola necessità che serve, perché è la possibilità di attivazione a escludere la certificabilità.

Definizione 14 (Potatura esterna)**.** Una *strategia di potatura esterna* è una funzione Π che, applicata a una coppia (R, D), produce un insieme $\Pi(R, D) \subseteq R$ usando esclusivamente il file delle regole, lo schema di D e il profilo delle colonne nulle $\mathrm{NullCols}(D)$. In particolare Π non accede al comportamento del sistema né può osservare quali regole siano attivate in esecuzione.

La restrizione è quella imposta dal capitolo 3: la potatura riscrive il file e non modifica l'eseguibile. È una restrizione reale, non una comodità dimostrativa: è ciò che rende la metodologia applicabile a un sistema distribuito senza sorgenti.

Definizione 15 (Rimozione certificata)**.** Una rimozione $r \in R \setminus \Pi(R, D)$ è *certificata* se esiste una condizione Φ, decidibile con l'informazione ammessa dalla Definizione 14, tale che per ogni dataset D' con lo schema e il profilo delle colonne nulle di D valga

$$\Phi(r, R, D) \implies r \text{ non è attivabile in alcuno dei due ruoli su } D',$$

oppure

$$\Phi(r, R, D) \implies \text{il veto di } r \text{ su } D' \text{ è implicato da una regola di } \Pi(R, D).$$

Una strategia è certificata se tutte le sue rimozioni lo sono.

La definizione merita una precisazione. La certificazione non è richiesta rispetto al solo dataset in esame, ma rispetto a ogni dataset che con esso condivide schema e profilo di valori mancanti: è questa quantificazione a dare forza al risultato, ed è anche la ragione per cui la potatura esterna non può giovarsi di circostanze accidentali. Una regola la cui rimozione non altera il comportamento *su questo* dataset può benissimo alterarlo su un altro con lo stesso profilo di nullità, e una strategia che operi senza eseguire il sistema non ha modo di distinguere i due casi. La sezione 4.5 mostra che la distinzione è verificabile a posteriori, e che le rimozioni osservate come innocue ricadono comunque nelle classi certificate.

4.2 Un esempio minimo

Le tre condizioni si leggono meglio su un caso concreto. Si consideri uno schema con tre attributi A, B, C, un profilo di valori mancanti $\mathrm{NullCols} = \{A, C\}$ e quattro regole; la Tabella 4.1 le classifica.

Tabella 4.1: Esempio minimo di classificazione delle regole. La tabella è illustrativa: serve a rendere leggibili le definizioni della sezione precedente, non riporta misure.

regola	espressione	classe	rimozione certificabile
r_2	$(A \le 3) \to (C \le 0)$	né inerte né coperta; è la copritrice di r_1	no: è la regola che copre
r_1	$(A \le 1) \to (C \le 0)$	[V2]: coperta da r_2	sì, purché r_2 sopravviva
r_3	$(B \le 1) \to (C \le 0)$	né inerte né coperta	no: può generare candidati
r_4	$(B \le 1) \to (B \le 0)$	[V3]: inerte	sì, per costruzione

Tre aspetti dell'esempio meritano di essere evidenziati, perché sono quelli che rendono la condizione [V2] controintuitiva. Il primo è la direzione delle soglie: r_2

copre r_1 pur essendo *più permissiva* a sinistra (3 contro 1) e non più debole a destra (la soglia di destra è la stessa); nella nozione classica di dominanza fra dipendenze si richiederebbe l'opposto. Il secondo è che r_1 è rimovibile solo *finché* r_2 sopravvive: la condizione si valuta sull'insieme corrente delle regole, non su quello di partenza, ed è la precisazione che il sottosezione 9.3.1 ha reso necessaria. Il terzo è che r_3 — che pure ha un lato sinistro senza valori mancanti, e quindi non può esercitare alcun veto — *non* è rimovibile senza rinunciare alla garanzia, perché la sua colonna di destra contiene celle nulle e la regola può quindi generare candidati: è l'asimmetria fra i due ruoli che il teorema generale formalizza.

4.3 Il teorema negativo

Teorema 16 (Limite della potatura esterna certificata)**.** *Siano D un dataset, R un insieme di RFD e Π una strategia di potatura esterna in possesso della certificazione della Definizione 15. Allora ogni regola $r \in R \setminus \Pi(R, D)$ è provabilmente inerte secondo la condizione* [V3]*, oppure il suo veto è implicato da una regola sopravvissuta.*

Il teorema non afferma che le regole fuori da queste due classi non possano essere rimosse senza effetti: afferma che la loro rimozione non può essere *certificata* con l'informazione disponibile a una strategia esterna. La distinzione fra le due affermazioni è il contenuto dell'ultima sezione del capitolo.

4.4 Dimostrazione

Dimostrazione. Sia $r \in R \setminus \Pi(R, D)$ una regola rimossa, e si supponga che r non sia inerte secondo la Definizione 11, cioè che valga

$$\mathrm{LHS}(r) \cap \mathrm{NullCols}(D) \neq \emptyset \quad \vee \quad \mathrm{RHS}(r) \in \mathrm{NullCols}(D).$$

Si distinguono i due casi.

Primo caso: $\mathrm{RHS}(r) \in \mathrm{NullCols}(D)$. Per la Definizione 13 la regola è attivabile in generazione: esiste almeno una cella nulla nella colonna di destra, e la regola partecipa quindi all'insieme dei candidati per quella cella. Il valore che il sistema scrive dipende dall'argomento del minimo fra i candidati del cluster, e l'argomento del minimo dipende dall'insieme delle regole presenti (sezione 3.5); nessuna proprietà della singola regola r, decidibile dal file, determina se la sua presenza cambi l'esito. La condizione Φ non può quindi stabilire la prima delle due alternative della Definizione 15.

Secondo caso: $\mathrm{LHS}(r) \cap \mathrm{NullCols}(D) \neq \emptyset$. Per la condizione [V1], che è necessaria e sufficiente, la regola è attivabile in veto: esistono un dataset D' con lo schema e il profilo di nullità di D e una coppia di tuple su cui la verifica di coerenza di r viene eseguita e fallisce. Se r non ricade nel primo caso, la prima alternativa della Definizione 15 è esclusa anche qui, e la certificazione può venire soltanto dalla seconda: il veto di r deve essere implicato da una regola che sopravvive alla potatura.

Dunque ogni rimozione certificata è una regola inerte oppure una regola il cui veto è implicato da una regola sopravvissuta. □

La dimostrazione poggia su due fatti stabiliti altrove: l'esattezza della condizione [V1], che discende dalla semantica del confronto fra valori, e la dipendenza della generazione dall'argomento del minimo, che discende dall'analisi delle due procedure di riempimento. Il primo è un fatto logico; il secondo è una proprietà dell'algoritmo, verificata sul codice e confermata dal fatto, misurato nella sezione 7.1, che le strategie di rimozione che non tutelano la generazione producono output diversi.

4.5 Conseguenza sul target di riduzione

Il teorema si traduce in un vincolo quantitativo solo una volta stabilito quali regole, fra quelle certificabili, siano *riconosciute* come tali dalla metodologia. La condizione [V2] è una condizione *sufficiente* di copertura del veto: quando è soddisfatta, la regola è eliminabile con garanzia. Non è detto che sia l'unica condizione sufficiente possibile, ed è opportuno distinguere le due affermazioni.

Corollario 17 (Incompatibilità del target)**.** *Se la copertura del veto è decisa mediante la condizione* [V2]*, allora la riduzione ottenibile da una potatura esterna certificata è limitata dalla dimensione dell'unione delle classi* [V2] *e* [V3].

La classe certificata non è vuota. Un teorema di impossibilità va difeso dalla prima obiezione che riceve, cioè che la classe su cui il risultato morde sia vuota. Non lo è: sui dataset esaminati la condizione [V3] individua poche regole — 6 su Bridges a soglia 9 — ma la condizione [V2] ne individua 1 531, pari al 17,2 % dell'insieme (Tabella 3.1), e la riduzione ottenibile cresce con la soglia fino al 20,9 %. La condizione è inoltre *decidibile* senza eseguire il sistema: la ricerca dei copritori della 2 esamina al più tutte le coppie di regole con la stessa colonna di destra, ed è quindi di ordine quadratico nella dimensione del file, con il parametro μ a limitare il numero di candidati esaminati per ciascuna regola.

La Tabella 3.1 fornisce la misura di quell'unione sui dataset impiegati: essa vale dall'0,0 % al 20,9 %, con il massimo raggiunto da Bridges alla soglia più alta. Ne segue che un obiettivo di riduzione pari o superiore al 50 % è incompatibile con la garanzia, e che l'incompatibilità non dipende dalla strategia adottata ma dalla struttura delle classi. La precisazione è necessaria perché il corollario è una proposizione *misurata* e non un teorema: la limitazione è dimostrata, la sua entità numerica è osservata su otto casi, e nulla esclude che su altri dataset la classe [V2] sia più ampia. Ciò che il teorema esclude è che una via diversa dalla copertura del veto possa essere percorsa *con garanzia*; ciò che resta aperto è quanto la copertura del veto possa valere su dati diversi da quelli esaminati.

4.6 Quanto è stretto il limite

La ricerca delle regole coprenti impiega un parametro che limita la differenza di ampiezza fra il lato sinistro della regola coprente e quello della regola coperta. Il parametro è una scelta di ricerca, non una condizione di correttezza: la Theorem 9 vale per qualunque valore. Si è quindi verificato se il tetto di riduzione osservato fosse

un artefatto del valore adottato. Lo sweep completo — valori da 3 a 20, su otto casi — mostra che la riduzione è *piatta*: su 8 939 rimozioni complessive, allargare la ricerca al massimo ne aggiunge *una*. Il limite è pertanto intrinseco al criterio, e il teorema negativo non è aggirabile per questa via. Il ricalcolo della tabella, dai soli file di regole, è in `pruning/verifica_tetto.py`; la prova su RENUVER del caso con la rimozione in più è raccolta in `results/results_maxextra_bridges15.json`.

4.7 Portata del risultato

Tre restrizioni delimitano il teorema, e conviene dichiararle esplicitamente perché ciascuna indica una direzione di lavoro.

Solo rimozione. La Definizione 14 considera trasformazioni che producono un sottoinsieme del file di partenza. Una trasformazione che *riscriva* le regole — restringendo o allargando i lati sinistri, modificando le soglie, fondendo più regole in una — non è coperta dal risultato, e non è detto che vi ricada: la condizione [V2] mostra del resto che una regola può essere eliminata quando il suo contenuto informativo è conservato da un'altra, e nulla vieta di cercare la stessa conservazione per via sintattica anziché per soppressione. È, questa, la direzione di sviluppo più promettente indicata nelle sezione 9.4.

Solo informazione esterna. La certificazione richiesta è quella ottenibile dal file, dallo schema e dal profilo dei valori mancanti. Una strategia che eseguisse il sistema, o che disponesse di una sua descrizione simbolica, potrebbe in linea di principio certificare rimozioni ulteriori: è il caso di `score-argmin`, che calcola staticamente l'argomento del minimo sul dataset e preserva la generazione, ma non il veto, e resta quindi fuori dalla classe garantita (sezione 7.9).

Completezza della condizione di copertura. Il teorema caratterizza le rimozioni certificabili, non le rimozioni innocue. Stabilire se la condizione [V2] esaurisca le regole il cui veto è implicato da un'altra regola è un problema aperto, e la sua soluzione non cambierebbe la natura del risultato ma potrebbe ampliarne la portata quantitativa. La sezione 7.1 fornisce, su questo punto, un'evidenza indiretta: le rimozioni effettivamente osservate come innocue ricadono tutte nelle classi certificate, e la copertura verificata a posteriori coincide con quella prevista.

La riduzione interna del sistema. La certificazione è definita rispetto all'insieme delle regole contenute nel file, ma il sistema ne consuma un sottoinsieme determinato da un criterio *proprio*: all'avvio esso rimuove le regole a soglia di destra nulla il cui lato sinistro si comporta come una chiave e le destina a una procedura distinta (sezione 2.5). Due osservazioni ne seguono. La prima è che questa riduzione interna non è modellata dal teorema: le classi [V2] e [V3] sono calcolate sul file, e nulla garantisce che una regola coprente sopravviva alla riduzione interna. La seconda è che la garanzia resta comunque valida sui dati esaminati, perché la proprietà di chiave implica che il veto della regola coprente e quello della regola coperta siano entrambi non esercitabili: se

il lato sinistro della coprente è una chiave, ogni suo soprainsieme — e quindi anche il lato sinistro della coperta — lo è, e nessuna coppia di tuple distinte può attivare la verifica. La riduzione interna non può dunque sottrarre un veto che il teorema presume implicato; resta il fatto che il modello formale non la rappresenta, e che una sua inclusione renderebbe l'enunciato più generale.

–5–
Ipotesi e criteri di falsificazione

Questo capitolo documenta il processo che ha condotto dalle osservazioni iniziali alla formulazione delle ipotesi. La sua inclusione risponde a una ragione metodologica: il lavoro non è consistito nell'applicare una tecnica nota a un sistema noto, ma nel ricostruire il comportamento di un sistema di cui non erano disponibili i sorgenti. Le ipotesi hanno quindi svolto una funzione euristica, e la loro falsificazione è parte del risultato. Il capitolo ricostruisce il processo di analisi, enuncia le ipotesi primarie con la loro derivazione e il criterio di falsificazione — senza anticiparne l'esito, che è discusso insieme ai risultati — e raccoglie separatamente le domande esplorative. Il protocollo di verifica e la metodologia statistica, che delle ipotesi costituiscono il banco di prova, sono esposti nel capitolo sulla valutazione sperimentale.

5.1 Come sono nate le ipotesi

L'analisi si è svolta in quattro passi, ciascuno dei quali ha determinato il successivo.

1. **Disassemblaggio.** Il sistema è distribuito senza sorgenti. La ricostruzione della logica è avvenuta per disassemblaggio del bytecode, con particolare attenzione alle procedure che consumano l'insieme delle regole.

2. **Individuazione del doppio ruolo.** È emerso che una RFD partecipa a due procedure con requisiti opposti (sezione 2.5). Questa osservazione ha trasformato il problema da "selezionare le regole utili" a "selezionare le regole utili sapendo che *utile* ha due significati incompatibili".

3. **Derivazione delle condizioni.** Dal comportamento delle due procedure sono state ricavate le condizioni [V1], [V2], [V3] e il teorema negativo che ne segue (capitolo 4).

4. **Verifica sperimentale.** Le condizioni sufficienti sono state verificate su stati reali; le strategie euristiche sono state misurate su più famiglie di dataset, con particolare attenzione a *dove* esse funzionano e dove no.

Un elemento del processo merita di essere esplicitato, perché ha modificato il corso del lavoro. L'analisi del passo 3 ha prodotto un risultato negativo, il teorema

della sezione 4.3, che esclude la possibilità di raggiungere l'obiettivo di riduzione mantenendo la garanzia. Una parte non trascurabile del lavoro è consistita nel *verificare* che quel risultato negativo non fosse un artefatto dell'implementazione, prima di accettarlo come limite del problema (sezione 4.6).

Origine delle ipotesi. Il lavoro è esplorativo: le ipotesi non precedono l'analisi ma nascono da essa, e la circostanza va dichiarata perché incide sul valore probatorio dei test. La Tabella 5.1 distingue perciò, per ogni ipotesi, l'*origine*: le ipotesi *derivate* discendono da una proprietà stabilita prima di osservare i dati sperimentali — una condizione dimostrata, una misura di struttura, l'analisi del costo — e la loro previsione è argomentabile a priori; le ipotesi *esplorative* sono state formulate durante l'analisi, e il loro esito va inteso come indicazione per ulteriori verifiche e non come conferma indipendente. Nessuna delle ipotesi primarie è stata formulata dopo aver visto l'esito dell'esperimento che la verifica; la distinzione è mantenuta anche nel capitolo dei risultati, dove gli esiti sono riportati per ciascuna.

5.2 Ipotesi primarie

Le ipotesi primarie sono sette e corrispondono alle domande che il lavoro pone: che cosa si può potare con garanzia, che cosa si può preservare senza garanzia, se esista una leva di qualità, se tale leva generalizzi, da che cosa dipenda il guadagno quando si manifesta, e che rapporto vi sia fra riduzione e tempo. La Tabella 5.1 le enuncia con la previsione e il criterio di falsificazione; l'esito non è qui riportato.

H1 — La strategia esatta lato generazione. `score-argmin` preserva la generazione dei candidati per costruzione, perché calcola staticamente l'argomento del minimo che il sistema sceglierebbe. Ci si attende pertanto che, associata a una politica che tuteli integralmente il potere di veto, essa produca un output identico a quello della configurazione di partenza, e che la riduzione residua sia ancora significativa: la politica di tutela ripristina le regole portatrici di veto, ma non quelle la cui rimozione è provata priva di costo dalla condizione [V2], e la classe [V2] vale fino a poco più di un quinto dell'insieme (Tabella 3.1). La previsione è una riduzione non inferiore al 30 %. L'ipotesi è derivata: entrambe le premesse — l'esattezza lato generazione e la dimensione della classe coperta — sono stabilite prima della misura.

H6 — La classe garantita. Se la condizione [V2] è corretta, la sua applicazione deve essere invisibile al sistema: le regole rimosse hanno un veto implicato da una regola sopravvissuta, e la proposizione della Theorem 9 assicura che nessun veto venga perso. La previsione è che l'output coincida con quello della configurazione di partenza su *tutte* le configurazioni della griglia, non soltanto su quelle di una famiglia. È l'ipotesi con il criterio di falsificazione più severo, perché una sola configurazione difforme la falsifica.

H7 — Verifica indipendente della copertura. La correttezza della condizione [V2] è stabilita per via logica, ma la sua *implementazione* potrebbe essere difettosa:

Tabella 5.1: Ipotesi primarie, previsione, criterio di falsificazione e sezione in cui ciascuna è risolta. L'origine — derivata da una proprietà stabilita prima della misura, oppure formulata durante l'analisi — è dichiarata nella derivazione che segue alla tabella.

#	ipotesi	previsione	falsificazione	risolta in
H1	`score-argmin` con veti protetti	riduzione $\geq$ 30 % con output identico	riduzione inferiore alla soglia	7.8
H6	`veto-cover` su tutte le famiglie	output identico alla configurazione di partenza	una sola configurazione con output diverso	7.1
H7	verifica empirica di [V2]	zero discrepanze fra copertura prevista e verificata	una sola discrepanza	7.1
H8	`precision-priority` da solo	guadagno di qualità	nessun guadagno o perdita	6.8
H11	`precision-priority` su Glass e Restaurant	guadagno anche fuori da Bridges	perdita su entrambe le famiglie	7.4
H14	attribuzione del guadagno alla precisione dei cluster	un riordino casuale non produce lo stesso guadagno	il riordino casuale produce lo stesso guadagno	7.8
H16	proporzionalità fra riduzione e tempo	riduzioni maggiori producono tempi minori	una configurazione con riduzione maggiore e tempo non inferiore	7.9

la ricerca dei copritori è un algoritmo, e un algoritmo può sbagliare. La previsione è che, controllando a posteriori su stati reali del dataset che ogni regola rimossa abbia effettivamente un copritore sopravvissuto che esercita il veto sugli stessi testimoni, non emerga alcuna discrepanza. La verifica è indipendente perché non riusa il codice della condizione ma ricostruisce i testimoni.

H8 — La leva di qualità. L'analisi del sistema mostra che l'esito di un'imputazione dipende dall'ordine in cui i cluster vengono tentati (sezione 2.3), e che la posizione di una regola nell'ordine di scansione non richiede la rimozione di alcunché. Ne segue la previsione che un riordino che disponga per prime le regole con la precisione empirica più alta migliori il risultato senza toccare il potere di veto. L'ipotesi è derivata dalla struttura delle due procedure, non da un'osservazione sui risultati.

H11 — Generalizzazione della leva di qualità. Se il meccanismo del guadagno è la composizione dei candidati e non una proprietà di Bridges, lo stesso riordino deve produrre un guadagno anche sulle altre famiglie. La previsione è che il guadagno si manifesti su Glass e su Restaurant; la falsificazione è una perdita su entrambe. L'ipotesi è deliberatamente severa: è la generalizzazione a decidere se il guadagno osservato su una famiglia sia un fenomeno del sistema o un accidente del dataset.

H14 — Attribuzione del guadagno. Il riordino per priorità di precisione impiega un segnale — la precisione empirica dei cluster — e la spiegazione naturale del guadagno è che quel segnale sia la causa. Se la spiegazione è corretta, allora un riordino casuale non deve produrre lo stesso guadagno: la previsione è che il guadagno del riordino informato superi in modo sistematico quello di permutazioni con seme fissato. Si tratta di un'ipotesi di *confutazione di una spiegazione* più che di verifica di una strategia, ed è classificata come esplorativa perché formulata dopo aver osservato il guadagno di cui cerca la causa.

H16 — Rapporto fra riduzione e tempo. L'analisi del costo (sezione 2.6) mostra che il tempo è assorbito dalla verifica dei veti, che esamina l'insieme delle regole per ogni candidato: se il costo per candidato è proporzionale al numero di regole, la riduzione dell'insieme deve tradursi in una riduzione del tempo. La struttura del sistema suggerisce però una previsione opposta: il veto non si limita a costare, ma *scarta* i candidati, e un insieme di regole più povero può lasciare in gara più alternative e quindi moltiplicare le verifiche. L'ipotesi assume la prima previsione; la seconda è la ragione per cui il criterio di falsificazione è enunciato in forma stretta — una sola configurazione con riduzione maggiore e tempo non inferiore falsifica l'ipotesi — e non come semplice assenza di correlazione.

5.3 Domande esplorative

Le restanti ipotesi non hanno il carattere di previsione argomentabile a priori: riguardano varianti di parametri, strategie alternative e proprietà di portata locale, e sono state formulate nel corso dell'analisi. Sono raccolte separatamente, senza esito,

perché il loro ruolo è delimitare lo spazio delle spiegazioni e indicare che cosa non funziona, non confermare una previsione. La Tabella 5.2 le elenca con la domanda a cui rispondono.

Tabella 5.2: Domande esplorative, con la sezione in cui ciascuna è risolta. Sono formulate durante l'analisi; il loro esito è riportato nella sezione 8.1 e non ha valore di conferma indipendente.

#	domanda	risolta in
H2	`score-argmin` preserva la copertura anche su Glass e Restaurant?	7.8
H3	esiste una strategia di copertura esatta delle celle, e a costo nullo?	7.8
H4	la conversione delle regole di tipo `B` in tipo `D` produce un guadagno gratuito?	9.2.1
H5	il riordino delle righe del dataset produce un guadagno?	7.8
H9	riordino e potatura sommano i loro effetti?	6.8
H10	una regolarizzazione della stima delle precisioni produce un guadagno stabile?	7.8
H12	la versione regolarizzata generalizza meglio delle altre?	7.8
H13	il riordino entro il cluster è una leva di qualità?	7.8
H15	l'ordinamento delle rimozioni sotto vincolo di budget è migliorabile?	7.8

–6–
Valutazione sperimentale

Questo capitolo descrive la valutazione sperimentale. Vi sono esposti il protocollo con cui le ipotesi formulate nel capitolo precedente sono tradotte in confronti controllati e la metodologia statistica adottata per sintetizzarli; l'infrastruttura materiale su cui le campagne si fondano; i dataset, la griglia delle configurazioni e le metriche con cui i risultati sono misurati; il termine di paragone banale che dà una scala ai valori assoluti; gli esperimenti condotti, con il collegamento esplicito alle ipotesi che essi verificano; e lo studio di ablazione che isola il contributo dei singoli componenti della strategia principale. L'effetto della potatura sul tempo di esecuzione, che non è una proprietà delle configurazioni ma un esito, è discusso insieme ai risultati.

6.1 Protocollo di verifica

Il protocollo di verifica si fonda su cinque scelte, adottate per evitare errori di interpretazione che si erano effettivamente presentati nel corso del lavoro.

Confronto appaiato. Ogni configurazione potata è confrontata con la medesima configurazione non potata, sullo stesso dataset, la stessa esecuzione e la stessa soglia. Il confronto fra medie aggregate su configurazioni diverse è stato evitato perché sensibile alla composizione del campione.

Unità di analisi dichiarata. L'unità di analisi è la *configurazione*, cioè la tripletta dataset, soglia e ripetizione, e non la cella nulla. La scelta è motivata dalla struttura del sistema: le imputazioni di una stessa esecuzione non sono indipendenti, perché condividono l'ordine delle regole e i cluster, e trattarle come osservazioni separate moltiplicherebbe artificialmente la numerosità campionaria. Ne segue che la numerosità dei test è quella delle configurazioni — decine, non migliaia — e che la potenza statistica è conseguentemente limitata.

Controllo della vacuità. Il confronto è privo di contenuto in due circostanze distinte. La prima è che il file di regole prodotto dalla potatura sia identico a quello di partenza: la potatura non ha fatto nulla e non vi è nulla da misurare. La seconda, meno evidente, è che la configurazione di partenza non produca *alcuna* imputazione: la potatura

non è vacua, ma il confronto lo è, perché tutte le grandezze di qualità valgono zero in entrambi i bracci. Il sistema di valutazione segnala entrambe le circostanze, e le configurazioni corrispondenti sono escluse dai test e contate a parte: trattarle come esiti nulli introdurrebbe una massa di zeri artificiale, e le tratterebbe come evidenza di assenza di effetto.

Controllo casuale. Per attribuire un effetto a un ordinamento — e non al mero atto di riordinare — è necessario un termine di paragone casuale. Sono state impiegate permutazioni con seme fissato, in numero di tre e indipendenti fra loro; il confronto è condotto sia contro ciascuna permutazione, sia contro la loro media per configurazione.

Verifica indipendente delle condizioni. Le condizioni teoriche sono verificate su stati reali del dataset, non soltanto per implicazione logica: per ogni regola rimossa dalla condizione [V2] si controlla che una regola coprente sopravvissuta eserciti effettivamente il veto sugli stessi testimoni.

6.2 Metodologia statistica

Il confronto fra configurazione potata e configurazione di partenza è un confronto appaiato su variabili che non si distribuiscono in modo normale — il numero di imputazioni corrette è una conta con molti valori estremi e molte differenze nulle — e il test adottato è quindi il test dei ranghi con segno di Wilcoxon [17].

Il test. Per ogni confronto si considerano le differenze $\Delta_i = B_i - A_i$ fra la variante e la configurazione di partenza sulla medesima configurazione, si scartano le differenze nulle e si ordinano i valori assoluti. La statistica è la somma dei ranghi delle differenze positive, W^+. Quando non vi sono ranghi medi e il numero di differenze non nulle non supera venticinque, la distribuzione nulla è calcolata in forma *esatta* per enumerazione; negli altri casi si impiega l'approssimazione normale con correzione di continuità e correzione per i ranghi medi. Il test è a due code. Il numero di differenze non nulle, e non il numero di configurazioni, è la numerosità effettiva del test: le colonne n e n_{test} della Tabella 6.1 li riportano entrambi.

Dimensione dell'effetto. La significatività non dice quanto grande sia l'effetto, e con numerosità dell'ordine di alcune decine è possibile ottenere valori di p piccoli per differenze di una sola unità. Si riporta quindi la *differenza mediana* — robusta rispetto agli estremi — e la correlazione rank-biserial per campioni appaiati, $r = (W^+ - W^-)/(W^+ + W^-)$, che è l'analogo non parametrico della d di Cohen: valori assoluti intorno a 0.1, 0.3 e 0.5 indicano effetto piccolo, medio e grande.

Intervalli di confidenza. Per la differenza mediana si riporta un intervallo di confidenza al 95 % ottenuto per ricampionamento delle coppie con reimmissione (diecidimila replicazioni, seme fissato), con il metodo dei percentili. L'intervallo è ampio per costruzione, data la numerosità, ed è riportato per rendere esplicita l'incertezza invece di affidare la lettura alla sola soglia di significatività.

Confronti multipli. Il lavoro verifica sette ipotesi primarie su più metriche e decine di configurazioni, e la probabilità di almeno un falso positivo cresce con il numero di test. I test che costituiscono la famiglia primaria — quelli su cui poggiano le conclusioni del capitolo 7 — sono cinque, dichiarati in `pruning/stats_effects.py`, e per essi si riporta il valore di p corretto con la procedura di Holm [18], che controlla la probabilità di almeno un rifiuto errato sull'intera famiglia. La correzione è conservativa e va letta come tale: il suo effetto è di alzare la soglia, non di abbassarla.

Tabella 6.1: Test della famiglia primaria. Δ è la differenza fra variante e configurazione di partenza, r la correlazione rank-biserial, l'intervallo è quello di confidenza al 95 % della differenza mediana. p è il valore grezzo, p_{Holm} quello corretto per i cinque test della famiglia.

confronto	n	n_{test}	mediana Δ [IC 95 %]	r	p / p_{Holm}
`A5b` su Bridges, corrette	50	39	+0.5 [0.0, +2.0]	+0.510	0.0053 / 0.0159
`A5b` su Bridges, errori	50	35	−1.0 [−2.0, 0.0]	−0.905	$2.5 \cdot 10^{-6}$ / $1.3 \cdot 10^{-5}$
taratura conservativa, corrette	10	0	0.0 [0.0, 0.0]	0.000	1.0000 / 1.0000
ordinamento, primo contro originale	15	13	+2.0 [+1.0, +5.0]	+1.000	0.0016 / 0.0064
ordinamento, primo contro casuale	15	8	0.0 [0.0, +0.7]	+0.083	0.8878 / 1.0000

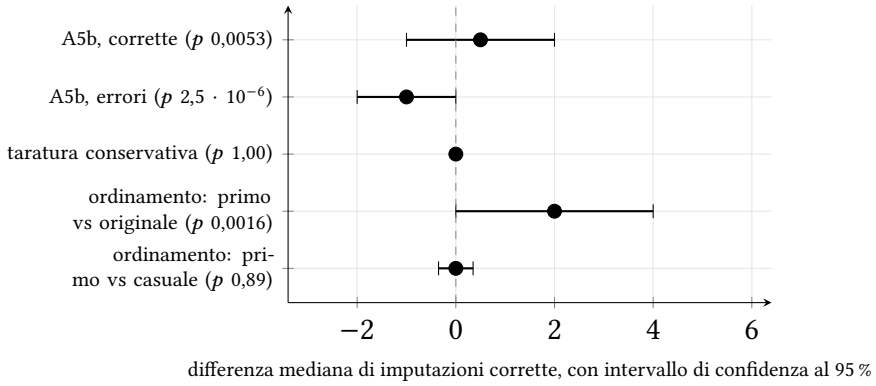


Figura 6.1: Test della famiglia primaria: differenza mediana di imputazioni corrette e intervallo di confidenza al 95 % ottenuto per ricampionamento. I valori di p sono quelli grezzi; quelli corretti con la procedura di Holm sono nella Tabella 6.1. La linea verticale indica l'assenza di differenza.

La Tabella 6.1 si legge in due modi. Sul piano della significatività, i due test sulla potatura aggressiva e quello sull'ordinamento sopravvivono alla correzione per confronti multipli; il confronto fra riordino informato e riordino casuale non è significativo, ed è il risultato che falsifica l'ipotesi H14, non un test fallito. Sul piano della dimensione dell'effetto, i due test che sopravvivono hanno valori di r medio e grande, mentre gli altri hanno effetti trascurabili o nulli: la distinzione conta perché una differenza mediana di mezza imputazione, pur significativa su cinquanta configurazioni, non è un guadagno utilizzabile, ed è la ragione per cui il capitolo 7 riporta le somme delle differenze e non soltanto i valori di p.

Due limiti della metodologia statistica vanno dichiarati. Il primo è che la famiglia primaria è stata dichiarata *dopo* la raccolta dei dati, sebbene prima del calcolo dei test: non vi è quindi pre-registrazione, e la correzione di Holm protegge dalla molteplicità

ma non dalla selezione dei confronti. Il secondo è la potenza: con alcune decine di configurazioni, differenze piccole rispetto alla variabilità fra dataset non sono distinguibili dal caso, e l'assenza di significatività non va letta come assenza di effetto.

6.3 Setup sperimentale

Infrastruttura. La valutazione si fonda su due strumenti sviluppati per il lavoro: un *pruner* esterno, che riscrive il file delle regole secondo la strategia richiesta, e un *benchmark* che esegue il sistema nella configurazione di partenza e in quella potata, raccoglie le metriche e le confronta. Il sistema oggetto di studio non è mai stato modificato durante la valutazione esterna: l'eseguibile impiegato è quello originale, verificato per checksum all'inizio e alla fine delle campagne.

Isolamento delle esecuzioni. Ogni esecuzione avviene in una cartella di lavoro dedicata, che viene ripulita prima dell'esecuzione successiva. Le esecuzioni sono condotte in sequenza, mai in parallelo: la cartella di lavoro è condivisa e due esecuzioni simultanee sovrascriverebbero reciprocamente gli output. La circostanza è stata verificata sperimentalmente e ha richiesto di rieseguire alcune configurazioni.

Riproducibilità. Tutti i numeri presentati nel capitolo 7 sono ricalcolabili dai file di risultati prodotti dalle campagne, e la ricalcolabilità è verificata automaticamente: una procedura di verifica ricalcola i valori asseriti nella documentazione e li confronta con quelli memorizzati, segnalando ogni divergenza. La procedura copre 67 asserzioni numeriche, incluse quelle relative alle condizioni teoriche. I dettagli sono nell'Appendice A.

6.4 Dataset e configurazioni

La valutazione impiega tre famiglie di dataset, scelte fra quelle disponibili nell'archivio secondo criteri di utilizzabilità e di diversità strutturale (Tabella 2.1). La Tabella 6.2 riassume la griglia effettivamente coperta.

Tabella 6.2: Griglia sperimentale. "Configurazioni" è il numero di triplette *dataset, run, soglia* distinte.

famiglia	dataset	run	soglie	configurazioni
Bridges	5	1–5	3, 9, 15	75
Glass	5	1–5	3, 9, 15	75
Restaurant	4	1–3	3, 6	22
totale	14			**172**

Le tre famiglie differiscono per caratteristiche che si riveleranno determinanti nell'interpretazione dei risultati: il numero di regole disponibili, la distribuzione delle celle nulle fra tuple con una sola cella nulla e tuple con più celle nulle (sezione 2.3), e la quota di imputazioni che il sistema ottiene per via esatta (sezione 7.5).

Due famiglie dell'archivio non sono state impiegate per l'intera griglia. La famiglia Nursery comprende cinque file di regole — le soglie 3, 6, 9, 12 e 15 — ma nessun dataset e nessun file di tuple iniziali, quindi nessuna configurazione eseguibile: è il residuo di una fase di estrazione di cui non è stato distribuito il dato. La famiglia Physician comprende venticinque configurazioni piene di 103 592 righe, di dimensioni tali da rendere impraticabile una campagna, e quattro versioni remodulate il cui materiale di ground truth presenta indici di riga eccedenti il dataset (capitolo 9). Le versioni remodulate sono state rese eseguibili escludendo le celle non localizzabili, ed è su di esse che si fonda la prova sulla quarta famiglia i cui esiti sono riportati nella sezione 7.7.

6.5 Metriche

Le metriche adottate sono le seguenti. Sia C l'insieme delle celle per cui è disponibile il valore di ground truth; si ricordi che tale insieme non coincide necessariamente con l'insieme delle celle nulle del dataset (sezione 2.4).

Tabella 6.3: Metriche adottate. C è l'insieme delle celle con valore di ground truth disponibile, $I \subseteq C$ l'insieme delle celle imputate, $K \subseteq I$ quelle imputate correttamente.

metrica	definizione	che cosa falsifica
copertura	$\lvert I\rvert/\lvert C\rvert$	una potatura che, togliendo regole, fa rifiutare al sistema celle che prima valorizzava
precisione	$\lvert K\rvert/\lvert I\rvert$	una potatura che rende accettate imputazioni di qualità peggiore
accuratezza	$\lvert K\rvert/\lvert C\rvert$	l'effetto complessivo; è il prodotto delle due precedenti
errori	$\lvert I\rvert - \lvert K\rvert$	l'aumento del danno, che l'accuratezza può mascherare quando la copertura cresce
quota esatta	frazione di I ottenuta per via esatta	la composizione delle imputazioni, primo fattore del meccanismo (sezione 7.5)
preprocessing	secondi per riscrivere il file delle regole	il costo dell'analisi, da confrontare con il risparmio in esecuzione
esecuzione	secondi del sistema, a parità di configurazione	l'effetto sul costo, che non segue la riduzione (sezione 7.9)

La distinzione fra copertura e precisione è essenziale nel seguito. Un metodo basato su vincoli può rifiutare di imputare, e una potatura che rimuova regole può indurre nuovi rifiuti: in quel caso la copertura cala mentre la precisione può salire, e l'accuratezza può restare invariata. La lettura congiunta delle due metriche è quindi necessaria, e le analisi che seguono la rispettano sistematicamente.

Per i dataset a prevalenza testuale (Restaurant) il confronto esatto fra stringhe è troppo severo: due ragioni sociali che differiscono soltanto per la forma societaria finale sono contate come valori diversi pur designando la stessa entità. Per questa

famiglia si adotta pertanto una soglia di similarità normalizzata di Levenshtein pari a 0,8, e le metriche testuali sono riportate separatamente da quelle esatte.

6.6 Termine di paragone banale

Le metriche del capitolo sono differenze rispetto alla medesima configurazione non potata: dicono se la potatura migliora o peggiora il risultato del sistema, non se il risultato del sistema sia buono in assoluto. Per dare una scala, la Tabella 6.4 confronta l'accuratezza del sistema con quella di un imputatore banale che sostituisce a ogni cella la *moda* della colonna, calcolata sui valori osservati. La moda è un termine di paragone volutamente povero — non impiega alcuna dipendenza — ed è calcolata sulle stesse celle valutate dal sistema.

Tabella 6.4: Confronto con un imputatore banale (moda di colonna) sulle stesse celle valutate. La copertura della moda è totale per costruzione. I valori si riferiscono alle configurazioni con schema allineato: 6 configurazioni di Bridges (prima ripetizione, soglie 3 e 9) e le 2 versioni eseguibili di Physician.

famiglia	configurazioni	celle	accuratezza moda	accuratezza sistema	copertura sistema
Bridges	6	224	0,536	0,545	61,6 %
Physician	2	40	0,275	0,150	35,0 %

La lettura del confronto è duplice, e va data per intero. Su Bridges la moda raggiunge un'accuratezza praticamente uguale a quella del sistema (0,536 contro 0,545) con copertura totale: su questo dataset la qualità *assoluta* delle imputazioni non è il punto di forza del sistema, e il valore che il lavoro misura è quello che la potatura aggiunge o toglie a parità di sistema. Su Physician il divario è più netto e in direzione opposta: la moda è migliore del sistema (0,275 contro 0,150), perché il sistema copre il 35 % delle celle e, fra quelle che copre, non sempre indovina.

Due avvertenze delimitano la portata del confronto. La prima è che il perimetro della tesi non comprende il confronto con metodi di imputazione alternativi (sezione 8.2): la moda non è un concorrente del sistema ma una *scala* per i suoi numeri. La seconda è che l'accuratezza è misurata soltanto sulle celle introdotte artificialmente, quelle per cui esiste il ground truth: il sistema può imputare anche celle nulle preesistenti, che non entrano in questa misura (sezione 2.4).

6.7 Esperimenti condotti

Le campagne condotte ammontano complessivamente a 1 618 esecuzioni del sistema in configurazione potata, oltre alle esecuzioni di riferimento. Esse non sono un insieme di prove giustapposte: ciascuna risponde a una delle ipotesi enunciate nel capitolo 5, e la Tabella 6.5 esplicita il collegamento, perché è su di esso che poggia la lettura degli esiti.

Due precisazioni completano il disegno. La prima riguarda le configurazioni *vacue*: le combinazioni in cui la strategia non rimuove alcuna regola producono un file

Tabella 6.5: Esperimenti condotti, obiettivo e ipotesi verificate. La colonna "bracci" indica che cosa viene confrontato; la colonna "risultati" la sezione in cui l'esito è riportato.

#	obiettivo	bracci confrontati	ipotesi	risultati
E1	equivalenza della potatura garantita	`veto-cover` contro configurazione di partenza, su tre famiglie	H6	7.1
E2	verifica indipendente della copertura	ricostruzione dei testimoni del veto per ogni regola rimossa	H7	7.1
E3	effetto della potatura aggressiva	`A5b` contro configurazione di partenza, su tre famiglie e tre soglie	H8, H11	7.3, 7.4
E4	attribuzione del guadagno	riordino per precisione, riordino inverso, tre permutazioni casuali	H14	7.8
E5	ablazione dei componenti	ciascun componente di `A5b` da solo, la combinazione, la somma	H8	6.8
E6	proporzionalità fra riduzione e tempo	tempi di esecuzione appaiati, per strategia e configurazione	H16	7.9
E7	strategie esatte alternative	`score-argmin` con tre politiche sui veti, su Bridges e Glass	H1, H2	7.9

identico a quello di partenza, e sono escluse dai test e conteggiate a parte. La seconda riguarda l'ordine di esecuzione: le campagne sono state condotte in modo strettamente sequenziale, perché la cartella di lavoro del sistema è condivisa fra esecuzioni e due esecuzioni simultanee sovrascrivono reciprocamente gli output senza segnalare alcun errore. La circostanza è stata verificata sperimentalmente, ed è la ragione per cui non si riportano misure di tempi concorrenti.

6.8 Studio di ablazione

La strategia `A5b` è una combinazione di due interventi: un riordino delle regole a soglia di destra nulla e una potatura adattiva. Per stabilire quale dei due produca il guadagno, entrambi sono stati misurati anche separatamente, sulle stesse configurazioni.

Tabella 6.6: Ablazione di `A5b` su Bridges, configurazioni comuni. Le variazioni sono rispetto alla medesima configurazione non potata.

componente	riduzione RFD	Δ corrette	Δ errori	esito
`exact-priority` (solo)	0 %	+25	−45	non stabile fra soglie
`dom-sub-adaptive` (solo)	81 %–87 %	−20	+20	danneggia
`A5b` (combinazione)	81 %–87 %	**+45**	**−32**	**migliora**
somma dei due componenti		+5		
sinergia		**+40**		

Il risultato dell'ablazione è che *nessuno dei due componenti funziona da solo*: il riordino migliora il risultato a una soglia e lo peggiora all'altra, mentre la potatura adattiva lo peggiora sistematicamente. La combinazione è invece stabile e positiva. La differenza fra il guadagno osservato (+45) e la somma dei due componenti isolati (+5) è pari a +40, e costituisce una *sinergia* che va spiegata, non registrata.

La spiegazione è che i due interventi agiscono su fattori diversi del medesimo prodotto. La potatura adattiva modifica la *composizione* dei candidati — quali regole concorrono a determinare il valore — e quindi la quota di imputazioni ottenute per via esatta; il riordino modifica la *sequenza* con cui i cluster vengono tentati e, nella procedura a più celle nulle, determina quale cluster scrive per ultimo, quindi quante celle vengono effettivamente valorizzate. Presi singolarmente, il primo peggiora la qualità delle imputazioni senza aumentarne il numero, il secondo aumenta il numero di imputazioni senza migliorarne la composizione; applicati insieme, producono un aumento di entrambi i fattori, e il guadagno risultante supera la somma dei guadagni isolati. Il sezione 7.5 verifica questa lettura sui tre regimi osservati — Bridges, Glass e Restaurant — e ne ricava il criterio diagnostico.

Va sottolineato un limite del disegno di ablazione: le configurazioni comuni ai tre bracci sono venti, e la lettura della sinergia poggia quindi su un campione più piccolo di quello della campagna principale. L'ablazione identifica il meccanismo, non ne stabilisce la generalità.

–7–
Analisi dei risultati

Questo capitolo presenta e interpreta i risultati. L'ordine non è per metrica ma per *caso*: la potatura che non perde nulla, il confronto con il criterio di ridondanza della teoria classica, la potatura che produce il guadagno e il confine oltre il quale il guadagno non si realizza, e il meccanismo che rende conto dei casi osservati. Da quel meccanismo discende un criterio diagnostico, di cui il capitolo tenta la verifica su una famiglia indipendente; seguono i risultati negativi e l'analisi del costo, che mostra perché la riduzione delle regole non basti a garantire un risparmio di tempo. La lettura d'insieme, gli esiti delle ipotesi e i limiti dell'evidenza sono oggetto del capitolo di discussione.

7.1 La strategia che non perde nulla

La condizione [V2] è l'unico criterio di potatura che offra una garanzia. La sua applicazione è stata verificata su 113 configurazioni — 111 distribuite su tre famiglie e 2 su una quarta famiglia indipendente (sezione 7.7) — con la condizione che l'output del sistema sia *identico* a quello della configurazione di partenza — non soltanto equivalente nelle metriche aggregate, ma identico cella per cella.

Tabella 7.1: Applicazione della condizione [V2] su tre famiglie. "Identiche" è il numero di configurazioni in cui corrette, errori e copertura coincidono esattamente con la configurazione di partenza.

famiglia	soglia	n	N	riduzione	Δ corrette	identiche
Bridges	3	10	1 830	11,7 %	0	**10/10**
Bridges	9	10	8 961	17,1 %	0	**10/10**
Glass	3	25	1 511	2,6 %	0	**25/25**
Glass	9	25	11 648	7,4 %	0	**25/25**
Glass	15	25	27 445	8,6 %	0	**25/25**
Restaurant	3	8	25	0,0 %	0	**8/8**
Restaurant	6	8	124	1,6 %	0	**8/8**
totale		**111**			**0**	**111/111**

Il risultato va letto nei due sensi. Nel senso positivo, è la conferma che la condizione [V2] è corretta: una potatura *esatta* è possibile, e la sua applicazione non altera di

una sola cella il comportamento del sistema. Nel senso negativo, la Tabella 7.1 mostra anche il prezzo: la riduzione è nulla su Restaurant a soglia 3, inferiore al 3 % su Glass a soglia 3, e non supera il 20,8 % in alcun caso — valore misurato sul caso di Bridges a soglia 15, che non appartiene alla campagna d'archivio e per questo non compare in tabella. Alle 111 configurazioni delle tre famiglie se ne aggiungono 2 di una quarta famiglia indipendente (sezione 7.7), per un totale di 113 configurazioni con equivalenza verificata.

La verifica è stata condotta anche in forma indipendente, controllando su stati reali del dataset — e non soltanto per implicazione logica — che per ogni regola rimossa esistesse una regola coprente sopravvissuta in grado di esercitare il veto sugli stessi testimoni. La verifica ha coperto 1 242 321 terne, con zero discrepanze.

7.2 La riduzione classica, e perché non è sicura

Il termine di paragone più ovvio per una potatura è la riduzione di ridondanza della teoria classica delle dipendenze funzionali: una regola $X \to A$ è superflua se un'altra regola $Y \to A$ ha Y sottoinsieme proprio di X, perché $X \to A$ si ottiene per augmentazione (sezione 1.5). Il criterio è sintattico e ignora le soglie; è stato implementato come strategia (`classical-cover`) e misurato sulle medesime configurazioni di Bridges a soglia 9, con la medesima ricetta di accoppiamento delle altre campagne.

Tabella 7.2: Riduzione classica (`classical-cover`) su Bridges a soglia 9, prima ripetizione: l'insieme passa da 8 961 a 230 regole in tutte e cinque le configurazioni (97,4 %). Ogni riga confronta la configurazione di partenza con quella potata.

configurazione	copertura	accuratezza	corrette	errori	tempo (s)
`bridges_14_1`	0.714 → 0.571	0.643 → 0.500	9 → 7	1 → 3	9.9 → 0.3
`bridges_28_1`	0.714 → 0.643	0.643 → 0.429	18 → 12	2 → 6	32.0 → 0.6
`bridges_42_1`	0.595 → 0.595	0.524 → 0.429	22 → 18	3 → 7	36.9 → 0.7
`bridges_56_1`	0.679 → 0.571	0.589 → 0.321	33 → 18	4 → 10	56.9 → 1.0
`bridges_70_1`	0.529 → 0.500	0.457 → 0.357	32 → 25	6 → 12	54.9 → 1.0
totale			**114 → 80**	**16 → 38**	

Il risultato va nella direzione opposta a quella che il solo numero di regole rimosse suggerirebbe. La riduzione è del 97,4 % — da 8 961 a 230 regole, molto oltre il 20,9 % della classe garantita e oltre l'84 % della strategia aggressiva — e il tempo di esecuzione cala di oltre cinquanta volte; ma l'output cambia in *tutte* le configurazioni, la copertura scende, l'accuratezza mediana perde 14,3 punti, le imputazioni corrette passano da 114 a 80 e gli errori da 16 a 38. La circostanza va letta con la numerosità che ha: con cinque configurazioni il test esatto dà $p = 0{,}062\,5$, e nessuna delle cinque migliora, ma la soglia convenzionale di significatività non è raggiunta. Ciò che il confronto stabilisce non è una differenza statisticamente significativa rispetto al criterio classico, ma che il criterio classico *non è innocuo*: la sua applicazione altera il comportamento del sistema in ogni caso osservato.

La spiegazione è quella argomentata nella sezione 1.5. Il criterio classico rimuove regole che il sistema usava, perché la relazione di sussunzione che lo fonda non tiene

conto né della *direzione* delle soglie sul lato destro né dell'*ampiezza* di quelle sul lato sinistro: la regola dichiarata superflua può generare candidati, e la regola che la sussume può essere più severa e quindi non implicarne il veto. La condizione [V2] è l'analogo operativo di quella relazione, ma richiede soglie più larghe a sinistra e non più larghe a destra, cioè la direzione opposta.

La conseguenza per la lettura del lavoro è duplice. Sul piano teorico, il confronto è la verifica empirica della distinzione fra ridondanza dichiarativa e sicurezza operativa. Sul piano della valutazione, mostra perché la riduzione non possa essere l'unico indicatore di una strategia: un criterio che rimuove il 97 % delle regole peggiorando il risultato è, per gli obiettivi di questo lavoro, peggiore di uno che ne rimuove il 17 % senza toccarlo.

7.3 La strategia che produce il guadagno

La combinazione `A5b`, che abbina un riordino delle regole a soglia di destra nulla a una potatura adattiva, è stata valutata sull'intera griglia di Bridges: cinque livelli di incompletezza, cinque esecuzioni indipendenti, due soglie.

Tabella 7.3: `A5b` su Bridges, griglia completa. Confronto appaiato con la medesima configurazione non potata.

soglia	n	riduzione RFD	Δ corrette	Δ errori	Δ precisione	accelerazione
9	25	81,2 %	+32	−40	+0.050	9,0×
15	25	87,2 %	+37	−32	+0.038	17,5×
totale	**50**	84,2 %	**+69**	**−72**	+0.044	11,0×

Il test di Wilcoxon signed-rank sulle corrette dà, sull'insieme delle 50 configurazioni, $n = 39$, $W^+ = 589$, $W^- = 191$, $p = 0{,}005\,3$; sugli errori, $p = 2.5 \cdot 10^{-6}$. Separando per soglia il quadro è più sfumato e va riportato: a soglia 9 il test dà $p = 0{,}056\,2$ ($n = 18$) e a soglia 15 $p = 0{,}048\,4$ ($n = 21$), sicché *nessuna delle due soglie da sola* è conclusiva al livello del 5 %, mentre l'insieme delle configurazioni lo è con ampio margine. La ragione è che in una parte delle configurazioni la potatura è neutra, e il test vede soltanto le differenze non nulle.

Il risultato va confrontato con quello della sezione 7.1. La strategia garantita non altera nulla e riduce poco; `A5b` riduce molto e *migliora* il risultato. Le due strategie non sono alternative sullo stesso asse: rispondono a due domande diverse.

7.4 Il confine: dove il guadagno non si realizza

La medesima strategia, applicata a Glass e Restaurant con la stessa taratura, produce un danneggiamento sistematico.

La causa è precisata dal segno della variazione di copertura: su Glass la potatura riduce le imputazioni effettuate di circa un terzo, e il danno è quindi una *perdita di copertura*, non un peggioramento della qualità di ciò che viene imputato. Su Bridges, invece, la copertura non cala.

Tabella 7.4: Effetto della potatura aggressiva su tutte le famiglie. La taratura è la medesima ($\alpha = 0.85$).

famiglia	n	riduzione RFD	Δ corrette	Δ copertura	esito
Bridges	50	84,2 %	**+69**	−0.003	**migliora**
Glass	45	82,8 %	−54	−0.034	danneggia
Restaurant	18	44,2 %	−12	−0.005	danneggia

Una taratura più conservativa della medesima strategia — ottenuta alzando la soglia di stabilità del parametro di potatura — elimina il danneggiamento su Glass e Restaurant, al prezzo di una riduzione molto inferiore:

Tabella 7.5: La medesima strategia con taratura conservativa ($\alpha = 0.99$) su Glass e Restaurant, griglia completa. "Identiche" indica le configurazioni in cui corrette, errori e copertura coincidono con la configurazione di partenza.

famiglia	config.	identiche	Δ corr.	Δ err.	riduz. (mediana / massima)
Glass	75	**74/75**	0	−1	0,1 % / 52,9 %
Restaurant	22	**20/22**	−1	−2	0,0 % / 45,2 %
totale	**97**	**94/97**	**−1**	**−3**	0,0 % / 52,9 %

Il confronto fra la Tabella 7.4 e la Tabella 7.5 è istruttivo: la stessa strategia passa da −54 a −1 imputazioni corrette su Glass al variare di un solo parametro. La spiegazione di questa sensibilità è l'oggetto della sezione successiva.

Un elemento della Tabella 7.5 va sottolineato perché limita la portata del risultato: la riduzione *mediana* è nulla. Su più della metà delle configurazioni la strategia conservativa non rimuove alcuna regola. Il beneficio si concentra in 27 configurazioni su 97, dove è sostanzioso. Ciò che la taratura conservativa garantisce su tutte le configurazioni non è la riduzione, ma l'assenza di perdite rilevanti.

7.5 Il meccanismo

Le sezioni precedenti hanno stabilito *che cosa* accade. Questa sezione stabilisce *perché*, e il risultato è il contributo interpretativo principale del lavoro.

Il sistema registra, per ogni imputazione effettuata, se essa sia stata ottenuta per via *esatta* — la scorciatoia che si attiva quando la regola impiegata ha soglia di destra nulla — oppure per via approssimata. La distinzione era disponibile nei dati di tutte le campagne, ma non era stata impiegata nell'interpretazione. La Tabella 7.6 la mette in relazione con il numero di imputazioni.

Il numero di imputazioni corrette è il prodotto di due fattori: il *numero* di imputazioni effettuate e la *quota* di quelle corrette, che a sua volta dipende dalla composizione dei candidati e in particolare dalla quota di imputazioni ottenute per via esatta. La potatura produce un guadagno soltanto se la composizione ha spazio per migliorare *e* il numero di imputazioni non diminuisce abbastanza da compensare il miglioramento. La Tabella 7.6 mostra che i tre casi si distinguono esattamente su questo:

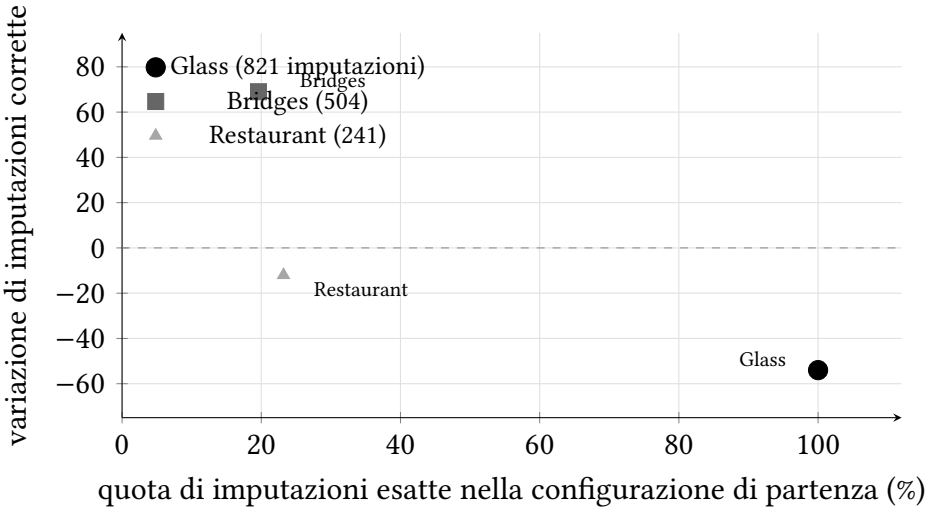


Figura 7.1: Il meccanismo su tre famiglie. In ascissa la quota di imputazioni ottenute per via esatta nella configurazione di partenza, in ordinata la variazione di imputazioni corrette prodotta dalla potatura aggressiva; la dimensione del marcatore è proporzionale al numero di imputazioni della configurazione di partenza. Il guadagno richiede che la quota esatta non sia satura: dove lo è (Glass) la potatura può soltanto togliere copertura.

Tabella 7.6: Il meccanismo del guadagno. Per ciascuna famiglia si confrontano il numero di imputazioni, la quota di quelle ottenute per via esatta e il numero di imputazioni corrette, prima e dopo la potatura aggressiva. I valori sono aggregati sulla *campagna completa* di ciascuna famiglia: 50 configurazioni per Bridges, 45 per Glass, 18 per Restaurant, alla taratura $\alpha = 0{,}85$.

	configurazione di partenza		**dopo `A5b`**		
famiglia	imputazioni	quota esatta	imputazioni	quota esatta	Δ corrette
Bridges (50)	1 309	21,8 %	1 306	40,4 %	**+69**
Glass (45)	821	100,0 %	721	100,0 %	−54
Restaurant (18)	241	23,2 %	229	49,8 %	−12

- **Bridges**: la quota esatta cresce dal 21,8 % al 40,4 % e il numero di imputazioni resta sostanzialmente invariato (1 309 contro 1 306). Il miglioramento della composizione si trasferisce integralmente al risultato: 69 imputazioni corrette in più.
- **Glass**: la quota esatta è *già* pari al 100 %, perché ogni imputazione passa per la via esatta. Non vi è spazio per migliorare la composizione, e la potatura può soltanto togliere copertura: le imputazioni scendono da 821 a 721 e le corrette da 517 a 463.
- **Restaurant**: la composizione ha spazio e migliora più che su Bridges (dal 23,2 % al 49,8 %), ma il numero di imputazioni cala da 241 a 229 e le imputazioni guadagnate in composizione sono meno di quelle perse in copertura: il saldo è di 12 imputazioni corrette in meno.

Sulle 20 configurazioni comuni all'ablazione, dove entrambi i bracci sono misurati sulle stesse configurazioni, i due fattori crescono entrambi e la variazione è di +45 (Tabella 6.6): la differenza rispetto al +69 della campagna completa è una differenza di campione, non di strategia, ed è la ragione per cui le due tabelle dichiarano ciascuna la propria base.

Questo risultato spiega anche un'osservazione che era rimasta senza interpretazione: il riordino `exact-priority` è risultato un *no-op esatto* su Glass, con output identico in 45 configurazioni su 45. La spiegazione è immediata una volta nota la quota esatta: se tutte le imputazioni passano già per la via esatta, non vi è alcuna regola da promuovere.

Perché la sinergia non è misteriosa. La Tabella 6.6 — che si riferisce alle 20 configurazioni comuni ai quattro bracci — mostra che il guadagno della combinazione (+45) supera di molto la somma dei componenti isolati (+5). Il meccanismo rende conto della differenza: i due componenti agiscono su fattori *diversi*. Il riordino per priorità di precisione dispone per prime le regole che producono valori esatti, e migliora quindi la composizione dei candidati; la potatura adattiva modifica quali regole concorrono alla generazione, e quindi quante celle trovano un candidato accettabile. Ciascun componente vale tanto più quanto più è alto il fattore su cui l'altro agisce, e la combinazione è per questo super-additiva: è la stessa legge dei due fattori, letta sul campione in cui entrambi sono misurati.

7.6 Il criterio diagnostico

Dal meccanismo discende un criterio operativo, che sostituisce la soglia euristica adottata in precedenza. Il criterio si fonda su una grandezza che il sistema *registra già*: la quota di imputazioni ottenute per via esatta nella configurazione di partenza, disponibile nei file di risultato prima di qualunque potatura.

Quota esatta $\approx$ 100 %. La leva della qualità non esiste: non vi è margine per sostituire imputazioni approssimate con imputazioni esatte, e la potatura può soltanto togliere copertura. *Non potare.* È il caso di Glass.

Quota esatta bassa. La leva esiste, e la potatura conviene *a condizione che la copertura non cali.* È il caso di Bridges, dove la copertura cresce, e di Restaurant, dove cala.

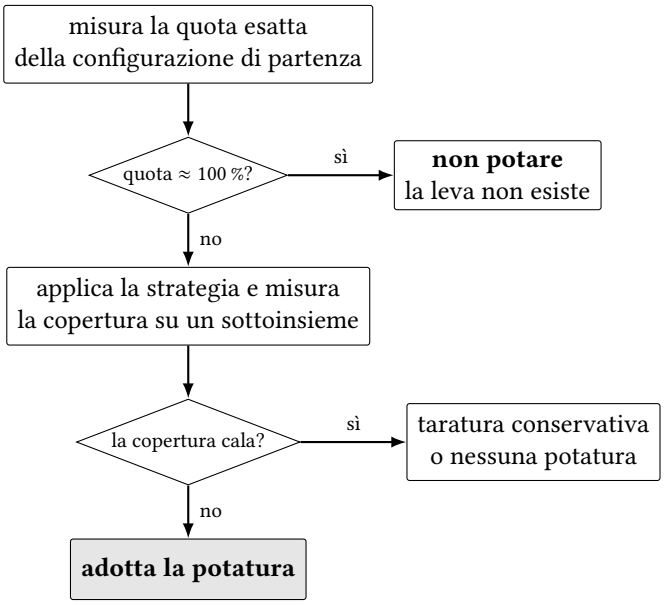


Figura 7.2: Procedura del criterio diagnostico. Il primo passo non richiede esecuzioni, perché la quota esatta è registrata dal sistema; il secondo richiede di misurare la copertura, ed è il solo costo del criterio.

La procedura. Il criterio si traduce in quattro passi, eseguibili senza modificare né il sistema né il dataset.

1. **Misurare la quota esatta** della configurazione di partenza, leggendola dai file di risultato che il sistema produce comunque: il costo di questo passo è nullo, perché non richiede esecuzioni aggiuntive.

2. **Decidere se la leva esiste.** Una quota prossima al 100 % la esclude, e la decisione è di non potare; una quota bassa lascia la leva aperta, e si prosegue.

3. **Misurare la copertura su un sottoinsieme** di configurazioni, applicando la strategia e confrontando il numero di celle valorizzate con quello della configurazione di partenza. È il solo passo che richieda esecuzioni, ed è quindi il costo del criterio.

4. **Adottare la potatura solo se la copertura non cala**; in caso contrario ricorrere alla taratura conservativa, che riduce il rischio rinunciando a parte della riduzione.

Che cosa il criterio decide e che cosa no. Il primo termine — se la leva esista — è decidibile a costo nullo e prima di qualunque esecuzione. Il secondo — se la copertura calerà — non è prevedibile senza eseguire il sistema, e il criterio non è quindi sufficiente a decidere in autonomia: la sua funzione è di *escludere* i casi in cui la potatura non può che danneggiare, non di autorizzarla. La Tabella 7.7 riassume le tre famiglie su cui il criterio è stato derivato.

Perché il criterio non è una soglia adattata ai dati. La grandezza impiegata non è un punteggio scelto per separare i casi favorevoli da quelli sfavorevoli, ma il termine che il meccanismo indica come necessario: il guadagno è il prodotto della

Tabella 7.7: Applicazione del criterio alle tre famiglie su cui è stato derivato. Le grandezze a cui il criterio si riferisce sono quelle della Tabella 7.6, che non vengono qui ripetute. La tabella documenta i casi da cui il criterio è stato ricavato e non costituisce una sua validazione: la prova su una famiglia indipendente è riportata nella sezione 7.7.

famiglia	decisione del criterio	esito osservato
Glass	*non potare*: la leva non esiste	coerente: la potatura toglie copertura senza poter migliorare la composizione
Bridges	*potare se la copertura tiene*	coerente: la copertura resta invariata e il risultato migliora
Restaurant	*misurare prima di adottare*	coerente: la copertura cala e il risultato peggiora

quota esatta e del numero di imputazioni, e una quota esatta pari al 100 % annulla il primo fattore per definizione. La soglia al 100 % non è quindi un parametro da tarare; la zona intermedia — quote basse ma non nulle — resta però indeterminata, ed è il limite principale del criterio.

7.7 La prova sulla quarta famiglia

Il criterio della sezione precedente è stato derivato su tre famiglie. Il test che servirebbe è su una famiglia mai impiegata, e l'archivio ne contiene una: la famiglia Physician, il cui schema è allineato ai propri file di regole. La prova è stata condotta su tutto il materiale eseguibile, e il suo esito è *parzialmente conclusivo*: conferma l'equivalenza della potatura garantita e fornisce due casi coerenti con il criterio, ma su un numero di celle troppo piccolo per fondare una conclusione statistica.

Perché le configurazioni piene non sono impiegabili. I dataset non remodulati contengono 103 592 righe e fino a 121 913 celle nulle. La ragione dell'impraticabilità non è però il tempo di imputazione: è il *preprocessing interno* del sistema, che all'avvio verifica per ciascuna regola a soglia di destra nulla se il suo lato sinistro si comporti da chiave, con un confronto a coppie di tuple quadratico nel numero di righe (sezione 2.5). La sonda condotta sulla configurazione più piccola (`Physician_Compare_National_13467_1`, 13 467 celle) è stata interrotta dopo 1 200 s senza che il sistema avesse prodotto *una sola* riga di risultato: il tempo era ancora interamente assorbito dal preprocessing. La circostanza è coerente con il materiale d'archivio, che per la medesima configurazione contiene un file di risultati parziale di 1 005 righe e un file di celle popolate vuoto.

Che cosa è stato possibile eseguire. Delle quattro versioni remodulate, due — 0.05 e 0.1 — non sono eseguibili per il difetto D3 (sottosezione 9.2.3): il sistema si arresta all'avvio su righe del dataset con un numero di campi inferiore allo schema. Le due versioni eseguibili, 0.005 e 0.01, valutano 13 e 27 celle; escludendo dalle altre due le celle non localizzabili, il materiale eseguibile salirebbe a 329 celle, ma la riparazione delle righe malformate non è stata applicata perché il costo di preprocessing la rende non conveniente rispetto al contributo ottenibile (sottosezione 9.2.3).

Verifica di equivalenza. La potatura garantita è stata applicata a tutte le configurazioni eseguibili, con la politica di tutela integrale dei veti. L'esito è netto: su 2 configurazioni su 2 i file delle celle popolate e dei risultati sono *byte-identici* a quelli della configurazione di partenza, con riduzioni dell'insieme di regole del 6 % e del 5 %. È la prima verifica della condizione [V2] su una famiglia indipendente da quelle su cui è stata derivata, e si aggiunge alle 111 configurazioni già verificate.

Il test del criterio. La Tabella 7.8 riporta i due casi eseguibili.

Tabella 7.8: Prova esplorativa sulla famiglia Physician. Confronto fra configurazione di partenza e potatura aggressiva sulle due versioni eseguibili; "esatte" sono le imputazioni ottenute per via esatta, "corrette" quelle conformi al ground truth.

versione	celle	imputate	esatte	quota esatta	corrette	copertura	riduzione
0.005, partenza	13	3	3	100 %	2	23,1 %	—
0.005, `A5b`	13	3	3	100 %	2	23,1 %	67 %
0.01, partenza	27	11	5	45 %	4	40,7 %	—
0.01, `A5b`	27	9	3	33 %	3	33,3 %	74 %

I due casi hanno esiti diversi e istruttivi. Sulla versione 0.005 la configurazione di partenza ottiene *tutte* le proprie imputazioni per via esatta: la leva della qualità non esiste, il criterio prescrive di non potare, e la potatura aggressiva — pur rimuovendo due terzi delle regole — non cambia nulla. Sulla versione 0.01 la quota esatta di partenza è del 45 %, la leva esiste, e la potatura *peggiora* il risultato: le celle valorizzate scendono da 11 a 9, le imputazioni esatte da 5 a 3, le corrette da 4 a 3. Il criterio prescrive in questo caso di misurare la copertura prima di adottare la potatura, e la misura dice di non adottarla.

Entrambi gli esiti sono quindi coerenti con il criterio, uno per esclusione della leva e uno per caduta della copertura. Il valore probatorio resta però limitato, per tre ragioni che vanno dichiarate: le celle valutabili sono 40 in tutto; la soglia disponibile è una sola (3, l'unica per cui l'archivio contenga i file di regole); e su questa famiglia il sistema imputa una frazione minoritaria delle celle, cosicché le differenze osservate valgono poche unità. La prova va letta come un *controllo di coerenza* su dati nuovi, non come una validazione.

7.8 Risultati negativi

Una parte rilevante delle strategie sperimentate non ha prodotto il guadagno atteso. I risultati sono riportati perché delimitano lo spazio delle soluzioni.

Il segnale di precisione non è la causa del guadagno del riordino. Il riordino dei cluster per precisione empirica produce un guadagno misurabile (+96 imputazioni corrette su 30 configurazioni, $p = 2.5 \cdot 10^{-6}$). L'ipotesi che tale guadagno fosse attribuibile al segnale di precisione è stata sottoposta a un esperimento di controllo: a parità di insieme di regole, si sono confrontati l'ordinamento per precisione, l'ordinamento inverso e tre permutazioni casuali indipendenti.

Tabella 7.9: Esperimento di controllo sull'ordinamento dei cluster. Quindici configurazioni, cinque bracci, insieme di regole invariato.

braccio	corrette	Δ vs orig.	migl. / pegg.	Wilcoxon
originale	333	—	—	—
riordino per precisione	386	+53	13 / 0	$p = 0{,}001\,6$
permutazione casuale (media di 3)	—	+52.7	13 / 0	$p = 0{,}001\,6$
riordino inverso	313	−20	6 / 1	$p = 0{,}270\,2$

Il confronto decisivo è fra il riordino per precisione e la permutazione casuale: la differenza è +0.33 imputazioni corrette, con $p = 0{,}887\,8$, e una delle permutazioni casuali supera l'ordinamento per precisione. Il guadagno *non è attribuibile al segnale impiegato*. Il riordino inverso, d'altra parte, peggiora il risultato rispetto all'originale: non è dunque vero che qualunque permutazione funzioni, ma che l'ordine di partenza è prossimo al caso peggiore e che qualunque ordine che non sia l'inverso lo migliora.

La regolarizzazione della stima non ha effetto. La sostituzione della stima di precisione con una stima contratta verso un valore a priori è stata sperimentata su quattro valori del parametro di contrazione. I risultati sono *identici* in tutte le configurazioni per tutti i valori: il parametro non è una variabile da tarare.

Il riordino entro il cluster è inerte per costruzione. La procedura che genera i candidati percorre tutte le regole di un cluster e ne conserva il minimo, senza uscite anticipate. L'ordine interno al cluster non può quindi influenzare il valore imputato. La previsione è stata confermata empiricamente su 15 configurazioni su 15, con zero differenze non nulle al test.

L'ordinamento delle rimozioni sotto vincolo di budget è un vicolo cieco. La politica `budget:F` ordina le regole da rimuovere secondo un indice di rischio. Tale indice è risultato *anti-correlato* con la rilevanza effettiva delle regole: su 8 961 regole, la correlazione di rango con il numero di celle che ciascuna regola può servire è $\rho = -0.237$. La causa è stata individuata: l'indice somma i valori mancanti sulle colonne del lato *sinistro*, mentre una regola può servire soltanto celle della propria colonna di *destra*. La correzione dell'indice produce un miglioramento strutturale netto — a parità di budget, rimuove 2.5 volte più regole che non servono alcuna cella e distrugge 2.6 volte meno celle servibili — ma il vantaggio *non si traduce in un miglioramento dell'esito*: su 13 configurazioni distribuite su tre famiglie, il saldo delle imputazioni corrette è identico. La conclusione è che, sotto vincolo di budget, conta la *dimensione* del budget e non l'*ordine* con cui si sceglie.

Una potatura esatta della generazione può rallentare il sistema. La strategia `score-argmin`, che preserva la generazione calcolando staticamente quali regole non sono mai l'argomento del minimo, raggiunge riduzioni fino al 98 % e preserva la copertura. Rende però l'esecuzione fino a 251 volte più lenta. Il risultato mostra che la relazione fra numero di regole e tempo di esecuzione non è monotona, e che la riduzione non è di per sé un indicatore di efficienza.

7.9 Il costo: perché ridurre non basta

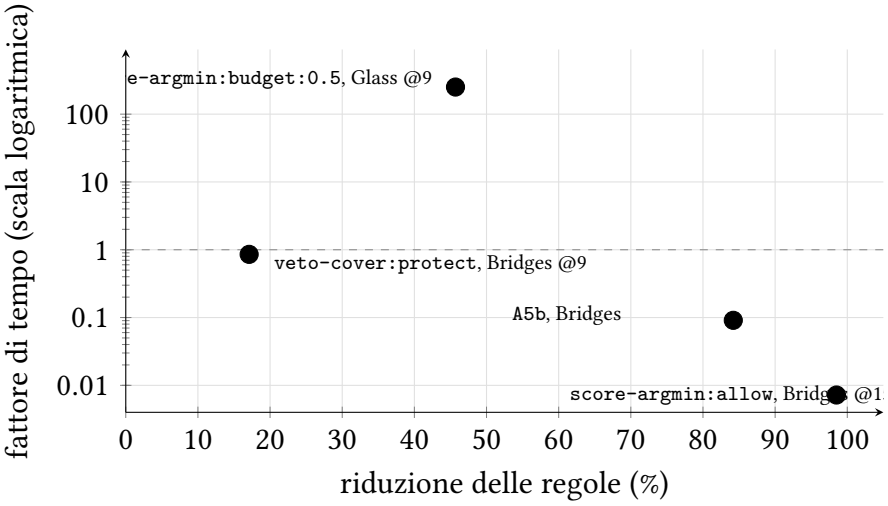


Figura 7.3: Riduzione ottenuta e fattore di tempo per quattro casi misurati. Il fattore è il rapporto fra il tempo di esecuzione della configurazione potata e quello della configurazione di partenza: valori inferiori a uno indicano un'accelerazione. La relazione fra le due grandezze non è monotona, e il caso con la riduzione maggiore è anche il più veloce mentre un caso con riduzione inferiore è il più lento.

L'ipotesi H16 prevedeva che il tempo di esecuzione seguisse il numero di regole rimosse. I dati la falsificano, e il modo in cui la falsificano è più interessante della previsione.

Il costo di *preprocessing* della potatura non è invece in discussione: la riscrittura del file delle regole è trascurabile rispetto all'esecuzione del sistema e cresce con il numero di regole, attestandosi nell'ordine dei secondi sui file maggiori (27 445 regole). Il confronto fra i due costi è quindi interamente a favore della potatura, e il problema non è il costo dell'analisi ma l'effetto sull'esecuzione.

`score-argmin` rimuove fino al 98 % delle regole e rende il sistema *drammaticamente più lento*: il rallentamento mediano è di 5,8 volte con la variante `allow` e di 20,1 volte con `budget:0.5`, con un massimo di 251 volte sulla configurazione `glass_43_2` a soglia 9, dove il tempo di esecuzione passa da 3,5 s a 873 s mentre le regole scendono da 11 648 a 6 323. Su quattordici configurazioni, la variante `allow` è più veloce della configurazione di partenza in *una sola*.

Tabella 7.10: Rallentamento del tempo di esecuzione prodotto da `score-argmin` su Glass, rispetto alla configurazione di partenza. Il rallentamento è il rapporto fra il tempo della variante potata e quello della variante non potata, sulla stessa configurazione.

variante	rallentamento mediano	rallentamento massimo
`score-argmin:allow`	5,8×	28×
`score-argmin:budget:0.5`	20,1×	251×

La circostanza che esclude la spiegazione ovvia è un confronto interno alla tabella: la variante `budget:0.5` *conserva più regole* di `allow` — 6 323 contro 1 098 sulla configurazione citata — ed è circa nove volte più lenta. Non è dunque la *quantità* delle regole a determinare il tempo, ma la loro *composizione*: il tempo dipende da quali regole sopravvivono, non da quante.

Una spiegazione candidata esiste ed è coerente con l'analisi del sistema, ma *non è verificata* e va presentata come tale. Il sistema dispone di una scorciatoia esatta che chiude una cella in tempo costante quando la tupla presenta una sola cella nulla e la regola impiegata ha soglia di destra nulla (sezione 2.3). Se l'insieme potato non contiene più le regole che attivano quella scorciatoia per la maggior parte delle celle, ogni cella ricade nella ricerca per similarità sull'intero dataset, il cui costo è lineare nel numero di righe anziché costante. La potatura rimuoverebbe cioè, insieme alle regole dannose, la via rapida. La verifica richiede di strumentare il percorso della scorciatoia, che non è stato possibile isolare con la strumentazione disponibile, e la questione è registrata fra gli sviluppi (sezione 9.4).

La conseguenza pratica è netta ed è il motivo per cui la sezione è necessaria: `score-argmin` non è una strategia utilizzabile, malgrado il criterio esatto che la fonda e la conferma sperimentale della sua proprietà principale. Un guadagno di copertura pagato con un tempo di esecuzione da sei a duecentocinquanta volte maggiore non è un compromesso proponibile. Il caso mostra inoltre che la domanda del lavoro — se la potatura migliori *e* acceleri — non ammette una risposta unica: le due grandezze possono muoversi in direzioni opposte, e la valutazione di una strategia deve riportarle entrambe.

–8–
Discussione

Questo capitolo raccoglie la lettura dei risultati e non introduce grandezze nuove: il suo contenuto è di interpretazione, e le misure a cui si riferisce sono quelle presentate nel capitolo precedente. Vi sono ricostruiti gli esiti delle ipotesi e il quadro complessivo che ne emerge, dichiarati i limiti dell'evidenza raccolta, e collocato il lavoro rispetto alla letteratura — la riduzione classica di un insieme di regole e la selezione guidata dai metadati — indicando quali verifiche estenderebbero i risultati e quali il materiale disponibile non ha consentito.

8.1 Esiti delle ipotesi

La Tabella 8.1 raccoglie gli esiti. Le ipotesi sono quelle enunciate nella Tabella 5.1 e nella Tabella 5.2; nessuna è stata modificata dopo la misura.

Le ipotesi confermate riguardano la teoria. H6, H7 e H8 sono state confermate senza riserve, e riguardano la garanzia e la leva di qualità presa da sola: la condizione [V2] sopravvive alla verifica indipendente, la sua applicazione non altera il comportamento del sistema su alcuna configurazione, e il riordino per priorità di precisione produce un guadagno quando è impiegato da solo. È il nucleo dimostrativo del lavoro.

Le ipotesi falsificate riguardano le euristiche e le attribuzioni. H1, H11, H14 e H16 sono state falsificate, e la loro falsificazione ha un valore che va oltre il caso singolo. H1 mostra che la strategia esatta lato generazione non è utilizzabile con la politica di tutela, perché la classe garantita è troppo piccola per produrre la riduzione attesa. H11 mostra che il guadagno non generalizza. H14 mostra che il segnale che si riteneva responsabile del guadagno non lo spiega: il riordino casuale produce lo stesso effetto, e l'interpretazione che era stata formulata in una fase precedente del lavoro è stata corretta. H16 mostra che la riduzione non implica il risparmio di tempo.

La lettura d'insieme. Il quadro che emerge è coerente e si può riassumere in quattro punti. *Primo*: esiste una potatura esatta, ed è piccola — la condizione [V2] è corretta e verificata, ma la riduzione che consente è limitata, e il limite è intrinseco al criterio (sezione 4.6). *Secondo*: esiste una potatura che migliora il risultato, ed è condizionata

Tabella 8.1: Esiti delle ipotesi. “Confermata” indica che la previsione è stata confermata dai dati; “falsificata” che i dati la contraddicono. Per H1 il valore riportato è quello ottenuto dopo la correzione del difetto D5 (sottosezione 9.3.1): con il codice precedente la riduzione misurata era dello 0,1 %.

#	ipotesi	previsione	esito
H1	`score-argmin` con veti protetti	riduzione $\geq 30\,\%$	**falsificata** (0,3 %)
H6	`veto-cover` su tutte le famiglie	output identico	**confermata** 113/113
H7	verifica empirica di [V2]	zero discrepanze	**confermata**
H8	`precision-priority` da solo	guadagno di qualità	**confermata** (+96 corrette su 30 configurazioni)
H11	`precision-priority` altrove	guadagno su Glass e Restaurant	**falsificata**
H14	guadagno attribuibile alla precisione	il casuale non produce lo stesso guadagno	**falsificata**
H16	riduzione e tempo proporzionali	tempi minori	**falsificata** (fino a 251×)
H2	`score-argmin` altrove	copertura non cala	confermata
H3	copertura esatta e gratuita	nessuna perdita	falsificata
H4	conversione B⟶D	guadagno gratuito	falsificata
H5	riordino delle righe del dataset	guadagno	falsificata
H9	riordino + potatura	guadagno maggiore	sub-additiva
H10	regolarizzazione $\alpha = 3$	guadagno $\geq 20\,\%$	falsificata
H12	la regolarizzata generalizza meglio	guadagno maggiore	falsificata
H13	riordino entro il cluster	leva di qualità	falsificata
H15	ordinamento del budget migliorabile	guadagno	falsificata

— `A5b` riduce dell'84,2 % e migliora il risultato su Bridges, mentre sugli altri dataset danneggia, e la condizione di applicabilità è ora nota e misurabile. *Terzo*: il guadagno non proviene dal segnale che si riteneva responsabile, e la spiegazione corretta è il prodotto di due fattori, la quota di imputazioni ottenute per via esatta e il numero di imputazioni effettuate. *Quarto*: il tempo non segue la riduzione, e una strategia può rimuovere quasi tutte le regole peggiorando insieme il risultato e la durata.

Va infine osservato che i tre interventi studiati — la potatura esatta, la potatura aggressiva e la potatura conservativa — non sono ordinabili. La prima non altera mai il risultato ma riduce poco; la seconda produce il guadagno maggiore ma soltanto su una famiglia; la terza non peggiora quasi mai ma rinuncia, nella maggior parte delle configurazioni, a qualunque riduzione. La scelta fra le tre dipende dall'obiettivo: garanzia, guadagno, o assenza di rischio.

8.2 Limiti dell'evidenza

I limiti che seguono sono dichiarati perché delimitano la portata delle conclusioni, e non perché ridimensionino i risultati: ciascuno indica anche la verifica che servirebbe a superarlo.

Il criterio diagnostico è post-hoc. Il meccanismo a due fattori e il criterio che ne discende sono stati ricavati osservando i risultati su tre famiglie, non previsti prima della misura. La prova su una quarta famiglia è stata condotta (sezione 7.7) e ha prodotto due esiti coerenti con il criterio, ma su 40 celle con copertura non nulla, a una sola soglia, e senza che gli esiti siano distinguibili dal caso: due delle quattro versioni non sono eseguibili per righe malformate nel dataset (sottosezione 9.2.3). Il criterio va quindi inteso come una spiegazione con evidenza preliminare, non come una legge verificata su dati indipendenti.

Una sola famiglia con guadagno pieno. Il guadagno di qualità è stato osservato su Bridges; su Glass e Restaurant la stessa strategia danneggia, e la taratura conservativa riduce il danno senza produrre guadagno. La generalità del meccanismo è sostenuta dalla coerenza dei tre casi con la stessa legge, non da una molteplicità di famiglie favorevoli.

Le misure di tempo provengono da una sola macchina. Le campagne sono state condotte su una macchina a otto core, con esecuzioni strettamente sequenziali, e i valori assoluti dipendono dall'ambiente. I confronti sono appaiati — la stessa configurazione con e senza potatura — e sono quindi robusti rispetto all'ambiente, mentre i fattori di accelerazione e di rallentamento riportati nel capitolo vanno letti come ordini di grandezza.

La numerosità è quella delle configurazioni. L'unità di analisi è la configurazione, non la cella nulla, e i test impiegano quindi alcune decine di osservazioni (sezione 6.1). La potenza statistica è conseguentemente limitata: le conclusioni sulle strategie il cui effetto è piccolo rispetto alla variabilità fra configurazioni vanno considerate indicative.

I test sono inoltre numerosi rispetto alle dimensioni del campione, e il capitolo riporta per questo i valori corretti per confronti multipli accanto a quelli grezzi (sezione 6.2).

Il perimetro non comprende il confronto con altri metodi di imputazione. Il lavoro studia l'effetto della selezione delle regole *a parità di sistema*: il termine di paragone è la stessa esecuzione senza potatura, non un algoritmo di imputazione alternativo. La scelta discende dal vincolo dichiarato — la potatura opera sul file delle regole di un sistema specifico — e va tenuta presente nel leggere i risultati: essi non dicono se il sistema studiato imputi meglio o peggio di altri, ma che cosa si guadagni o si perda potando le sue regole.

Due famiglie dell'archivio non sono utilizzabili. La famiglia Nursery non contiene dataset e la famiglia Physician, nelle versioni piene, ha dimensioni incompatibili con una campagna (103 592 righe e fino a 121 913 celle nulle). La griglia effettiva è quindi composta da tre famiglie, ed è la ragione per cui la prova su una famiglia indipendente non ha potuto che essere esplorativa.

8.3 Collocazione e verifiche ulteriori

Rispetto alla riduzione classica di un insieme di regole. La teoria classica delle dipendenze funzionali affronta la ridondanza in forma dichiarativa: una dipendenza è superflua se la sua rimozione non cambia l'insieme dei vincoli soddisfatti, e il calcolo di una copertura minimale è un'operazione standard (sezione 1.5). Il lavoro mostra che quella nozione non trasferisce al caso operativo, e la ragione è precisa: nel sistema studiato il ruolo di una regola è definito da come il programma la consuma, non dalla sua semantica dichiarativa, e due regole logicamente equivalenti possono avere effetti diversi perché differiscono per la direzione delle soglie. La condizione [V2] è l'analogo operativo della dominanza classica, ma richiede che la regola coprente sia *più permissiva* a sinistra e *più severa* a destra: la direzione opposta a quella che si otterrebbe ragionando sui soli vincoli. Il teorema negativo stabilisce che con l'informazione disponibile a una strategia esterna non si può andare oltre questa classe senza rinunciare alla garanzia.

La distinzione non è soltanto argomentata: è misurata. Applicando il criterio classico come strategia di potatura (`classical-cover`) sulle configurazioni di Bridges a soglia 9, la riduzione raggiunge il 97,4 % — molto oltre il tetto della classe garantita — ma l'output cambia in tutte le configurazioni, l'accuratezza mediana perde 14,3 punti e le imputazioni corrette scendono da 114 a 80 (sezione 7.2). Il confronto mostra insieme due cose: che la ridondanza dichiarativa non implica la sicurezza operativa, e che il numero di regole rimosse, da solo, non è un indicatore di qualità di una strategia.

Rispetto alla selezione guidata dai metadati. Una linea di ricerca recente impiega modelli linguistici per inferire relazioni fra colonne dal loro nome o per guidare l'esplorazione di fonti eterogenee (sezione 1.7). L'ambizione è la stessa — impiegare il significato invece della forma — ma l'oggetto è diverso: quelle tecniche *interpretano* i metadati per produrne di nuovi, mentre il lavoro qui presentato *seleziona* metadati

esistenti rispetto al consumo che ne fa un algoritmo noto. La differenza ha una conseguenza pratica: la garanzia di equivalenza che il capitolo 4 dimostra non dipende dalla qualità di una stima, ed è quindi verificabile per confronto di output; una selezione guidata da un modello linguistico richiederebbe invece una validazione statistica del modello stesso.

Manca la misura dell'esercizio effettivo dei veti.

Manca il valore marginale del pruning esterno rispetto alla riduzione interna.

Che cosa renderebbe più forti le conclusioni. Tre verifiche sono state identificate come decisive e non sono state condotte, ciascuna per una ragione dichiarata. La prima è la *misura dell'esercizio effettivo dei veti*: il meccanismo a due fattori è oggi inferito dalla composizione degli output, e la strumentazione già disponibile consentirebbe di contare le verifiche effettivamente eseguite prima e dopo la potatura, sostituendo l'inferenza con una misura (sezione 9.4). La seconda è il *valore marginale della potatura esterna* rispetto alla riduzione che il sistema compie già da sé sulle regole a soglia nulla con lato sinistro chiave (sezione 2.5): quantificarlo richiede di calcolare la sovrapposizione fra le due classi, e direbbe quanto del lavoro è già svolto dall'eseguibile. La terza è la *validazione prospettica del criterio* su una famiglia con quota esatta intermedia e copertura stabile: la prova sulla quarta famiglia ha fornito due casi coerenti ma su 40 celle, e non è sufficiente (sezione 7.7).

Che cosa non è stato possibile verificare. Le configurazioni piene della famiglia Physician restano fuori portata perché il preprocessing interno del sistema è quadratico nel numero di righe, e le due versioni remodulate riparabili non sono state ripristinate perché il costo di preprocessing non è giustificato dal numero di celle che se ne ricaverebbero (sottosezione 9.2.3). Il confronto con metodi di imputazione alternativi è fuori dal perimetro dichiarato del lavoro, che misura l'effetto della selezione delle regole a parità di sistema (sezione 8.2). Entrambe le circostanze sono limiti del materiale o del perimetro, non dell'impostazione, e sono state dichiarate prima che un lettore le incontrasse.

–9–
Difetti e criticità del sistema

Questo capitolo raccoglie i difetti individuati nel corso dell'analisi. Essi non sono un residuo accessorio del lavoro: la loro individuazione è stata resa necessaria dalla necessità di stabilire se i risultati negativi fossero da attribuire alle strategie sviluppate o al materiale di partenza, e la distinzione si è rivelata decisiva in più di un caso. Il capitolo separa i difetti del sistema studiato da quelli del codice sviluppato in questo lavoro — i primi limiti del materiale, i secondi questione di attendibilità delle misure — ne quantifica l'impatto, ne indica il rimedio e ne trae le conseguenze generali per chi impiegasse il sistema su altri dati.

9.1 Classificazione dei difetti

I difetti individuati appartengono a quattro categorie, che è opportuno tenere distinte perché richiedono interventi di natura diversa. La distinzione che più conta, ai fini della lettura del capitolo, è però un'altra: quella fra i difetti del *sistema studiato* — quattro, tutti nel archivio — e gli errori del *codice sviluppato in questo lavoro* — due, entrambi individuati e corretti durante l'analisi. I primi limitano ciò che si può concludere sul sistema; i secondi riguardano l'attendibilità delle misure e sono esposti perché il loro trattamento fa parte del metodo.

Tabella 9.1: Difetti individuati, per categoria e provenienza.

#	difetto	categoria	provenienza
D1	ramo irraggiungibile nel calcolo della similarità	algoritmo	materiale d'archivio
D2	disallineamento di schema nei dataset Glass	dati	materiale d'archivio
D3	indici di ground truth non ricostruibili (Physician)	dati	materiale d'archivio
D4	eseguibile di build contenente istruzioni di debug	artefatto	materiale d'archivio
D5	computo errato del budget nella politica sui veti	codice sviluppato	**questo lavoro**
D6	indice di rischio anti-correlato con la rilevanza	codice sviluppato	**questo lavoro**

Tabella 9.2: Impatto dei difetti, come sono stati rilevati e stato attuale. Le classi sono quelle della Tabella 9.1.

difetto	impatto quantificato	come è stato rilevato	stato
D1	la conversione `B`→`D` è peggiore in 25 configurazioni su 25, con una perdita di 21,7 punti di accuratezza	confronto fra i due rami sul bytecode, poi misura	non corretto: il vincolo è di non modificare l'eseguibile
D2	la famiglia Glass non è utilizzabile come distribuita: lo schema dei dataset è posizionale, quello dei file di regole usa i nomi	prima esecuzione fallita, poi ispezione delle intestazioni	corretto esternamente, con rinomina documentata
D3	due configurazioni non eseguibili per righe malformate; 115 celle non localizzabili su 404; celle valutabili da 329 a 40	esecuzione fallita per eccezione di indice fuori intervallo; conteggio delle righe con campi mancanti	non riparato: costo di preprocessing non giustificato
D4	una build alternativa con due istruzioni di diagnostica, con comportamento identico sull'output	confronto fra i due eseguibili sulla stessa configurazione	documentato: esclude che l'archivio contenga una versione emendata di D1
D5	riduzione allo 0,1 % invece del 17,1 % su Bridges a soglia 9; 43,7 % delle rimozioni dichiarate esatte senza copritore sopravvissuto	confronto fra riduzione dichiarata ed effettiva su una pipeline composita	corretto e verificato su 8 configurazioni su 8
D6	indice di rischio con $\rho = -0.237$ su 8 961 regole; dopo la correzione, 2.5 volte più regole inutili rimosse e 2.6 volte meno celle servibili distrutte	validazione dell'indice contro il numero di celle che ciascuna regola può servire	corretto, senza guadagno sull'esito

9.2 Difetti del sistema studiato

I quattro difetti che seguono appartengono al materiale d'archivio del sistema studiato. Essi hanno una caratteristica comune: incidono sulla *possibilità* di interpretare i risultati, non soltanto sulla loro qualità, e in due casi hanno reso inutilizzabile una parte della griglia sperimentale. Ciascuno è presentato con il sintomo, l'evidenza che lo stabilisce, l'impatto sui risultati e il rimedio adottato.

9.2.1 D1 — Ramo irraggiungibile nel calcolo della similarità

Il sistema calcola il contributo di similarità di un attributo con una procedura che distingue fra attributi numerici e attributi categorici. Il disassemblaggio mostra che la diramazione verso il caso categorico è *irraggiungibile*: la condizione di selezione confronta riferimenti a oggetti anziché il contenuto, e non può quindi risultare vera. In conseguenza di ciò, un attributo categorico viene elaborato dalla procedura numerica, che tenta una conversione a numero, fallisce, e ricade su una distanza di stringa applicata a etichette che possono essere lunghe e prive di relazione.

Osservazione 18. Il difetto è confermato dall'analisi statica e non è stato corretto: il vincolo di non modificare l'eseguibile è stato mantenuto per tutta la durata della valutazione esterna. La sua esistenza non è un'ipotesi ma un fatto verificabile nel bytecode.

Impatto sull'analisi. Il difetto ha conseguenze sulla tipizzazione delle colonne impiegata nelle campagne. Si è verificato sperimentalmente che la conversione di una colonna da categoria a numerica, lungi dal costituire un miglioramento gratuito, peggiora il risultato: su 25 configurazioni, la conversione produce una perdita media di 21,7 punti di accuratezza, con un peggioramento in 25 configurazioni su 25. Il risultato ha richiesto di rivedere una conclusione formulata in una fase precedente del lavoro, nella quale la conversione era stata presentata come un guadagno.

9.2.2 D2 — Disallineamento di schema nei dataset Glass

I dataset della famiglia Glass presentano nomi di colonna in forma posizionale (`A`, `B`, …), mentre i corrispondenti file di regole impiegano nomi descrittivi. Per le altre famiglie i due insiemi di nomi coincidono; per Glass no.

Il disallineamento non è innocuo. Il sistema risolve le colonne *per posizione*, non per nome: quando gli schemi non coincidono, la corrispondenza fra le colonne del dataset e quelle delle regole è apparentemente corretta ma sostanzialmente errata, e il sistema scrive le imputazioni nella colonna sbagliata. La condizione è di corruzione silenziosa: nessun errore viene segnalato.

Intervento. La correzione adottata è esterna e consiste nel rinominare posizionalmente le colonne del dataset perché coincidano con quelle dei file di regole, e nel riscrivere di conseguenza il materiale di ground truth. L'ordine delle colonne non è modificato, sicché gli indici posizionali impiegati dal sistema restano validi.

Impatto sull'analisi. Prima della correzione la famiglia Glass risultava inutilizzabile, con copertura nulla. Dopo la correzione la copertura si attesta intorno al 30 %. Va osservato che il difetto ha un effetto anche sull'interpretazione delle statistiche di training: i file di risultato prodotti prima della correzione registrano valori non imputati per tutte le celle, e non sono quindi impiegabili come riferimento. La circostanza ha richiesto di raccogliere statistiche ex novo su esecuzioni condotte con lo schema corretto.

9.2.3 D3 — Righe malformate e indici non ricostruibili nei dataset Physician

La famiglia Physician comprende quattro versioni "remodulate" di dimensioni contenute, pensate per la sperimentazione. Il loro materiale presenta due difetti distinti, che una verifica successiva ha permesso di separare.

Righe con un numero di campi inferiore allo schema. I dataset delle versioni 0.05 e 0.1 contengono righe malformate: 7, 11 o 1 campi invece di 13. Il file della versione 0.05 termina a metà di un record, e nella 0.1 una riga mostra due record concatenati, con un valore testuale che risulta tagliato e ricomposto ("`PENNSYLVANIA COLLEGSICAL THERAPY`"). È questa la causa diretta della non eseguibilità: il controllo di chiave che il sistema esegue all'avvio accede ai campi di una tupla per indice, e su una riga corta esce con `IndexOutOfBoundsException`. La diagnosi è confermata dal materiale d'archivio: per entrambe le versioni il log, il file delle celle popolate e il file dei risultati spediti con l'archivio sono *vuoti*, mentre per le versioni 0.005 e 0.01 sono popolati.

Indici di ground truth eccedenti il dataset. Il materiale di ground truth — in formato `riga;attributo;valore` — contiene indici di riga che eccedono il numero di righe del dataset corrispondente, ed è ben formato: il difetto riguarda il *riferimento*, non la sintassi.

Tabella 9.3: Difetto D3. Righe malformate e voci di ground truth con indice fuori intervallo, per versione remodulata. Le colonne "valutabili" indicano le celle che restano dopo l'esclusione degli indici fuori intervallo.

versione	righe	righe malformate	voci ground truth	fuori intervallo	valutabili
0.005	103	0	13	0	13
0.01	207	0	27	0	27
0.05	914	**1**	135	**14**	**121**
0.1	1374	**2**	269	**101**	**168**

Gli indici fuori intervallo provengono dal dataset completo, prima della riduzione, e non sono stati rimappati; le righe malformate indicano che la riduzione è avvenuta per *troncamento* del file, circostanza che spiega anche perché gli indici non corrispondano più. Escludendo le voci non localizzabili, le celle valutabili passano da 40 a 329: la correzione non richiede di modificare il dataset, ma soltanto di dichiarare quali celle si escludono.

La ricostruzione della corrispondenza non è praticabile. Si è verificato se gli indici potessero essere ricostruiti confrontando le righe delle versioni ridotte con quelle del dataset completo. La verifica ha dato esito negativo: su 200 righe esaminate, 29 non trovano corrispondenza e 73 risultano ambigue. Le versioni remodulate non sono un sottocampionamento del dataset completo ma una sua perturbazione, e l'informazione necessaria alla rimappatura non è contenuta nei dati disponibili.

Riparazione possibile, non intrapresa. Le righe malformate possono essere sostituite da righe di soli valori mancanti, conservando la numerazione su cui il ground truth è indicizzato e producendo righe che il sistema ignora in entrambi i ruoli; la procedura è implementata e documentata (`pruning/repair_physician_datasets.py`). La riparazione non è stata applicata alla campagna, perché il costo di preprocessing del sistema su queste configurazioni — il controllo di chiave è quadratico nel numero di righe, e vale decine di minuti per esecuzione — non è giustificato dal numero di celle che se ne ricaverebbero, e perché il contributo scientifico delle due versioni sarebbe comunque marginale rispetto alle versioni già eseguibili.

Osservazione 19. La famiglia Physician è utilizzabile come quarta famiglia per la verifica di equivalenza, e lo è stata (sezione 7.7); per la verifica di qualità fornisce 40 celle con copertura non nulla, un campione insufficiente a trarre conclusioni. La limitazione è del materiale, non dell'impostazione sperimentale, e va dichiarata come tale.

9.2.4 D4 — Eseguibile di build con istruzioni di debug

L'archivio contiene un secondo eseguibile, di dimensioni ridotte, la cui struttura interna è danneggiata (la directory centrale dell'archivio è assente). L'estrazione delle voci è comunque possibile, e il confronto con l'eseguibile principale mostra una sola differenza: la presenza di due istruzioni di stampa a scopo di diagnostica. Il comportamento è per il resto identico, come verificato eseguendo entrambi gli eseguibili sulla medesima configurazione e confrontando gli output.

Non si tratta quindi di una versione con correzioni, ma di una build di sviluppo. La circostanza è rilevante in senso negativo: esclude che l'archivio contenga una versione emendata del difetto D1.

9.3 Errori rilevati e corretti nel codice sviluppato

I due difetti che seguono non appartengono al sistema studiato ma agli strumenti sviluppati in questo lavoro. La loro esposizione è deliberata e costituisce parte del contributo: entrambi sono stati individuati da verifiche interne — il confronto fra riduzione dichiarata e riduzione effettiva, la validazione di un indice di rischio mai messo alla prova — e in un caso la correzione ha cambiato il valore di una misura riportata in una fase precedente dell'analisi. Un lavoro che misuri l'effetto di una trasformazione su un sistema opaco deve poter esibire il controllo di qualità dei propri strumenti, perché è da lì che provengono gli errori più difficili da vedere.

9.3.1 D5 — Computo errato del budget nella politica sui veti

Questo difetto appartiene al codice sviluppato in questo lavoro, ed è esposto perché la sua individuazione e correzione costituiscono parte del contributo.

Il difetto. La politica `budget:F` limita la frazione di regole portatrici di veto che possono essere rimosse. Il computo della frazione includeva però anche le rimozioni *provate prive di costo* dalla condizione [V2]: regole il cui potere di veto è implicato da una regola sopravvissuta. Tali rimozioni non sacrificano alcun veto, e addebitarle costituiva una doppia contabilizzazione.

Secondo difetto, nella stessa procedura. La condizione [V2] veniva valutata sull'insieme di regole *in ingresso* anziché su quello *sopravvissuto*. Se una strategia successiva rimuove la regola coprente, la garanzia decade. Su una pipeline composita si è misurato che il 43,7 % delle rimozioni presentate come esatte non aveva più alcun copritore sopravvissuto.

Un terzo effetto, nella politica di tutela. La politica `protect`, che deve ripristinare le regole portatrici di veto non altrimenti garantite, ripristinava anche le regole la cui rimozione era dimostrabilmente priva di costo, annullando di fatto il risultato del teorema. Sulla configurazione esaminata la riduzione scendeva allo 0,1 % invece del 17,1 % che la condizione autorizza.

Correzione e verifica. La condizione [V2] è stata riformulata in modo da essere valutata sull'insieme corrente delle regole sopravvissute, e il computo del budget è stato corretto in modo che le rimozioni garantite non lo consumino. Dopo la correzione, `veto-cover` con tutela attiva raggiunge la medesima riduzione della variante senza tutela (17,1 % su Bridges a soglia 9), con output identico alla configurazione di partenza su 8 configurazioni su 8.

Nota 20. Le campagne condotte prima della correzione non sono inficiate: la strategia `veto-cover` era sempre stata impiegata *da sola*, circostanza nella quale l'insieme corrente coincide con quello in ingresso e i difetti non possono manifestarsi. Il difetto era latente, non realizzato.

9.3.2 D6 — Indice di rischio anti-correlato

La politica `budget:F` sceglie quali regole rimuovere ordinandole per un indice di rischio definito come somma dei valori mancanti sulle colonne del lato sinistro. L'indice non era mai stato validato. La validazione, condotta confrontando l'indice con il numero di celle che ciascuna regola può effettivamente servire, ha dato una correlazione di rango $\rho = -0.237$ su 8 961 regole: l'indice non è semplicemente debole, è *anti-correlato*.

La causa è stata individuata con precisione: l'indice somma i valori mancanti sulle colonne del lato *sinistro*, mentre una regola può servire soltanto celle della propria colonna di *destra*. Ne segue che le regole la cui colonna di destra non presenta mai valori mancanti — e che quindi non possono servire alcuna cella — ricevono un indice basso

e vengono rimosse per prime. L'ordinamento promuoveva a bersaglio esattamente le regole innocue.

La correzione adotta un indice calcolato sulle celle nulle reali. A parità di budget, il nuovo ordinamento rimuove 2.5 volte più regole che non servono alcuna cella e distrugge 2.6 volte meno celle servibili. Come riportato nella sezione 7.8, il miglioramento strutturale *non* si traduce in un miglioramento dell'esito, e la circostanza ha portato a concludere che sotto vincolo di budget conti la dimensione del budget e non l'ordinamento.

9.4 Che cosa insegnano questi difetti

Alcune caratteristiche del sistema non costituiscono difetti ma sono rilevanti per l'interpretazione dei risultati, e sono raccolte qui insieme alle conseguenze generali del catalogo.

Codice non raggiungibile. Il disassemblaggio individua procedure che il codice applicativo non invoca mai: una procedura di ricerca locale affetta da un errore di segno, due procedure di riordino delle tuple basate su euristiche, un comparatore di soglie affetto da troncamento nella conversione a intero, e un meccanismo di gestione di regole privilegiate di fatto inerte. La circostanza è stata verificata empiricamente oltre che staticamente.

Osservazione 21. Il comparatore di soglie merita una nota, perché il suo troncamento lo rende non transitivo e avrebbe potuto spiegare l'irregolarità dell'ordine di partenza osservata nella sezione 7.8. La verifica sui sorgenti decompilati mostra però che nessuna classe lo istanzia: è codice morto, e non può essere la causa di alcun comportamento osservato. L'ipotesi è stata scartata su questa base.

L'argomento di selezione dell'euristica è inerte. Il sesto argomento della riga di comando, che dovrebbe selezionare l'euristica di riempimento, non è letto. La circostanza è stata verificata eseguendo il sistema con valori diversi dell'argomento e confrontando gli output, che risultano identici. Tutte le osservazioni di questo lavoro valgono per la sola procedura effettivamente eseguita.

Che cosa avrebbe potuto rilevare prima questi difetti. Nessuno dei sei difetti è stato individuato da un test automatico del materiale d'archivio, e per ciascuno esiste una verifica elementare che lo avrebbe fatto prima. D1 e D4 sono difetti di codice: un test di copertura dei rami avrebbe mostrato che il ramo categorico non è mai eseguito, e un confronto fra gli eseguibili distribuiti avrebbe mostrato le due istruzioni di diagnostica. D2 e D3 sono difetti di dati: un controllo di schema — il numero di campi per riga e la corrispondenza fra intestazione e riferimenti — li avrebbe segnalati al momento della distribuzione, senza bisogno di eseguire nulla. D5 e D6 sono difetti propri del codice sviluppato in questo lavoro, e in entrambi i casi il controllo che li ha rivelati è stato un confronto fra ciò che la procedura *dichiara* di fare e ciò che *fa*: nel primo caso fra riduzione annunciata e riduzione effettiva, nel secondo fra l'indice di rischio e la grandezza che dovrebbe approssimare. La raccomandazione che se ne trae

è di introdurre questi quattro controlli — copertura dei rami, confronto fra artefatti, validazione dello schema, coerenza fra dichiarazione ed effetto — prima di misurare qualunque cosa su un sistema opaco.

Che cosa il catalogo insegna sul progetto di algoritmi a RFD. I sei difetti, letti insieme, indicano tre proprietà ricorrenti. La prima è la distanza fra documentazione e comportamento: un sistema può esporre un parametro che non legge, eseguire una sola delle procedure che dichiara di offrire, e contenere rami che il codice non può raggiungere; per questo l'analisi del capitolo 2 è stata condotta sul bytecode e non sui documenti. La seconda è la concentrazione del costo: la quasi totalità del tempo è in una procedura sola (sezione 2.6), e ciò rende la riduzione dell'insieme di regole l'unica leva temporale di rilievo, ma anche la più delicata, perché quella stessa procedura è quella che il pruning può indebolire. La terza riguarda chi misura: due dei sei difetti appartengono agli strumenti sviluppati in questo lavoro, e la loro individuazione ha richiesto di confrontare in modo sistematico ciò che una strategia *dichiara* di fare con ciò che *fa* (sezione 9.3). Ne segue una raccomandazione operativa per chi impiegasse il sistema su altri dati: verificare l'effettiva equivalenza degli output prima di attribuire un effetto a una strategia di potatura, e considerare le configurazioni in cui il file prodotto coincide con quello di partenza come prove nulle anziché come conferme.

Conclusioni e sviluppi futuri

Sintesi del lavoro

Il lavoro è partito da una domanda operativa: se sia possibile selezionare le regole rilevanti in un insieme di metadati impiegato per l'imputazione di valori mancanti, riducendo il costo computazionale senza degradare la qualità delle imputazioni. Il sistema studiato, RENUVER, riceve le regole come input esterno, e la selezione è stata quindi affrontata nella forma di *preprocessing*: riscrivere il file delle regole, senza toccare l'eseguibile.

La risposta a quella domanda si è articolata in tre passaggi. Il primo è stato la ricostruzione del comportamento del sistema, per il quale non erano disponibili i sorgenti: l'analisi del bytecode ha mostrato che una regola partecipa a due procedure con requisiti opposti — la generazione dei candidati e la verifica dei veti — e che nulla, nel formato del file, distingue i due ruoli. Il secondo passaggio è stato la derivazione delle condizioni esatte di potatura che ne discendono, e del risultato negativo che ne consegue. Il terzo è stato la misurazione sistematica dell'effetto delle strategie sviluppate, condotta su tre famiglie di dataset per un totale di 1 618 esecuzioni in configurazione potata.

Alla domanda iniziale il lavoro risponde su tre piani distinti, che conviene richiamare in chiusura senza ripetere l'analisi: la *riduzione garantita* non può superare il 20,9 % sui dataset esaminati, e il limite è del criterio e non dell'implementazione (capitolo 4); la *qualità* può migliorare, ma soltanto dove la composizione delle imputazioni lascia spazio al miglioramento e dove la copertura non cala, come mostra il confronto fra Bridges e le altre famiglie (capitolo 7); il *tempo* non segue la riduzione, perché dipende da quali regole sopravvivono e non da quante (sezione 7.9). Le tre risposte sono riassunte nella tabella della sezione e discusse nel capitolo 8, dove sono raccolti anche gli esiti delle ipotesi e i limiti dell'evidenza.

Possibili sviluppi

Riscrivere le regole invece di rimuoverle. Il teorema negativo delimita le trasformazioni che *eliminano* regole operando sul solo file. Una trasformazione che riscriva le regole — restringendo o allargando i lati sinistri, correggendo le soglie, fondendo più regole in una — non ricade in quel limite, e la condizione [V2] mostra del resto che una regola può essere eliminata quando il suo contenuto informativo è conservato da un'altra: la stessa conservazione potrebbe essere cercata per via sintattica anziché

per soppressione. È la direzione che più probabilmente consente di superare il tetto del 20,9 % senza rinunciare alla garanzia.

Completezza della condizione di copertura. La condizione [V2] è sufficiente ma non necessaria: stabilire se esistano altre classi di regole il cui veto è implicato da una regola sopravvissuta — e se siano decidibili con l'informazione disponibile — amplierebbe la portata quantitativa del risultato senza cambiarne la natura. Il problema è aperto e riguarda la teoria delle dipendenze rilassate, non l'implementazione.

Verificare il meccanismo del costo. L'ipotesi che il rallentamento osservato dipenda dalla perdita della scorciatoia esatta non è verificata. Isolare il percorso della scorciatoia con la strumentazione e contare quante celle lo impiegano, prima e dopo la potatura, è un esperimento circoscritto che trasformerebbe una spiegazione plausibile in un dato, e che consentirebbe di prevedere il tempo di una strategia oltre che la sua qualità.

Ottimizzare il percorso dei veti. La caratterizzazione del costo indica una direzione precisa e diversa da quella seguita finora. La verifica dei veti esegue 354 809 scansioni del dataset su 71 tuple distinte: la circostanza suggerisce che il verdetto per coppia (regola, tupla) sia memoizzabile. Un *indice* sulle colonne del dataset, o un ordinamento delle regole che incontri presto quelle che violano, sono direzioni alternative. Si tratta di interventi che richiedono la modifica del sistema, ora disponibile come possibilità ma non ancora sperimentata su questo fronte.

Un teorema lato generazione. La condizione [V2] copre il ruolo di veto. Per il ruolo generativo non si dispone di una condizione sufficiente analoga: la strategia `score-argmin` è esatta rispetto alla generazione ma calcola l'argomento del minimo sull'insieme *originale*, e la sua esattezza decade quando altre regole sono state rimosse. Una condizione che garantisca l'invarianza dell'argomento del minimo *dopo* altre rimozioni colmerebbe la lacuna e renderebbe componibili i due criteri.

Misurare l'esercizio effettivo dei veti. La politica di tutela dei veti impiega un indice di rischio che, come si è mostrato, è anti-correlato con la rilevanza. La sua sostituzione con una misura del numero di veti effettivamente esercitati a runtime — ottenibile dalla strumentazione già disponibile — consentirebbe di sostituire la stima con un dato.

Validare il criterio diagnostico su una famiglia indipendente. Il criterio della sezione 7.6 è stato derivato su tre famiglie e non è stato sottoposto a validazione prospettica. Una famiglia con quota esatta intermedia e poche regole costituirebbe il test decisivo.

Il risultato di questo lavoro non è una strategia di potatura che funzioni ovunque: è la determinazione di *che cosa* si può garantire, di *quanto* si può guadagnare abbandonando la garanzia, e del *meccanismo* che decide quale dei due regimi si applichi a un dataset dato. In un ambito in cui la letteratura presenta prevalentemente risultati

positivi su configurazioni favorevoli, la determinazione di un limite dimostrabile e la sua quantificazione hanno un valore che non dipende dal segno del risultato.

–A–
Riproducibilità dei risultati

Questa appendice documenta come i risultati presentati nel capitolo 7 possano essere riprodotti e verificati.

A.1 Struttura del materiale

Il materiale è organizzato nelle seguenti cartelle.

`RENUVER/` l'archivio di partenza: eseguibile, dataset, file di regole, materiale di ground truth e file di risultato delle esecuzioni originali. L'eseguibile non è mai stato modificato; la verifica di integrità è effettuata per checksum. L'archivio include un `README` che documenta la struttura delle cartelle e i parametri della riga di comando: le discrepanze fra quel documento e il comportamento osservato del codice sono discusse nella sezione 2.2 e nel capitolo 9.

`pruning/` gli strumenti sviluppati: il pruner, il sistema di valutazione, gli script di verifica e quelli di orchestrazione delle campagne.

`results/` i file di risultato di tutte le campagne, in formato JSON, con una voce per ogni esecuzione e i relativi indicatori; in `results/analisi/` i dati delle prove che non appartengono a una campagna.

`analisi/` l'analisi del sistema: disassemblaggio commentato, modello dei dati, inventario del codice non raggiungibile.

`report.md` la documentazione estesa del lavoro, con la derivazione completa delle condizioni teoriche e il dettaglio di tutte le campagne.

A.2 Verifica automatica dei risultati

I numeri asseriti nella documentazione sono ricalcolati dai file di risultato da una procedura automatica di verifica, che li confronta con i valori asseriti e segnala ogni divergenza. La procedura copre 67 asserzioni, fra cui le condizioni teoriche derivate dal bytecode, le metriche aggregate di ciascuna campagna e i risultati dei test statistici.

La procedura è stata impiegata durante la stesura: ha individuato e consentito di correggere tre valori non aggiornati, fra cui un risultato il cui segno si invertiva estendendo il campione da 14 a 15 configurazioni, e che era stato inizialmente riportato con il segno errato. La circostanza motiva l'adozione della procedura: un risultato che cambia segno aggiungendo una configurazione su quindici non è robusto, e non deve essere presentato come se lo fosse.

Due procedure ulteriori completano la verifica dei numeri citati nella tesi, ed entrambe operano sui soli file di risultato, senza rieseguire il sistema.

- la prima, `stats_effects.py`, ricalcola i test della famiglia primaria con la medesima ricetta di appaiamento impiegata dalla campagna — chiave (dataset, soglia, ripetizione), baseline e variante entrambe concluse con esito regolare — e produce, oltre ai valori di p già pubblicati, la dimensione dell'effetto, l'intervallo di confidenza bootstrap della differenza mediana e il valore di p corretto con la procedura di Holm (Tabella 6.1);
- la seconda, `make_appendix_tables.py`, genera le tabelle complete dell'Appendice B dai file di risultato; i totali che esse riportano coincidono con i valori discussi nel capitolo 7, e la generazione è quindi anche un controllo di coerenza fra i numeri citati e i dati grezzi.

Le due procedure sono deterministiche: a parità di file di ingresso producono lo stesso output, e il ricampionamento impiegato per gli intervalli di confidenza usa un seme fissato. I dati dell'esperimento di ordinamento, che non appartengono a una campagna ma a un controllo dedicato, sono archiviati con gli altri in `results/analisi/`.

A.3 Verifica di equivalenza della ricompilazione

La caratterizzazione del costo computazionale (sezione 2.6) ha richiesto di ricompilare il sistema, distribuito senza sorgenti. La procedura adottata è la seguente.

1. **Decompilazione** delle classi applicative dell'archivio.
2. **Ricompilazione** per la versione 52 del formato `class`, contro le medesime librerie incorporate nell'archivio originale.
3. **Ricostruzione** dell'archivio sostituendo *soltanto* le classi applicative dentro una copia dell'originale: le restanti classi di libreria restano quelle di partenza.
4. **Verifica di equivalenza** eseguendo l'archivio originale e quello ricostruito sulla medesima configurazione, in cartelle isolate, e confrontando gli output.

L'esito della verifica è riportato nella Tabella A.1: i file di output coincidono byte per byte, e le uniche differenze riguardano i tempi di esecuzione registrati, che variano fisiologicamente fra esecuzioni.

Solo dopo questa verifica il sistema è stato strumentato, aggiungendo contatori e misure di tempo mediante *wrapper* sui metodi di interesse anziché modificandone i corpi, in modo che il flusso di controllo restasse quello originale.

Tabella A.1: Verifica di equivalenza della ricompilazione. Configurazione `bridges_70_1`, soglia 15.

file	dimensione	esito del confronto
`Populated`	1 377 B	**identico** (checksum `682428f09943`)
`Results`	11 522 B	**identico** (checksum `8eca4ca48f2a`)
`Logs`	—	differisce *solo* nelle righe di tempo

A.4 Comandi principali

Le campagne sono orchestrate da script che invocano il sistema di valutazione. A titolo esemplificativo, la valutazione di una strategia su una famiglia di dataset richiede:

```
python3 pruning/benchmark.py \
-datasets <elenco> -runs <elenco> -thresholds <elenco> \
-variants "<nome>=<strategia>:<politica>" \
-out results/<nome>.json
```

Il sistema di valutazione provvede a eseguire la configurazione di riferimento, a produrre il file di regole potato per ciascuna variante, a eseguire il sistema su entrambi e a calcolare le metriche in forma appaiata. I risultati sono memorizzati incrementalmente, cosicché una campagna interrotta può essere ripresa.

A.5 Ambiente di esecuzione

Le campagne sono state condotte su una macchina a otto core, con esecuzioni strettamente sequenziali. La sequenzialità non è una scelta di comodo: la cartella di lavoro del sistema è condivisa fra esecuzioni, ed esecuzioni simultanee sovrascrivono reciprocamente gli output producendo risultati errati senza segnalare alcun errore. La circostanza è stata verificata sperimentalmente.

Le versioni delle librerie impiegate per la ricompilazione sono fissate e documentate, poiché versioni diverse possono produrre differenze nel comportamento della generazione dei numeri casuali e quindi nei risultati.

–B–
Tabelle complete dei risultati

Questa appendice raccoglie le tabelle complete delle campagne, che il capitolo 7 riporta in forma aggregata. Le tabelle sono *generate* dai file di risultato delle campagne dalla procedura `pruning/make_appendix_tables.py`, e i totali che esse riportano coincidono con i valori discussi nel corpo della tesi: la generazione è quindi anche una verifica di coerenza fra i numeri citati e i dati grezzi, con la stessa logica della procedura di verifica automatica descritta nell'appendice sulla riproducibilità.

B.1 Campagna principale su Bridges

La Tabella B.1 riporta le 50 configurazioni della campagna principale, con la medesima ricetta di accoppiamento impiegata nei test: per ogni configurazione la baseline e la variante `A5b`, eseguite sullo stesso dataset, alla stessa soglia, con lo stesso seme. I totali in fondo alla tabella — 69 imputazioni corrette in più e 72 errori in meno — sono quelli discussi nella sezione 7.3.

Tabella B.1: Campagna principale su Bridges, configurazione per configurazione. "RFD" sono le regole in ingresso e in uscita dalla potatura, "rid." la riduzione percentuale, Δ corr. e Δ err. le variazioni rispetto alla medesima configurazione non potata.

dataset	soglia	run	RFD in	RFD out	rid. (%)	Δ corr.	Δ err.
bridges_14	9	1	8961	1366	84,8	+1	-1
bridges_14	9	2	8961	1381	84,6	+0	+0
bridges_14	9	3	8961	1388	84,5	+0	+0
bridges_14	9	4	8961	1368	84,7	+0	+0
bridges_14	9	5	8961	1391	84,5	+0	+0
bridges_28	9	1	8961	1395	84,4	+0	-1
bridges_28	9	2	8961	1406	84,3	+0	+0
bridges_28	9	3	8961	1410	84,3	-2	-1
bridges_28	9	4	8961	1415	84,2	-1	+0
bridges_28	9	5	8961	1427	84,1	-1	+0
bridges_42	9	1	8961	1450	83,8	+2	-1
bridges_42	9	2	8961	1548	82,7	+8	-5
bridges_42	9	3	8961	1545	82,8	+1	-2
bridges_42	9	4	8961	1566	82,5	+1	-2
bridges_42	9	5	8961	1433	84,0	-2	+0
bridges_56	9	1	8961	1718	80,8	+0	-2

dataset	soglia	run	RFD in	RFD out	rid. (%)	Δ corr.	Δ err.
bridges_56	9	2	8961	1854	79,3	-3	+1
bridges_56	9	3	8961	1877	79,1	+1	-1
bridges_56	9	4	8961	1568	82,5	-1	-2
bridges_56	9	5	8961	1628	81,8	+5	-5
bridges_70	9	1	8961	2248	74,9	+5	-5
bridges_70	9	2	8961	2317	74,1	+6	-5
bridges_70	9	3	8961	2362	73,6	+3	-3
bridges_70	9	4	8961	2266	74,7	+5	-3
bridges_70	9	5	8961	2808	68,7	+4	-2
bridges_14	15	1	18844	1853	90,2	+2	-1
bridges_14	15	2	18844	1881	90,0	+0	+1
bridges_14	15	3	18844	1891	90,0	+0	+0
bridges_14	15	4	18844	1849	90,2	+1	+1
bridges_14	15	5	18844	1896	89,9	-1	+0
bridges_28	15	1	18844	1904	89,9	+2	-2
bridges_28	15	2	18844	1918	89,8	+2	-1
bridges_28	15	3	18844	1923	89,8	-3	-1
bridges_28	15	4	18844	1934	89,7	-1	+0
bridges_28	15	5	18844	1961	89,6	-1	+0
bridges_42	15	1	18844	1994	89,4	+3	+0
bridges_42	15	2	18844	2139	88,6	+7	-3
bridges_42	15	3	18844	2158	88,5	+1	+0
bridges_42	15	4	18844	2177	88,4	+0	-2
bridges_42	15	5	18844	1974	89,5	-4	-1
bridges_56	15	1	18844	2438	87,1	+7	-4
bridges_56	15	2	18844	2663	85,9	-3	+1
bridges_56	15	3	18844	2707	85,6	-1	-1
bridges_56	15	4	18844	2200	88,3	-1	-2
bridges_56	15	5	18844	2253	88,0	+7	-4
bridges_70	15	1	18844	3524	81,3	+6	-4
bridges_70	15	2	18844	3472	81,6	+0	+0
bridges_70	15	3	18844	3672	80,5	+4	-4
bridges_70	15	4	18844	3413	81,9	+5	-2
bridges_70	15	5	18844	4502	76,1	+5	-3
				totale		**+69**	**-72**

Tabella B.3: Taratura conservativa della potatura aggressiva ($\alpha = 0{,}99$) sulle 97 configurazioni con baseline disponibile. Riduzione mediana 0,0%, massima 52,9%; variazioni complessive Δ corr. -1, Δ err. -3.

dataset	soglia	run	rid. (%)	Δ corr.	Δ err.
glass_107	3	1	0,0	+0	+0
glass_107	3	2	0,0	+0	+0
glass_107	3	3	0,0	+0	+0
glass_107	3	4	0,0	+0	+0
glass_107	3	5	0,0	+0	+0
glass_22	3	1	24,1	+0	+0
glass_22	3	2	15,0	+0	+0
glass_22	3	3	23,5	+0	+0
glass_22	3	4	32,3	+0	+0
glass_22	3	5	41,1	+0	+0
glass_43	3	1	1,1	+0	+0
glass_43	3	2	0,4	+0	+0
glass_43	3	3	0,9	+0	+0

dataset	soglia	run	rid. (%)	Δ corr.	Δ err.
glass_43	3	4	1,3	+0	+0
glass_43	3	5	1,1	+0	+0
glass_64	3	1	0,0	+0	+0
glass_64	3	2	0,3	+0	+0
glass_64	3	3	0,5	+0	+0
glass_64	3	4	0,0	+0	+0
glass_64	3	5	0,3	+0	+0
glass_86	3	1	0,1	+0	+0
glass_86	3	2	0,0	+0	+0
glass_86	3	3	0,0	+0	+0
glass_86	3	4	0,0	+0	+0
glass_86	3	5	0,1	+0	+0
restaurant_104	3	1	0,0	+0	+0
restaurant_104	3	2	4,0	+0	+0
restaurant_104	3	3	0,0	+0	+0
restaurant_206	3	1	0,0	+0	+0
restaurant_206	3	2	0,0	+0	+0
restaurant_206	3	3	0,0	+0	+0
restaurant_259	3	1	0,0	+0	+0
restaurant_259	3	2	0,0	+0	+0
restaurant_52	3	1	16,0	+0	+0
restaurant_52	3	2	24,0	+0	+0
restaurant_52	3	3	16,0	+0	+0
restaurant_104	6	1	5,6	+0	+0
restaurant_104	6	2	9,7	+0	+0
restaurant_104	6	3	0,0	+0	+0
restaurant_206	6	1	0,0	+0	-1
restaurant_206	6	2	0,0	+0	+0
restaurant_206	6	3	0,0	+0	+0
restaurant_259	6	1	0,0	+0	+0
restaurant_259	6	2	0,0	-1	-1
restaurant_52	6	1	37,9	+0	+0
restaurant_52	6	2	45,2	+0	+0
restaurant_52	6	3	45,2	+0	+0
glass_107	9	1	0,0	+0	+0
glass_107	9	2	0,0	+0	+0
glass_107	9	3	0,0	+0	+0
glass_107	9	4	0,0	+0	+0
glass_107	9	5	0,0	+0	+0
glass_22	9	1	31,7	+0	+0
glass_22	9	2	17,4	+0	+0
glass_22	9	3	24,8	+0	+0
glass_22	9	4	38,7	+0	+0
glass_22	9	5	52,9	+0	+0
glass_43	9	1	0,4	+0	+0
glass_43	9	2	0,2	+0	+0
glass_43	9	3	0,5	+0	+0
glass_43	9	4	0,6	+0	+0
glass_43	9	5	0,4	+0	+0
glass_64	9	1	0,0	+0	+0
glass_64	9	2	0,1	+0	+0
glass_64	9	3	0,2	+0	+0
glass_64	9	4	0,0	+0	+0
glass_64	9	5	0,0	+0	+0
glass_86	9	1	0,0	+0	+0
glass_86	9	2	0,0	+0	+0
glass_86	9	3	0,0	+0	+0
glass_86	9	4	0,0	+0	+0
glass_86	9	5	0,0	+0	+0
glass_107	15	1	0,0	+0	+0
glass_107	15	2	0,0	+0	+0
glass_107	15	3	0,0	+0	+0
glass_107	15	4	0,0	+0	+0
glass_107	15	5	0,0	+0	+0
glass_22	15	1	34,6	+0	+0
glass_22	15	2	20,3	+0	+0
glass_22	15	3	23,2	+0	+0
glass_22	15	4	36,0	+0	-1
glass_22	15	5	51,6	+0	+0

dataset	soglia	run	rid. (%)	Δ corr.	Δ err.
glass_43	15	1	0,3	+0	+0
glass_43	15	2	0,2	+0	+0
glass_43	15	3	0,5	+0	+0
glass_43	15	4	0,5	+0	+0
glass_43	15	5	0,2	+0	+0
glass_64	15	1	0,0	+0	+0
glass_64	15	2	0,1	+0	+0
glass_64	15	3	0,1	+0	+0
glass_64	15	4	0,0	+0	+0
glass_64	15	5	0,0	+0	+0
glass_86	15	1	0,0	+0	+0
glass_86	15	2	0,0	+0	+0
glass_86	15	3	0,0	+0	+0
glass_86	15	4	0,0	+0	+0
glass_86	15	5	0,0	+0	+0

B.2 Studio di ablazione

La Tabella B.2 riporta le venti configurazioni comuni ai quattro bracci dell'ablazione. La tabella rende verificabile la lettura discussa nella sezione 6.8: la somma delle variazioni dei due componenti isolati è pari a +5, mentre la variazione della combinazione è pari a +45, e la differenza (+40) è la sinergia che il sezione 7.5 spiega.

B.3 Taratura conservativa

La Tabella B.3 riporta le 97 configurazioni della campagna con il parametro di aggressività portato al valore conservativo. Vi compare il dato su cui poggia la conclusione della sezione 7.4: la riduzione mediana è nulla, la massima è pari al 52,9 %, e le variazioni complessive sono di una imputazione corretta in meno e di tre errori in meno. È questa la ragione per cui la strategia, in quella taratura, viene descritta come *non dannosa* e non come vantaggiosa: l'effetto sulla qualità non è distinguibile da zero (Tabella 6.1), mentre quello sulla riduzione è concentrato in poche configurazioni.

Tabella B.2: Ablazione di `A5b` sulle 20 configurazioni comuni ai quattro bracci. "priorità" è il solo riordino, "adattiva" la sola potatura adattiva; la somma delle due variazioni è +5, la variazione della combinazione è +45.

dataset	soglia	Δ priorità	Δ adattiva	somma	Δ `A5b`
bridges_14	9	+1	+0	+1	+1
bridges_14	9	+0	+0	+0	+0
bridges_28	9	+1	-1	+0	+0
bridges_28	9	+1	-1	+0	+0
bridges_42	9	+2	-1	+1	+2
bridges_42	9	+9	-1	+8	+8
bridges_56	9	+3	-4	-1	+0
bridges_56	9	+2	-4	-2	-3
bridges_70	9	+7	-2	+5	+5
bridges_70	9	+7	+0	+7	+6
bridges_14	15	+1	+1	+2	+2
bridges_14	15	+0	+0	+0	+0
bridges_28	15	+2	+1	+3	+2
bridges_28	15	+2	+0	+2	+2
bridges_42	15	+2	-1	+1	+3
bridges_42	15	+7	+0	+7	+7
bridges_56	15	+8	+1	+9	+7
bridges_56	15	+1	-5	-4	-3
bridges_70	15	-33	-2	-35	+6
bridges_70	15	+2	-1	+1	+0
	totale	+25	-20	+5	+45

Bibliografia

[1] Bernardo Breve, Loredana Caruccio, Vincenzo Deufemia e Giuseppe Polese. «RENUVER: A Missing Value Imputation Algorithm based on Relaxed Functional Dependencies». In: *Proceedings of the 25th International Conference on Extending Database Technology (EDBT)*. Advances in Database Technology. OpenProceedings.org, 2022, pp. 52–64. DOI: 10.5441/002/edbt.2022.05.

[2] Richard Y. Wang e Diane M. Strong. «Beyond Accuracy: What Data Quality Means to Data Consumers». In: *Journal of Management Information Systems* 12.4 (1996), pp. 5–33. DOI: 10.1080/07421222.1996.11518099.

[3] Carlo Batini e Monica Scannapieco. *Data and Information Quality: Dimensions, Principles and Techniques*. Data-Centric Systems and Applications. Springer, 2016. DOI: 10.1007/978-3-319-24106-7.

[4] Donald B. Rubin. «Inference and Missing Data». In: *Biometrika* 63.3 (1976), pp. 581–592. DOI: 10.1093/biomet/63.3.581.

[5] Roderick J. A. Little e Donald B. Rubin. *Statistical Analysis with Missing Data*. 3ª ed. Wiley, 2019.

[6] Stef van Buuren e Karin Groothuis-Oudshoorn. «mice: Multivariate Imputation by Chained Equations in R». In: *Journal of Statistical Software* 45.3 (2011), pp. 1–67. DOI: 10.18637/jss.v045.i03.

[7] Daniel J. Stekhoven e Peter Bühlmann. «MissForest—non-parametric missing value imputation for mixed-type data». In: *Bioinformatics* 28.1 (2012), pp. 112–118. DOI: 10.1093/bioinformatics/btr597.

[8] Olga Troyanskaya et al. «Missing value estimation methods for DNA microarrays». In: *Bioinformatics* 17.6 (2001), pp. 520–525. DOI: 10.1093/bioinformatics/17.6.520.

[9] Theodoros Rekatsinas, Xu Chu, Ihab F. Ilyas e Christopher Ré. «HoloClean: Holistic Data Repairs with Probabilistic Integrity Constraints». In: *Proceedings of the VLDB Endowment* 10.11 (2017), pp. 1190–1201. DOI: 10.14778/3137628.3137650.

[10] Wenfei Fan e Floris Geerts. *Foundations of Data Quality Management*. Synthesis Lectures on Data Management. Morgan & Claypool, 2012.

[11] Loredana Caruccio, Vincenzo Deufemia e Giuseppe Polese. «Relaxed Functional Dependencies—A Survey of Approaches». In: *IEEE Transactions on Knowledge and Data Engineering* 28.1 (2016), pp. 147–165. DOI: 10.1109/TKDE.2015.2472010.

[12] William W. Armstrong. «Dependency Structures of Data Base Relationships». In: *Information Processing 74: Proceedings of the IFIP Congress*. North-Holland, 1974, pp. 580–583.

[13] David Maier. *The Theory of Relational Databases*. Computer Science Press, 1983.

[14] Vladimir I. Levenshtein. «Binary Codes Capable of Correcting Deletions, Insertions and Reversals». In: *Soviet Physics Doklady* 10 (1966), pp. 707–710.

[15] Immanuel Trummer. «Can Large Language Models Predict Data Correlations from Column Names?» In: *Proceedings of the VLDB Endowment* (2024). Cornell Database Group.

[16] Juliana Freire et al. «Large Language Models for Data Discovery and Integration: Challenges and Opportunities». In: *SIGMOD Record* (2024). New York University.

[17] Frank Wilcoxon. «Individual Comparisons by Ranking Methods». In: *Biometrics Bulletin* 1.6 (1945), pp. 80–83. DOI: 10.2307/3001968.

[18] Sture Holm. «A Simple Sequentially Rejective Multiple Test Procedure». In: *Scandinavian Journal of Statistics* 6.2 (1979), pp. 65–70.